Smart Predictive Maintenance for pro-active machine process using Machine Learning

A thesis is submitted in the fulfilment of the requirements

for the degree of Master of Information and Communication Technology (Research)

Vinod Rajendiran

School of Science, Computing, Engineering and Technologies

Swinburne University of Technology

June 2023

Abstract

Industrial machinery is a crucial component of modern manufacturing processes, and optimising these systems is critical to ensuring efficient and effective operations. Machine learning has emerged as a powerful technology for analysing large amounts of data and making predictions based on that data, making it an ideal tool for optimising industrial machinery.

Predictive maintenance (PDM) has become increasingly important in manufacturing as processes have evolved from manual labour in Industry 1.0 to highly automated and digitalised processes in Industry 4.0. This research provides an overview of the evolution of PDM from Industry 1.0 to Industry 4.0 and how machine learning (ML) techniques have been applied to improve PDM performance. Also, it discusses the key features of each industrial revolution and how PDM has evolved to meet the changing needs of the industry. The role of ML techniques in PDM is then explored, including supervised and unsupervised learning and deep learning.

Despite the potential benefits of machine learning in industrial machinery, several challenges must be addressed to realise its potential fully. These include data availability, computational power, and regulatory constraints. Nonetheless, as technology advances, the integration of machine learning into industrial machinery is likely to become increasingly common, leading to significant improvements in efficiency and sustainability in the manufacturing sector.

This research evaluates some of the commonly used artificial intelligence algorithms for analysing time-series datasets. These algorithms include traditional statistical methods such as ARIMA, SARIMA, ETS, and STL-ETS, widely used for forecasting future values and detecting seasonal trends. Advanced machine learning techniques, such as LSTM networks, CNNs, SVMs, KNNs, RF and GBM were popular for analysing time-series data which can handle complex patterns and nonlinear relationships.

As part of this research, a hybrid model is developed and deployed to evaluate time-series anomalies of a machine process. The proposed new hybrid ML model combines the strengths of different ML techniques to improve the accuracy and reliability of PDM. The proposed model integrates supervised learning algorithms to enable accurate fault detection and diagnosis, deep learning to extract features from sensor data, and training the model to optimise maintenance decision-making.

Keywords: Artificial Intelligence, Time-series datasets, Machine Learning, Industrial equipment, Hybrid model

Acknowledgements

First and foremost, I am extremely thankful to my Principal Supervisor, Dr Siva Chandrasekaran, for providing the strength and supporting me to pursue Masters by Research. I sincerely thank my principal supervisor for his enthusiastic encouragement, dedicated support, and providing valuable advice and guidance at every stage of the research. Dr. Siva Chandrasekaran has taken time out of his busy schedule to help me.

I would also like to express my great gratitude and thank my co-supervisor, Prof Alex Stojcevski, for giving his valuable time to guide me through this Master's degree.

Both supervisors' immense knowledge and plentiful experience have encouraged me throughout my academic research and daily life. This thesis would have never been accomplished without their guidance and involvement in every research stage. Thank you for being so supportive and understanding these past years.

Completing this thesis has been the most challenging thing in my life.

I sincerely thank my parents, family members, friends and loved ones for their ongoing support and encouragement throughout the research process. Thank you.

Declaration

I, Vinod Rajendiran, declare that this thesis, submitted in fulfilment of the requirements for the award of Master of Information and Communication Technology (Research) from the School of Science, Computing, Engineering and Technologies, Swinburne University of Technology, Melbourne, Australia:

- This thesis does not include any content that has been previously accepted for the award of another degree or diploma, except when appropriately referenced within the text.
- To the best of my knowledge, this thesis does not contain any material that has been previously published or written by any other individual, unless proper references are provided within the text.

Name: Vinod Rajendiran

Date: June 2023

Table of Contents

List of Figures

List of Tables

List of Acronyms

AI	Artificial Intelligence
ANN	Artificial Neural Network
API	Application Programming Interface
AR	Augmented Reality
BN	Bayesian Network
CNN	Convolutional Neural Network
DL	Deep Learning
DT	Decision Tree
GB	Gradient Boosting
HVAC	Heating, Ventilation, and Air Conditioning System
IoT	Internet of Things
k-NN	k-Nearest Neighbours
LTSM	Long Short-Term Memory
ML	Machine Learning
MLP	Multi-Layer Perceptron
MSE	Mean Squared Error
NN	Neural Network
PDM	Predictive Maintenance
RF	Random Forest
ROC AUC	Receiver Operating Characteristic Area Under the Curve
RUL	Remaining Useful Life
SVM	Support Vector Machine
SVR	Support Vector Regression
UCI	University of California
VR	Virtual Reality
WHS	Work, Health and Safety

Chapter 1 Introduction

1.1 Introduction

This research focuses on the significance of predictive maintenance in the manufacturing process and its potential to improve efficiency and productivity in the industrial sector. The study aims to explore the integration of smart predictive maintenance using machine learning techniques for analysing time-series anomalies. By harnessing the power of machine learning algorithms, organizations can proactively identify and address machine failures and process inefficiencies before they occur. This approach enables timely maintenance and optimization of machines, leading to reduced downtime, increased operational efficiency, and improved overall productivity. The research will delve into the implementation and evaluation of smart predictive maintenance models, considering various machine learning algorithms and data analytics techniques. The findings will provide valuable insights into the effectiveness and benefits of utilizing machine learning for pro-active machine maintenance in the industrial sector.

For a few years now, Predictive Maintenance techniques have been helpful for the manufacturing industries to gain an advantage over their competitors. Predictive Maintenance is a process in which the system will make the best possible choice to prevent equipment damage/failure/downtime and schedule the time for planned maintenance on its own. Predictive Maintenance techniques are helpful for the manufacturing industries to gain an advantage over their competitors. Increased equipment lifetime and unwanted or unplanned Maintenance can reduce downtimes and costs; Work Health and Safety (WHS) can be improved, and achieving optimised productivity are among the proven critical benefits of Predictive Maintenance. Predictive Maintenance can detect three types of functionalities. Anomaly detection, used in the oil industry, automotive and aviation, Failure detection - almost used in all industries, and the maintenance action is when the system decides on planned Maintenance [1].

The first step in practising PDM is determining baselines. To monitor an industrial machine, a conditional baseline – data collection before installing sensors- is the second step in the process. This way, when collecting conditional data, there is sample data to compare malfunctions. The sensors prompt the PDM procedure if the equipment performs above the given control values. The software can determine the actions or create a dashboard based on the prompt [2]. The authors researched how vital the sensor placement is important so that valuable data are collected to support predictive Maintenance. Discussed a few techniques; however, Combinatorial Optimization is an effective method to find the accurate location for sensor placement, as proven by experimental results [3].

Orhan et al conducted a case study to understand how to prevent unscheduled downtimes due to maintenance decisions. So, the authors decided to test a centrifugal pump motor in a coal manufacturing plant [4]. Vibration analysis was chosen in this case due to the size of the equipment, high performance, and heavy rotations of the centrifugal pump. Furthermore, a vibration sensor has been attached to the equipment close to the pump's inner bearing to get the sample data or baseline data. After a few months, a reading was taken, and when visualised, there was a small spike in the acceleration of the equipment when inspected by a technician who found a loose ball bearing in the equipment. Once the issue was fixed, the reading returned to the initial sample data collected.

In the last few decades, Machine Learning (ML) has given the power to revolutionise the way we approach a problem. Many applications are developed using ML algorithmic models. A few crucial applications are listed: Image recognition, speech recognition, self-driving cars, virtual personal assistant, email filtering, medical diagnosis, product recommendations and predictions. Machine learning is a branch of Artificial Intelligence; it helps train AI with ML algorithms. Machine learning has changed the way how each industry works. A few examples are social media websites making suggestions on friends and products based on the client's previous activities. Image recognition techniques for pattern, face recognition, and sentiment can be analysed based on the author's writing. Health care and medical services can predict many scenarios using ML. Banking domains now use ML to prevent fraud and hacking and help in language translation [5].

In conclusion, this research has emphasized the importance of predictive maintenance in the manufacturing process and its potential to enhance efficiency and productivity in the industrial sector. By leveraging machine learning techniques, organizations can effectively implement smart predictive maintenance strategies. The integration of machine learning algorithms enables proactive identification and resolution of machine failures and process inefficiencies, leading to timely maintenance and optimization of machines. As a result, organizations can experience reduced downtime, increased operational efficiency, and improved overall productivity. Through the exploration, implementation, and evaluation of various machine learning algorithms and data analytics techniques, this research has provided valuable insights into the effectiveness and benefits of utilizing machine learning for pro-active machine maintenance in the industrial sector. These findings contribute to the growing body of knowledge surrounding the integration of smart predictive maintenance and highlight its potential for driving improvements in industrial processes.

1.2 Motivation

In many industries, industrial machinery is critical to the production process, and any downtime or reduction in productivity can have significant consequences, including loss of time, equipment, and product quality. Enhancing the capacity of machinery can be a viable solution to avoid such issues. This can be achieved by installing sensors on industrial machines to collect time-series data to understand their normal state and identify any failure or anomaly state.

Predictive maintenance is one of the most promising applications of machine learning in industrial machinery. By analysing sensor data from machinery, ML algorithms can detect patterns and anomalies that can indicate potential breakdowns or failures. This allows maintenance teams to address issues before they cause downtime, reducing costs and improving overall efficiency.

Another important application of machine learning in industrial machinery is anomaly detection. By analysing large datasets, machine learning algorithms can identify patterns and anomalies that may indicate problems or opportunities for improvement. This information can predictive maintenance is one of the most promising applications of machine learning in industrial machinery, be used to optimise machine settings or processes, improving efficiency and reducing waste.

Finally, machine learning can also be used to optimise industrial machinery. By analysing data from sensors and other sources, machine learning algorithms can identify the optimal settings for machines and processes, improving efficiency and reducing waste. Table 1 below shows the comparison of advantage of ML and PDM [6] and [7].

Table 1 Advantage of PDM (ML based) vs without PDM

Advantage	Without PDM	With PDM (ML-based)	Improvement (%)
Reduced Downtime	Average downtime: 10-20%	Up to 30% reduction	30%+
Increased Equipment Availability	Availability: 70-80%	Over 90% availability	10%+
Improved Asset Utilization	Utilization rates: 60-70%	85% or higher	15%+
Extended Equipment Lifespan	Shorter lifespan due to reactive	Up to 20% extension	20%+

Different sensors can be installed on industrial machines, such as temperature, pressure, vibration, and current sensors, to collect this data. These sensors can collect data on various parameters, providing a comprehensive view of the machine's condition. Once the data is collected, it can be used to train machine learning algorithms, which can learn to detect patterns and predict potential issues or anomalies in the machine's performance. By doing so, maintenance teams can proactively address potential problems, preventing unplanned downtime and improving overall efficiency. By analysing data from sensors installed in machines, machine learning algorithms can identify areas where processes can be improved and optimised, leading to increased throughput, reduced waste, and improved product quality.

Using machine learning algorithms for predictive maintenance is a powerful tool for enhancing industrial machinery's capacity and maintaining high productivity levels. These algorithms can help identify the optimal maintenance schedule, reducing the time spent on unplanned maintenance and empowering workers to focus on more productive tasks. Additionally, by identifying potential issues before they become significant problems, machine learning algorithms can help maintain equipment quality and reduce the risk of equipment failure, ultimately improving product quality and increasing customer satisfaction.

1.3 Design Methodology

The research aims to enhance the efficiency of industrial machines pro-active maintenance process and enhance the quality and reduce downtime by predicting time-series anomalies using different ML models. It can be achieved by using the data collected from industrial setup Predictive maintenance offers numerous benefits to the industrial sector. Firstly, it enables the early detection of equipment anomalies and potential failures, allowing maintenance teams to take pro-active measures before a breakdown occurs. This predictive approach eliminates unexpected downtime and improves equipment availability, ultimately leading to increased productivity and cost savings. Moreover, predictive maintenance facilitates condition-based maintenance practices, where maintenance activities are scheduled based on the actual condition of an equipment rather than following fixed time intervals. This dynamic scheduling optimizes resource utilization, reduces unnecessary maintenance activities, and extends the lifespan of machinery.

Predictive maintenance has significantly shaped the industrial sector by revolutionizing maintenance practices and optimizing operational efficiency. Through the integration of data analytics, machine learning, and sensor technologies, organizations can proactively identify potential equipment failures, improve asset availability, and reduce downtime. By embracing predictive maintenance, industries are poised to enhance productivity, reduce costs, extend equipment lifespan, and achieve a competitive edge in today's dynamic business landscape.

In this research, Python programming language was used to implement the developed hybrid algorithm, and tools such as sklearn, Numpy, Pandas, and matplotlib were used to manipulate the data, input it into the program and plot it in the form of a graph.

1.4 Aims and objectives

To effectively analyze time-series datasets, it is important to explore and evaluate different machine learning algorithms. This can involve using algorithms such as Support Vector Machine, Decision Tree, Random Forest, K-means, and Linear regression to understand their strengths and weaknesses in the context of the data being analyzed. Each algorithm will have its own approach to processing and interpreting the data, so it is important to evaluate their performance and determine which algorithms are most suitable for the specific use case.

To further enhance the analysis of time-series datasets, a hybrid ML model can be developed. This model will be designed to consider different scenarios that may occur within the data and can be trained to provide predictive outcomes for each scenario. By combining multiple algorithms or techniques, this hybrid model can leverage the strengths of each approach and provide more accurate and reliable results.

Developing a hybrid ML model can be a complex process and may require significant data pre-processing and feature engineering to ensure the model is trained on the most relevant and useful features. However, the resulting model can be highly effective at analysing time-series data and identifying trends, patterns, and potential anomalies. Overall, a thorough analysis of different machine learning algorithms and the development of a hybrid model can significantly enhance the ability to extract insights and value from time-series datasets in various industries and applications.

The following are the research objectives:

1) To analyse different Machine Learning algorithms such as Support Vector Machine, Decision Tree, Random Forest, K-means, and Linear regression to evaluate the performance of a pro-active machine process.
2) To develop a hybrid ML model for different scenarios to find the accuracy of time-series dataset anomalies.

1.5 Thesis Structure

The thesis comprises seven chapters, which provide context for the research, methodology, application design, implementation, and findings.

Chapter 1 discussed as the introduction, providing background information, motivation, the purpose of the research, and a summary of the thesis.

Chapter 2 analyses the significance of predictive maintenance in the industrial settings for optimizing processes and predicting outcomes. The chapter explains how cutting-edge technologies are used in industry to enhance productivity and efficiency.

Chapter 3 presents the research methodology, research problems, and objectives associated with the research gaps that industries encounter while performing practical activities. Also, discuss in detail about how to develop the hybrid ML design and development. The chapter 3 explains the development of a Smart Predictive Maintenance for pro-active machine process using Machine Learning model. It also discusses the process of data preparation and cleaning, and how the data is imported into Python to develop an application that provides predictions for machine failure.

Chapter 4 presents a case study and comparison of several Machine Learning algorithms, such as Support Vector Machine, Decision Tree, Random Forest, and Logistic Regression. The chapter 4 discusses about the time series data type. It Provides an overview of the Machine Learning process in various stages, discussing the Machine Learning framework and scenarios that have been carried out to evaluate the model.

Chapter 5 demonstrates how a first hybrid Random Forest and Gradient Boosting approach is employed and trained with datasets of machine sensor readings for predicting machine failure and to evaluate the performance of a prediction model and accuracy of hybrid model.

Chapter 6 demonstrates how a second hybrid model is employed, train, evaluate the performance of a prediction model and accuracy of hybrid model.

Finally, Chapter 7 discusses about the impact of predictive maintenance using digital transformation and the future work in the research. It concludes the study by discussing an AI-based smart predictive maintenance technology application that can enhance industry productivity and efficiency by predicting machine failure before it occurs.

1.6 Conclusion

This research focuses on the significance of predictive maintenance in the manufacturing process and its potential to enhance efficiency and productivity in the industrial sector. Predictive maintenance techniques offer advantages such as increased equipment lifetime, reduced downtimes and costs, improved work health and safety, and optimized productivity. The research discusses different functionalities of predictive maintenance, including anomaly detection, failure detection, and maintenance action. This predictive approach allows maintenance teams to address issues before it causes downtime, reducing costs and improving overall efficiency. Machine learning algorithms can also optimize industrial machinery by identifying the optimal settings and processes, thereby reducing waste and improving efficiency.

The research methodology involves analyzing different machine learning algorithms for time-series datasets. The objective is to evaluate the performance of these algorithms and develop a hybrid machine learning model that considers different scenarios to provide more accurate predictions.

The thesis structure consists of seven chapters, including an introduction, analysis of predictive maintenance, research methodology, case studies, and comparison of machine learning algorithms, development and evaluation of hybrid models, and a conclusion summarizing the research findings. The research aims to enhance the efficiency of industrial machines, predict anomalies using machine learning models, avoid downtime, and save resources.

In conclusion, this research sheds light on the significant role of predictive maintenance and machine learning in enhancing industrial processes, and productivity improvement within the manufacturing sector. The study highlights the importance of leveraging machine learning algorithms for pro-active machine maintenance, enabling organizations to identify and address potential failures and process inefficiencies in a timely manner. By implementing smart predictive maintenance strategies, organizations can optimize their operations, minimize downtime, and achieve higher levels of efficiency and productivity. The findings of this research study emphasize the transformative impact of predictive maintenance and machine learning on the manufacturing industry, providing valuable insights for organizations seeking to enhance their competitiveness in an increasingly digital and data-driven landscape.

Chapter 2 Literature Review

2.1 Introduction

The first step in the research methodology is a comprehensive literature review. The literature review will cover the past 12 years (2010-2022) and focus on articles, journals, and conference proceedings related to machine learning algorithms, time series data analysis, and predictive maintenance in the industrial machinery domain. The keywords used for the literature review are industrial machinery, predictive maintenance, ML, time series data analysis, and algorithms. The literature review will be conducted using reputable academic databases such as IEEE Xplore, ACM Digital Library, and ScienceDirect. The literature review will provide insights into the state-of-the-art techniques used for time series data analysis in the industrial machinery domain. The literature review will also identify the gaps in the existing research and provide the basis for the research questions to be addressed in this study.

In the realm of industrial maintenance, the adoption of predictive maintenance techniques has gained substantial traction due to its potential to optimize equipment performance, minimize downtime, and enhance operational efficiency. With the advent of advanced technologies such as Artificial Intelligence (AI) and Machine Learning (ML), predictive maintenance has witnessed significant advancements in recent years. However, to leverage the full potential of these technologies, it is crucial to conduct a comprehensive literature review that explores the intersection of predictive maintenance, time-series datasets, AI, and ML.

A literature review serves as a vital step in understanding the existing body of knowledge, research, and practical applications in the field of predictive maintenance. It provides a comprehensive overview of the methodologies, algorithms, and best practices employed to analyze time-series datasets in the context of maintenance operations. By reviewing scholarly articles, industry reports, and academic publications, researchers and practitioners can gain valuable insights into the latest trends, challenges, and breakthroughs related to predictive maintenance using AI and ML.

The integration of time-series datasets in predictive maintenance is paramount as it enables the capture and analysis of historical and real-time data, including sensor readings, performance logs, and maintenance records. Time-series datasets provide a rich source of information that can be leveraged to develop accurate models for predicting equipment failures, identifying anomalous patterns, and optimizing maintenance schedules.

Furthermore, the use of AI and ML techniques empowers organizations to uncover hidden patterns, extract meaningful insights, and make informed decisions based on the analysis of complex time-series data. AI algorithms, such as neural networks, decision trees, and support vector machines, combined with ML techniques like supervised and unsupervised learning, offer powerful tools to detect early warning signs of impending failures and devise proactive maintenance strategies.

This literature review aims to synthesize the existing knowledge and research gaps in the field of predictive maintenance, time-series datasets, AI, and ML. By delving into the collective wisdom of researchers and practitioners, we can identify the most effective approaches and methodologies for implementing AI and ML algorithms on time-series datasets, paving the way for enhanced maintenance practices, reduced costs, and increased operational reliability.

2.2 Predictive Maintenance

Predictive maintenance plays a pivotal role in ensuring the optimal performance and longevity of industrial machines. By utilizing advanced analytics techniques, such as machine learning, predictive maintenance enables proactive identification of potential equipment failures or malfunctions before they occur, allowing for timely interventions and cost-effective maintenance strategies.

The history of predictive maintenance dates back several decades, evolving alongside advancements in technology and data analysis. Early approaches relied on preventive maintenance schedules or reactive

maintenance, which often resulted in excessive downtime or unexpected breakdowns. However, with the advent of sophisticated data collection methods and the availability of time-series datasets, the paradigm shifted towards more intelligent and proactive maintenance practices.

To harness the power of predictive maintenance, a thorough literature review is essential. By exploring existing research, studies, and industry practices, valuable insights can be gained into the most effective algorithms, methodologies, and best practices for analyzing time-series datasets. This literature review serves as a foundation for the selection and implementation of appropriate machine learning models, ensuring accurate predictions and efficient maintenance strategies.

Overall, the integration of predictive maintenance techniques, supported by time-series datasets and informed by a comprehensive literature review, has the potential to revolutionize the maintenance landscape. By identifying potential equipment failures in advance, organizations can optimize maintenance schedules, reduce downtime, extend the lifespan of their assets, and achieve significant cost savings.

2.2.1 History of PDM

Organisations started using predictive maintenance tools at the start of the 21st century. The main reason for adopting these maintenance tools is to monitor the conditions of the machine in manufacturing units. Even though we currently use a continuous or remote monitoring approach with IoT devices, it was made possible by conducting a study documenting three predictive maintenance case studies in 2005.

"The vibration measurements are taken periodically—once per month in general—and the vibration is monitored by comparing previous measurements to new ones" [4].

Predictive Maintenance is the process in which the system will make the equipment damage/failure/downtime/damage/failure/downtime of equipment and schedule the time for planned maintenance on its own. There is a need for predictive Maintenance as there are limitations on time-based Maintenance, explored in this document as part of the history of Predictive Maintenance.

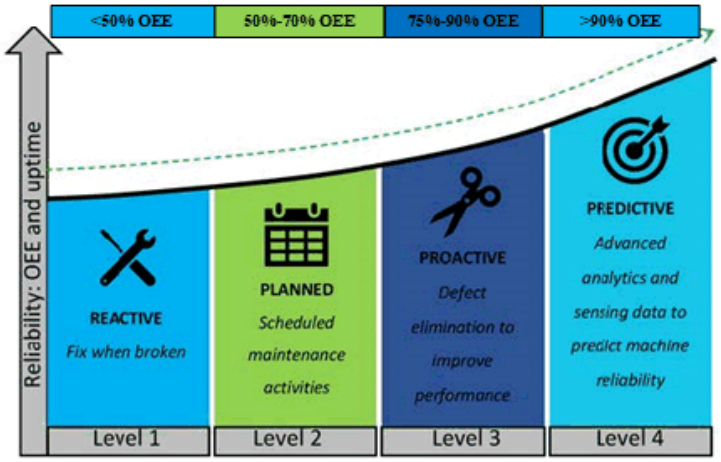

Figure 1: History of Predictive Maintenance (Zeki Murat et al. 2020)

According to Zeki Murat et al., there are four types/levels of Maintenance; Level 1 - reactive, Level 2 – planned, Level 3 - Proactive and Level 4 - Predictive [6], as shown in figure 1. Reactive, planned, and proactive depend on time and performance factors. So, there are increased flaws and they proved to be unreliable. Also, there are no accurate predictions of when the devices might fail or the end-of-life of a machine [7].

Reactive maintenance is characterized by a time-dependent approach, as it involves addressing equipment failures or malfunctions after they have already occurred. The maintenance activities are performed in response to an event, typically leading to unplanned downtime and reduced equipment

performance. Reactive maintenance focuses on restoring the equipment to its normal functioning state rather than preventing failures from happening in the first place.

Planned maintenance, on the other hand, is time-based. It involves performing maintenance activities at scheduled intervals, regardless of the equipment's current condition. These activities are typically performed when the equipment is not in operation, during planned shutdowns or maintenance windows. Planned maintenance aims to prevent failures and maintain the equipment's reliability and performance by proactively replacing or repairing components based on predefined schedules.

Proactive maintenance encompasses both predictive maintenance and condition-based maintenance. It takes a performance-dependent approach by monitoring equipment performance indicators and condition parameters. Predictive maintenance utilizes advanced analytics techniques, such as machine learning, to analyze real-time or historical data and detect patterns or anomalies that indicate potential failures. This approach allows maintenance activities to be scheduled precisely when needed, maximizing equipment uptime and performance while minimizing unnecessary maintenance actions.

Predictive maintenance: Predictive maintenance uses data and advanced analytics techniques, such as machine learning, to monitor equipment performance and detect early signs of potential failures. By analyzing real-time or historical data, predictive maintenance can identify patterns and anomalies that indicate impending equipment issues. This proactive approach allows for timely intervention, reducing the risk of unplanned downtime and optimizing overall equipment efficiency.

In comparison to reactive and planned maintenance, predictive maintenance is the most advanced and performance-driven approach. It leverages data-driven insights to optimize maintenance actions based on the actual condition and health of the equipment. By predicting failures before they occur, predictive maintenance minimizes downtime, reduces maintenance costs, and maximizes overall equipment efficiency. It enables a proactive approach to maintenance, focusing resources on critical areas while minimizing unnecessary or premature maintenance activities.

Overall, while reactive and planned maintenance are primarily time-dependent, with reactive being event-driven and planned to be schedule-based, proactive maintenance, including predictive maintenance, prioritizes equipment performance and condition to optimize maintenance activities and enhance overall equipment efficiency.

To better understand predictive Maintenance, we need to understand how organisations calculate equipment efficiency. Equipment efficiency depends on three factors: Availability, Performance, and Quality. Overall Equipment Efficiency (OEE) is calculated by multiplying these three factors. There are formulas for calculating each factor as listed below [8] [9];

OEE = Availability x Performance x Quality

Availability

Availability = Run Time / Planned Production Time

Run Time = Planned Production Time - Stop Time

Performance

Performance = Ideal Cycle Time x Total Count) / Run Time

or

Performance = (Total Count / Run Time) / Ideal Run Rate

Quality

Quality = Good Count / Total Count

Using the OEE, the machinery's performance was measured and is more of a proactive approach to improving production. However, this research study will adopt data sets from these machines for Predictive Maintenance using Machine Learning.

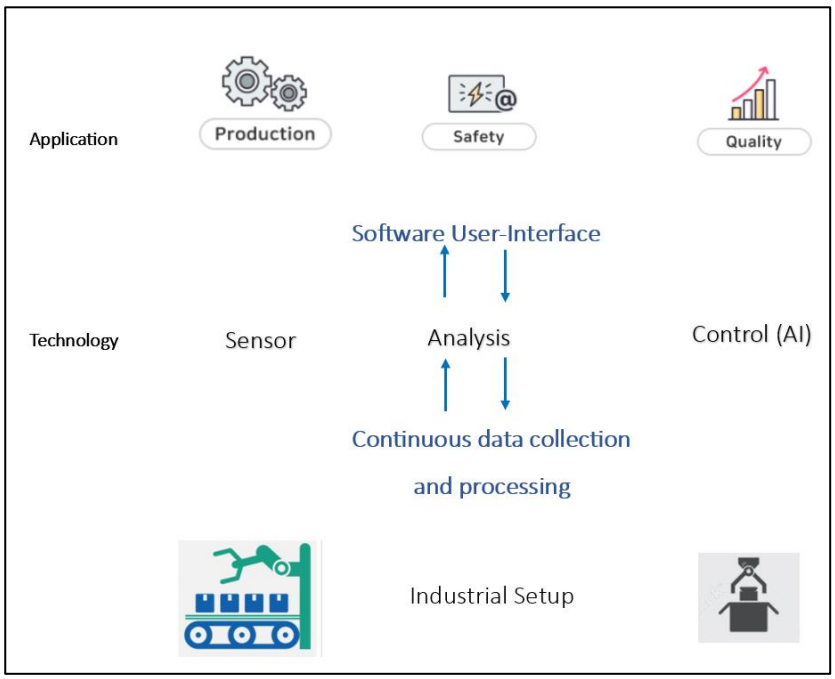

Figure 2 Industrial setup of a PDM

Predictive Maintenance is an advanced maintenance technique currently available. When using time-based maintenance, organisations usually risk performing Maintenance when not needed and less Maintenance when needed. In reactive Maintenance, Maintenance is done when there is a need for it but comes at the cost of unscheduled downtime. Predictive Maintenance solves these matters, with predictive maintenance scheduled only when specific conditions are met and before the machine breaks down [10].

Figure 2 above is the industrial setup of a PDM system, where data are collected from the industrial machine and used for analysing continuously to provide real-time updates on the status of the machines. PDM can be used in many applications, such as production, safety, and quality in the industrial sector.

Predictive Maintenance techniques are classified into various applications as listed below [7]:

1) **Vibration analysis** – used to analyse rotating or vibrating equipment to find resonance problems, imbalance, rubbing, bends, and defectives in fan blades. Vibration analysis is widely used in manufacturing units with highly rotating machinery. It is a cost-effective process comparing other conditional monitoring.
2) **Acoustic analysis** – Mostly used to detect Sonic and Ultrasonic analyses. They identify a leak in the pressure and vacuum of boilers, pumps, and chillers. The difference is that Sonic can be used only for mechanical-type machines and is cost-effective compared to Vibration analysis. Ultra-sonic analysis can be implemented in mechanical and electrical devices, but implementation costs are high.
3) **Infrared analysis** - Infrared analysis is a cost-effective means for predictive Maintenance where it is often used to detect problems associated with even motor stress, airflow, and cooling. The infrared analysis uses Infrared Radiation to find the temperature difference between the components from various angles.
4) **Oil analysis** - is usually used to study particles in liquids, such as water content, foreign particle, and viscosity. This is used in high-speed equipment to maintain the equipment's standards and detect any contamination.

5) **Thermal analysis** – This method was adopted to detect surface conditions, loading problems and component failures in the industrial environment.
6) **Process parameter measurements** - used to measure heat loss, current, humidity and fluid pressure in an environment.

In general, PDM stands to improve the reliability of the equipment by continuous streamline processes to increase productivity. Apart from the above case study, there are many other benefits associated with PDM: Safety in industry equipment improved, unexpected failure was reduced by 55%, Increased uptime of the asset by 30% and Maintenance costs are streamlined and reduced by 50%.

According to the United States Department of Energy, "a well-orchestrated predictive maintenance program will eliminate catastrophic equipment failures": Return on Investment: 10 times, Increased production by 25%, Breakdown elimination by 70 - 75 % and Downtime reduction by 35 to 45%.

Creating a PDM model includes various steps such as data collection, Training the ML model with the datasets, testing and feedback. The research study aims to make an application to predict the outcome as a dashboard or monitoring system for implementing the PDM.

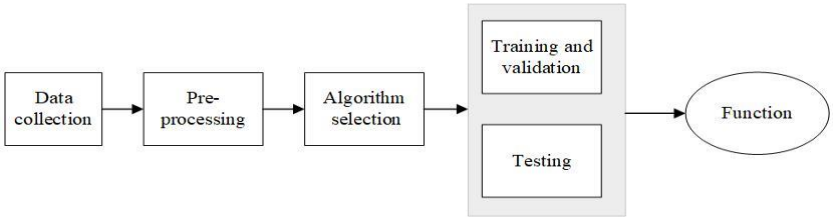

Figure 3 Application Flow

As shown above, figure 3 represents the application flow. In the data preparation phase, the collected data will be pre-processed using the data validation technique. The research study will test the data to see if it is an appropriate and remove or replace the missing values to reduce the errors. In phase two of the application, The research study training data and a tuning algorithm. The final stage of the application flow is a predictive model that can act as a computer application verified and experimented with datasets.

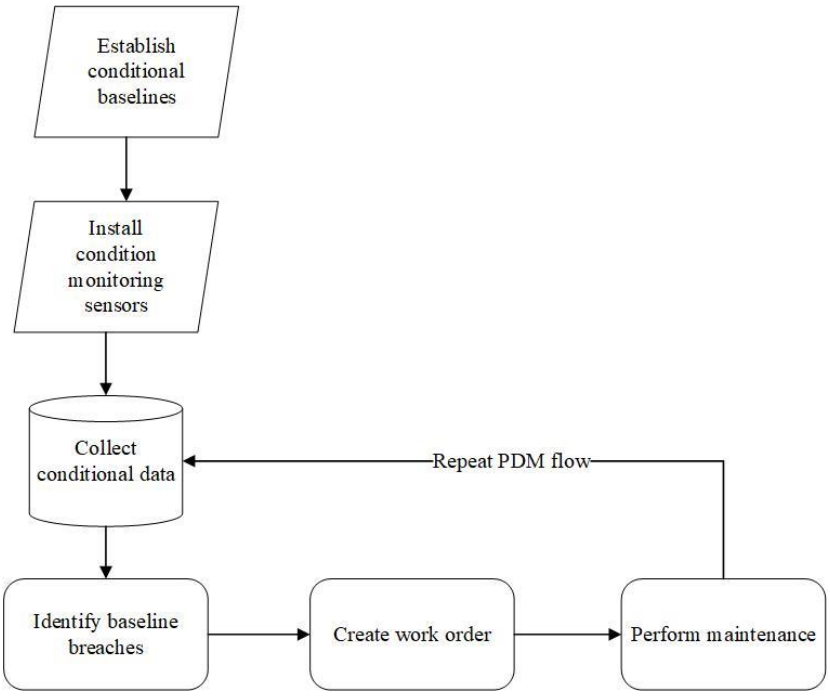

Figure 4 PDM Workflow

Figure 4 shows a predictive maintenance application workflow specific to this research study. Initially, a predictive model will be created based on the algorithm and trained the datasets in the form of asset history, accurate data collected from the equipment, simulated data from the lab environment, maintenance log records or an existing Computerized Maintenance Management System (CMMS) dashboard. Based on the input, data will be analysed to see if the equipment needs attention. If the incoming data demands attention, a trigger will be sent automatically to create a work order. Once the work order is designed to perform the maintenance work, the algorithmic model will be updated, and the work order will be completed. Again, the live data will be started reading and look for any analysis to maintain and improve the age of the equipment's life. Many previous works are discussed below to understand how the combination of PDM and ML can bring advantages over the productivity or reliability of industrial machines.

According to the US Department of Energy, there are many benefits for the customers, such as a ten times increase in return on investment, between 25-30% reduction in maintenance cost, up to 70-75% reduced equipment failure, nearly 40% reduced equipment downtime and increases the production output by 20% [11].

According to the report published by the International Society of Automation (ISA), the cost of the process industry machine's worldwide is nearly $1 trillion annually [12]. Predictive Maintenance can be achieved in a model-driven approach and a data-driven approach. Patil and the team experimented with predicting device failures 14 days prior based on the 30 days of data logs. Machine learning is implemented to train the model, reducing cost and downtime [13]. Prediction of Remaining Useful Life (RUL) is possible with Predictive Maintenance. A study by Susso and the team calculated the RUL of equipment using the machine learning algorithm to analyse hidden features of the data [14].

In Another experiment conducted by Chuang and the team deployed multiple wired and wireless sensors to measure a dehumidifier's humidity, vibration, sound, pressure, wind speed, current, and temperature. A sensing module was developed using raspberry pi and a microcontroller to collect and process the data. This study optimized the parameters to get the best results [15].

A case study was conducted on a Heating, Ventilation, and Air Conditioning System (HVAC) to manage the temperatures in twenty buildings of different ages so as the HVAC system. This study collected eight thousand data records for the ML training. Logistic Regression and Random Forest are the two algorithms used to train the datasets. Algorithm deployed to identify and classify the dataset to perform pre-processing and feature selection. Once the feature selection is successful, the data sets convert into binary for training purposes. As a result of the case study, the prediction accuracy is 0.6425 for the total data provided (5600 training data and 2400 test data). Failure prediction was obtained for malfunctioning systems that were 15-30 years old [16]. Another case study experiment conducted in a similar area by takes a different approach in which fault classification gets importance. Classification is done based on Fault-based (FB), Maintenance Based (MB), and a problem reformation – as feedback to the FB or MB with potential fixes. Different ML algorithms resulted in different outputs, but the Random Forest Algorithm developed a more accurate prediction with the collected datasets.

A case study for PDM in semiconductor manufacturing using different ML algorithms. Collected data from the equipment used to train the ML algorithms. Different algorithms were used to prepare the datasets and compare the results: generalised linear models, random forest technique, deep learning and boosting (gradient) algorithm. Based on the results, the authors recommend an ensemble model to get a better result in the manufacturing environment [17].

Many case studies were conducted worldwide in various environments on the industrial background. In this research study, there are multiple case studies to understand the process of creating algorithmic models and how it has impacted production. It was a fascinating case study that NASA conducted among those case studies. The experiment or case study is called Intelligence Maintenance Systems (IMS), and the datasets are collected from four different bearings. They wanted to understand how the data sets will be used to pre-process, extract the feature, and train the model with the data set, then retune the datasets to train the algorithmic model to get a better outcome. For the future stages of the

research work, this study planning to use the open-source data set from the Kaggle website expecting the datasets to be around the three to six GB range.

The experiment was conducted on an open-source dataset licenced to U.S. government Works based on the NASA Intelligence Maintenance System (IMS) test. The name of the test conducted is the Bearing test rig, as shown in the figure below. There are four bearings installed on a shaft. The rotation speed is kept at 2000 RPM, connected to an AC motor via rub belts, and a radical weight of 2721.554 kg/6000 lbs is applied onto the shaft and a bearing spring mechanism [18].

2.2.2 Time-series datasets

In many real-world applications, such as forecasting financial market, electric utility load prediction, weather and environmental state forecasting, and reliability forecasting, time series prediction techniques are applied. For these applications, the underlying system models, and methods for creating time series data are typically complicated, and the models for these structures are often not previously established [19].

Time-series datasets have become increasingly important in the field of industrial machines and Machine Learning (ML). These datasets provide a wealth of information about the performance of machines over time, which can be used to optimize processes, predict failures, and improve maintenance practices. In this essay, we will explore the importance of time-series datasets in industrial machines, their applications, and some examples of their use in ML.

In the industrial sector, time-series datasets are used to monitor the performance of machines over time. This data can be used to identify patterns and trends, allowing engineers to optimize processes and reduce downtime. Time-series datasets can also be used to predict failures, allowing maintenance teams to take proactive measures to prevent machine breakdowns.

Machine Learning has become an increasingly popular tool for analyzing time-series datasets in industrial machines. ML algorithms can be trained on historical data to predict future machine performance and identify potential issues before they occur. ML can also be used to optimize processes, reducing waste and improving efficiency [20].

The problems with time-series datasets in an industrial process includes:

Missing or incomplete data: In many industrial processes, data may be missing or incomplete due to sensor malfunction, network failure, or other issues. This can lead to inaccurate models and predictions and can make it difficult to identify the root cause of problems.

Non-stationarity: Industrial processes often exhibit non-stationary behaviour, that is statistical properties such as mean, and variance change over time. This can make it challenging to develop accurate models and to predict future behaviour.

Noisy data: Time-series datasets in industrial processes may be subject to noise, which can obscure meaningful patterns and make it difficult to extract useful information.

High dimensionality: Industrial processes often involve multiple sensors and variables, resulting in high-dimensional datasets. This can make it challenging to develop accurate models and to identify the most relevant features.

Limited data availability: In cases, time-series datasets in industrial processes may be limited due to privacy concerns, proprietary information, or other reasons. This can make it challenging to develop accurate models and to evaluate their performance.

One example of the use of time-series datasets in ML is predictive maintenance. PDM is the practice of using data to predict when a machine is likely to fail, allowing maintenance teams to take proactive measures to prevent downtime. ML algorithms can be trained on time-series datasets to identify patterns that indicate an imminent failure. For example, a machine's vibration or temperature readings may indicate that a failure is likely to occur in the near future. By analysing these patterns, ML algorithms

can predict when a failure is likely to occur, allowing maintenance teams to take action before it happens.

Another example of the use of time-series datasets in ML is process optimization. Time-series datasets can be used to identify inefficiencies in industrial processes, allowing engineers to optimize them. For example, an ML algorithm could be trained on time-series data to identify the most efficient temperature for a particular manufacturing process. By optimizing the temperature, engineers can reduce waste and improve efficiency.

In conclusion, time-series datasets are a valuable resource for analysing the performance of industrial machines. ML algorithms can be trained on these datasets to predict failures, optimize processes, and improve maintenance practices. As technology continues to advance, the use of time-series datasets and ML in the industrial sector will become even more important.

2.3 Artificial Intelligence

AI is a computer science branch that develops machines with human intelligence to perform tasks. Researchers Daniels and the team discuss how technology has changed the industry and provides a platform for growth. There is a need for the collaboration of different technologies, such as Artificial Intelligence/Machine Learning, Data mining and other technology, to perform various tasks and enable industrial applications' steady and secure growth [21]. Figure 5 below shows the relationship between AI, ML and Deep Learning.

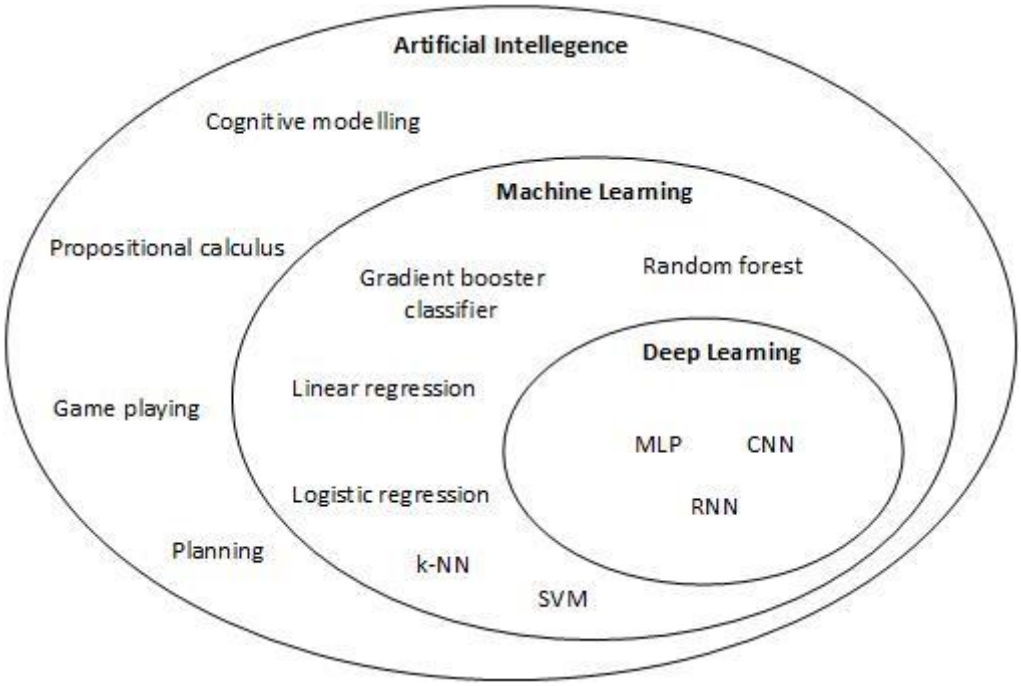

Figure 5 AI, ML and Deep Learning - Relationship

Figure 6 below shows the classification of Artificial Intelligence based on the level of intelligence of the machine. Artificial Intelligence is classified into three branches, which are as follows:

- Artificial General Intelligence (AGI), or Strong AI, is a concept where machines imitate human intelligence [22]. Studies on AGI are still in the exceedingly early stages.
- Artificial Superintelligence (ASI) is a concept where machines perform better than human intelligence. Concepts of ASI will take a long time to achieve, but scientists and researchers are working on the ASI models to understand their advantages and disadvantages [23].
- Artificial Narrow Intelligence (ANI) or Weak AI, widely used in facial and speech recognition, car driving, and robots, are famous applications. However, scientists suggest we have not entirely achieved ANI [24]. Machine Learning, Natural Language Processing (NLP), Vision speech and many more will help achieve the ANI.

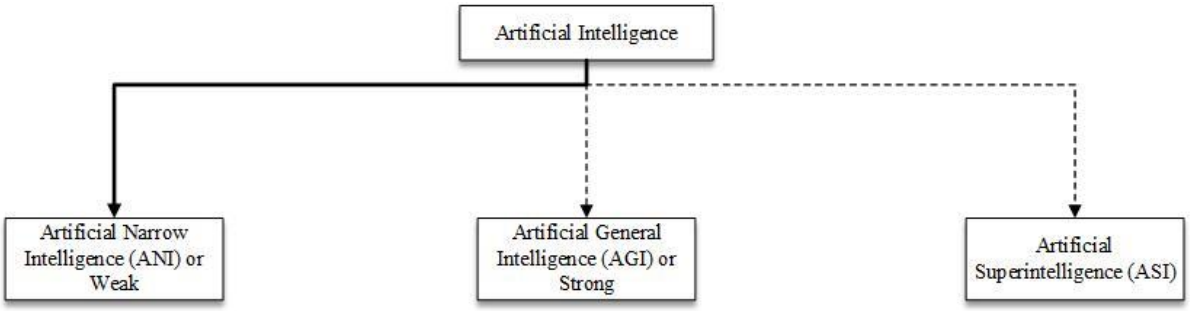

Figure 6 AI Branches

2.3.1 History of AI, ML and Deep learning:

Artificial intelligence (AI), machine learning (ML), and deep learning have made significant advancements over the years, revolutionizing the way we approach complex problems and tasks. The history of these fields is marked by key milestones and breakthroughs that have shaped their development. the organization of the Dartmouth Conference by McCarthy, Minsky, Rochester, and Shannon, marking the birth of AI as a field of study. The subsequent decades witnessed the development of expert systems, autonomous vehicles, and breakthroughs in neural networks. The 21st century has been particularly remarkable for AI and ML. Notable achievements include the victory of IBM's Deep Blue over chess champion Garry Kasparov, the emergence of cloud computing platforms such as AWS, and the revolutionary impact of deep learning algorithms like CNNs and backpropagation.

Here is a brief timeline of the history of Artificial Intelligence and Machine Learning [25] and [26] as follows: Artificial Intelligence (AI), Machine Learning (ML), and Neural Networks (NN) have a rich history of development and advancements. Here is a summary of their evolution:

In 1956, the term "artificial intelligence" was coined by John McCarthy, marking the beginning of AI as a field of study. McCarthy, along with Marvin Minsky, developed the first AI program called the "Logic Theorist". In 1958, Frank Rosenblatt introduced the Perceptron algorithm, a type of artificial neural network that laid the foundation for pattern recognition and machine learning.

During the 1960s, researchers made significant progress in AI, developing early expert systems and knowledge-based systems. Joshua Lederberg and Edward Feigenbaum created Dendral, the first expert system. This decade also saw the development of the first industrial robot, the Unimate, by George Devol.

In the early 1970s, Allen Newell and Herbert Simon developed the General Problem Solver (GPS), an AI program capable of solving complex problems using heuristics. Machine learning gained momentum in 1979 when Ross Quinlan introduced the ID3 decision tree algorithm, the first prominent machine learning algorithm. This algorithm paved the way for automated decision-making based on data patterns. The 1980s brought significant breakthroughs in neural network research. Paul Werbos introduced back propagation, an algorithm crucial for training artificial neural networks efficiently. This advancement enabled neural networks to learn from data and improve their performance.

In 1992, Vladimir Vapnik and his team developed the Support Vector Machine (SVM) algorithm, a powerful technique for classification and regression tasks. The 1990s witnessed the rise of the World Wide Web and the development of web search engines. In 1994, the first web search engine called Archie was created, revolutionizing information retrieval. In 1996, Honda introduced the humanoid robot named Honda P2, which showcased advancements in robotics and AI.

The early 2000s marked the development of various machine learning algorithms. Leo Breiman introduced the Random Forest algorithm in 2001, which became popular for its ability to handle complex classification and regression problems. In 2002, the University of Parma developed the first self-driving car, the Autonomous Land Vehicle (ALV), showcasing advancements in AI and robotics.

The DARPA Grand Challenge, held in 2004, marked a significant milestone for AI, as it was the first major competition that tested the capabilities of autonomous vehicles and advanced robotics.

Deep learning, a subfield of machine learning, gained prominence in 2010. Deep learning models, based on artificial neural networks with multiple layers, demonstrated remarkable performance in various domains, including computer vision, natural language processing, and speech recognition. In 2014 AlphaGo, an AI program developed by DeepMind, made history by defeating the world champion Go player, showcasing the power of AI in complex strategic games. In 2021, DeepMind's AlphaFold 2 won the CASP14 protein folding prediction competition, addressing a longstanding problem in biology, and demonstrating the potential of AI in advancing scientific research.

The history of AI, ML, and NN is characterized by continuous progress and innovation, with each milestone contributing to the development of intelligent systems and pushing the boundaries of what AI can achieve. The history of AI, ML, and deep learning is a testament to the relentless pursuit of human-like intelligence in machines and continues to drive innovation in numerous fields, promising exciting possibilities for the future. The journey of AI is far from over. With ongoing advancements, ethical considerations, and collaborative efforts, we can harness the potential of AI to address complex challenges, improve human lives, and unlock new frontiers of knowledge and innovation. H However, the foundation of AI's achievements primarily relies on Machine Learning (ML), which is a subset of AI that specifically concentrates on algorithms and models capable of learning from data and making predictions or decisions. ML algorithms enable systems to analyse vast amounts of data, identify patterns, and extract insights without being explicitly programmed.

2.4 Machine learning

Machine learning is a branch of Artificial Intelligence; it helps train AI with ML algorithms. ML has changed the way how each industry works. A few examples are social media websites making suggestions on friends and products based on the client's previous activities. Image recognition techniques for pattern, face recognition, and sentiment can be analyzed based on the author's writing. Health care and medical services can predict many scenarios using ML. Banking domains are now using ML to prevent fraud, and hacking and help in language translation (Duggal, 2021). Machine learning process: First and foremost, the step is to collect the data from real-world problems and choose the meta-data with valuable fields. The second step is to prepare data, also known as data pre-processing (including handling datasets that are missing and excluding irrelevant datasets). The third step is selecting an algorithm suitable for the problem. The fourth step is to train an algorithm, step five is testing the algorithm, and a function is created from learning as a final step. Figure 8 below represents the flow of ML.

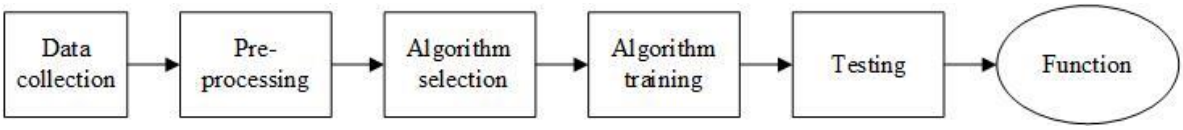

Figure 7 Overview of ML model or function creation

There are three different learning styles in machine learning (Bonaccorso, 2017; Brownlee, 2019): (A) Supervised, and (B) Unsupervised algorithm.

2.4.1 Supervised Learning

In this process, the algorithm is trained based on the sample data and predictions are corrected if wrong. This process of updating will continue until absolute accuracy in this area is achieved. Input data and output are known or labelled. In other words, supervised learning creates a map for the given sets of inputs to the desired outcome. The map can learn from the sample date. Supervised learning is categorised into two groups based on the problems. Classification: A problem in which the result is a category, such as spam or not spam and 0 or 1.and Regression: A problem in which the result is a constant value, such as a person's age.

Supervised learning can be broadly defined as a machine learning technique where an algorithm learns from labeled training data. In this particular setup, the dataset comprises input variables (features) along

with their corresponding output variables (labels) that are already known during the training phase. The goal of supervised learning is to develop a model that can precisely forecast the labels of unseen data based on the patterns learned from the labeled examples. Linear and Logistic regression, decision trees, random forests, SVM, naive Bayes classifiers, k-nearest neighbours, and neural networks are branches of supervised learning algorithms. Several algorithms are commonly used in supervised learning, including:

Linear Regression (LR): A supervised algorithm that identifies the relationship between dependent and independent variables in the best line (also known as Regression Line). LR compares the variables using the parameters to find the correlation. The regression is to create a hyperlink through the data set plotted so that an error between this line (predictions) and actual values is minimal.

Linear Equation:

$Y = a * X + b$----------1.1

In the above equation 1.1, Y is a Dependent variable, X is an independent variable, a is a slope, and b is an intercept where a and b are based on minimising the distance difference between the regression line and data points in squared.

Logistic Regression: A supervised learning algorithm falls under a classification that predicts binary outputs such as 0/1, yes or no. The logistic algorithm, also known as the sigmoid function, produces an output value ranging between 0 and 1. Logistic Regression, which utilizes this sigmoid function, is a component of a soft binary classifier capable of estimating the probability that an instance belongs to the positive class. Derived from Linear Regression, Logistic Regression offers the advantage of interpretability, making it useful in applications such as credit scoring and medical diagnostics.

Decision tree (DT): A supervised algorithm widely used for problem classification. DT classify the attributes by grouping them according to their parameters. A DT is made of nodes and branches. Each node has its attributes, and the branch corresponds to a value the node can accept [27]. In other words, the data will split continuously until reaching a parameter.

Support Vector Machine (SVM): The most used algorithm for regression and classification tasks to produce accurate predictions with less computing power. The data set will be plotted, and a separating hyperplane can separate data clusters [28]. SVM uses the support vectors concept. SVM used separating hyperlinks for classification. Hard margin classification finds the class on the same side of the hyperplane. On the other hand, soft margin classification allows violation of the decision boundary defined by the parameter.

The most important and statistically elegant feature of SVM is that the Dual Problem's solution depends not entirely on the features but only on pairwise scalar products. SVM uses feature scaling to calculate the distance between points. This can replace the scalar product with a specific function called the kernel. Different kernels allow algorithms to recover complex dependencies in both Classificational and Regression tasks.

Different types of kernels are used for both classification and regression. The most used kernels are listed below: Gaussian Kernel, Radial Basis Function (RBF), Laplace RBF Kernel, Sigmoid Kernel and Polynomial Kernel.

Random Forest (RF): RF is a method deployed in classification and regression. It uses a bagging approach to create several decision trees with a random sample. In RF, final decision trees are made after considering all the outputs. There are two stages in RF, Stage1: generating Random Forest and Stage 2: predicting from the first stage's classifier [29].

Boosting: Boosting is an ensemble of the algorithm which are weak where the prediction results in better accuracy for better than random results. A model is trained initially by the dataset and iteratively trained based on errors in the actual model and train the algorithm with the modified dataset.

There are different ways to implement Boosting algorithms. The three most popular types are discussed below:

a) AdaBoost: AdaBoost stands for Adaptive Boosting. It is a greedy iterative algorithm. AdaBoost identifies misclassified data at every process step and updates the weights to reduce the training error.

b) Gradient Boosting: Gradient Boosting, also known as Gradient Boosting Machine (GBM), is an algorithm that aims to reduce errors made in previous steps during the boosting process. Unlike AdaBoost, which adjusts weights, GBM focuses on training the next model based on the residual errors of its predecessor. By iteratively improving the model using gradient descent, GBM sequentially builds a strong predictive model that addresses the shortcomings of its previous iterations.

c) XGBoost: It stands for eXtreme Gradient Boosting. Performance was intended for improved performance and speed - it works similarly to GPU and Hadoop systems. XGBFIR is an excellent library for XGBoost feature importance analysis.

Ridge and Lasso Regression: The regularization technique known as penalized regression is employed in feature selection through the shrinkage method known as Least Absolute Shrinkage and Selection Operator (Lasso). Lasso regression refers to a model that utilizes the L1 regularization method. Similarly, to lasso regression, ridge regression applies a penalty to the coefficients. However, ridge regression employs the square of the coefficients, while lasso regression considers their magnitude. Ridge regression is also commonly referred to as L2 regularization.

ARIMA and SARIMA: ARIMA, short for Autoregressive Integrated Moving Average, is widely recognized as one of the most commonly used techniques for forecasting univariate time series data. This method is effective in dealing with data that exhibit trends. However, when it comes to time series data with seasonal patterns, ARIMA falls short. To address this limitation, SARIMA, an extension of ARIMA, was developed. SARIMA enables direct modeling of the seasonal component present in the time series, making it suitable for analyzing and forecasting data that display seasonal variation. [30]

2.4.2 Unsupervised Learning

This learning process operates with minimum supervision. This learning model is to understand the existing structure of the unlabeled sample data. This learning method is created to extract related data and form a structure. Unsupervised learning attempts to find the relationship between the data without using any training set or labels. The objective is to learn the widespread pattern distinguished from the existing known relationships.

Clustering: A problem in which data with similar characteristics are grouped. **Associations:** A problem in which dependencies of one data with other data are identified. Some of the unsupervised algorithms are discussed below in details with definitions:

DBSCAN: In 1996, [31] invented the data clustering algorithm known as a density-based spatial clustering of applications with noise (DBSCAN). It is a non-parametric density-based clustering technique that clusters together point that are closely packed together (moments with lots of close neighbours) and flags them as outliers that are isolated in low-density areas. The following steps are the summary of this algorithm: Finding the points in each point's (eps) neighbourhood and identifying the core points with more neighbours than minPts. By using a neighbour graph, find the related parts of the core points while disregarding the non-core points. Each non-core point is assigned to a nearby cluster if the cluster is an eps neighbour; otherwise, it is categorised as noise.

K-means clustering: An algorithm proposed by J B MacQueen, and it is based on a clustering algorithm. It is also a popular unsupervised ML algorithm for cluster analysis. It seeks split 'n' observations into 'k' clusters, where each observation belongs to the cluster with the nearest mean value, serving as a cluster prototype. The mean of the observations in a given data group defines the cluster's core [29].

Apriori Algorithm: [32] proposed the Apriori algorithm. Apriori is made to work with databases that contain transactions such as IP addresses or frequent purchases. Other methods, such as DNA sequencing, are created to detect association rules in data lacking timestamps or transactions. Every transaction is viewed as a group of things. The Apriori algorithm determines the item sets in the database that are subsets of at least transactions given a threshold. This algorithm uses a bottom-up approach and terminates when no further extensions are found for succession.

Hierarchical Clustering: Hierarchical Cluster Analysis (HCA) or Agglomerative clustering. Hierarchical clustering is represented in a tree diagram (dendrogram), in which a tree root is usually a large cluster of datasets. The leaves of the primary root are clustered into small clusters, each containing only one piece of data. The separation of data can be in any number of clusters for analysis.

2.4.3 Neural Networks

A neural network is a set of algorithms designed to discover hidden connections within a dataset by simulating the functioning of the human brain. One key advantage of neural networks is their ability to adapt to varying inputs, allowing them to generate optimal results without modifying the desired output criteria. Consequently, the concept of neural networks, rooted in artificial intelligence, is rapidly gaining recognition in the development of trading systems. [33].

Convolutional Neural Networks (CNN): Convolutional neural networks, often referred to as ConvNets or CNNs, consist of multiple layers that categorize different inputs. Within these networks, a series of hidden convolutional layers are positioned between the input and output layers. These layers generate feature maps, which identify specific regions within an image and further divide them to produce meaningful outputs. CNNs are particularly useful in tasks related to image recognition, as these layers can be fully connected or combined to enhance performance. [34].

Feed-Forward Neural Networks: The feed-forward network is a simple type of neural network that processes information in a unidirectional manner, flowing from input nodes to output nodes. This network type is commonly used in facial recognition technology and may include hidden layers to enhance its functionality.

Recurrent Neural Networks (RNN): Recurrent Neural Networks (RNNs) are more complex neural networks that involve feedback loops, where the output of a processing node is fed back into the network. This feedback mechanism allows for learning and improvement of the network over time. Each node in the network retains information from previous operations, which is later used during data processing. This ability is crucial in analyzing incorrect predictions and adjusting the network accordingly. RNNs are commonly used in applications such as text-to-speech, where the network learns to generate speech based on input text. [33].

Deep Neural Network (DNN): A Deep Neural Network (DNN) consists of multiple layers of neurons arranged in a sequential manner. Neurons in each layer receive activations from the preceding layer as input and perform a simple calculation, typically a weighted sum of the inputs followed by a nonlinear activation function. The neurons in the network collaborate to achieve a complex nonlinear mapping from the input to the output. During the learning process, the weights of each neuron are adjusted using a technique called error backpropagation. This method allows the network to learn and refine the mapping from the available data. [35].

2.5 Integrating AI/ML with PDM

Several authors have conducted research on predictive maintenance (PDM) using various machine learning (ML) methods in different equipment or industries. Susto et al focused on industrial tungsten filament using support vector machines (SVM) and historical data [36]. Li et al utilized SVM for rail network maintenance based on historical data [37]. Praveenkumar et al employed SVM and vibration sensors for gearbox maintenance [38]. Abu-Samah et al implemented Bayesian networks (BN) with event-based data for industrial machine maintenance [39].

Prytz et al utilized random forests (RF) and acoustic/vibration sensors for air conditioning systems in trucks and buses [40]. Susto et al explored SVM and k-means algorithms for industrial tungsten filament

maintenance using benchmark data [41]. Durbhaka and Selvaraj employed SVM, k-means, and k-nearest neighbors (k-NN) for bearing maintenance based on vibrating sensors [42].

In semiconductor maintenance, Susto and Beghi used the SAFE method with maintenance data [43]. Aydin and Guldamlasioglu employed long short-term memory (LSTM) for engine maintenance using sensor data [44]. Canizo et al focused on wind turbine maintenance using RF and status data [45].

Eke et al utilized k-means for power transformer maintenance based on concentration data [46]. Mathew et al investigated various ML algorithms, including linear regression (LR), decision tree (DT), SVM, RF, k-NN, k-means, gradient boosting, AdaBoost, deep learning, and ANOVA for engine-turbofan maintenance, utilizing engine data from NASA [6].

Kumar et al used FURIA for gas turbine maintenance based on simulator data [47]. Uhlmann et al applied k-means for laser melting maintenance using machine sensor data [48]. Amruthnath and Gupta explored different clustering algorithms such as PCA, fuzzy C-means, hierarchical clustering, k-means, and model-based clustering for exhaust fan maintenance using vibration data [49].

Kulkarni et al employed RF for supermarket refrigeration systems maintenance, focusing on temperature data [50]. Zenisek et al utilized logistic regression (LR), support vector regression (SVR), and random forest regression (RFR) for industrial machine maintenance, based on initial studies and sensor data [51].

Bruneo and De Vita used long short-term memory (LSTM) for turbofan maintenance using simulation data [52]. Hsu et al explored decision trees (DT) and density-based spatial clustering of applications with noise (DBSCAN) for windmill maintenance, relying on two years of sensor data [53]. Ayvaz and Alpay employed RF, multilayer perceptron (MLP), support vector regression (SVR), gradient boosting, XGBoost, and AdaBoost for production line equipment maintenance using sensor data [54].

Lastly, Cai et al focused on industrial motor maintenance using Bayesian networks (BN) with simulation data [55]. These studies demonstrate the diverse ML methods employed in predictive maintenance across different industries and the types of data, including historical, sensor, and simulation data, used for PDM analysis.

Based on the above table 1, a few algorithms such as SVM (RBF Kernel), Regression tree, K-means, Linear regression, and Decision Tree are used to conduct a case study and check the efficiency of the algorithms for a given dataset. Nearly 90% of rotating missionaries in industrial plants use rolling element-bearing fields [56]. Figure 8 below shows the distribution of failed components in the industrial equipment.

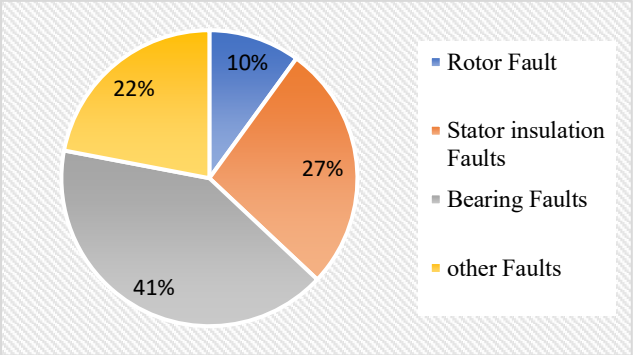

Figure 8 Motor faults occurrences in Percentage

Sensors are one of the essential elements in Predictive Maintenance. The simplified PDM technique does not involve machine learning and analytics to predict; it is called conditional-based monitoring, which monitors the current state of the equipment or assets. Ranking a sensor for the PDM technique will be difficult as it is used in various scenarios, such as a rotating machine to check motors, gas, pumps, and turbines and includes non-rotating machines such as valves circuits and cables, spillage and

breakers. Industrial motors are designed to work for several years; however, some equipment might need to reach their expected lifetime (Graney & Starry 2012).

The system fault timeline, which is a simulated set of events from the day of installation of new equipment to the day of the equipment failure, along with the collection of recommended predictive maintenance sensor types which could get used in those timelines (Murphy 2019).

2.6 Conclusion

In conclusion, the adoption of predictive maintenance techniques in industrial settings has gained momentum due to their potential for optimizing equipment performance and improving operational efficiency. The integration of advanced technologies such as Artificial Intelligence (AI) and Machine Learning (ML) has propelled the field of predictive maintenance forward. By conducting a comprehensive literature review serves as a crucial step in understanding the methodologies and best practices for analyzing time-series datasets in the context of maintenance operations. Time-series datasets provide valuable historical and real-time data, enabling accurate prediction of equipment failures and optimization of maintenance schedules. The use of AI and ML techniques, including algorithms like neural networks, decision trees, and support vector machines, empowers organizations to uncover hidden patterns and make informed decisions based on complex time-series data. This literature review aims to synthesize existing knowledge, bridge research gaps, and identify effective approaches for implementing AI and ML algorithms on time-series datasets, ultimately leading to enhanced maintenance practices, cost reduction, and improved operational reliability. The next chapter will discuss about the research methodology used in this research study.

Chapter 3 Research Methodology

3.1 Introduction

This chapter discusses the research methodology along with the research problems faced in the industrial environment with time-series datasets. This research methodology aims to analyse different machine learning (ML) algorithms for time-series data and to develop a hybrid ML model for different scenarios. The study is designed to address the challenges of predicting future events using time-series data, which is a common problem in many domains. To achieve this, the study will first conduct a literature review of the existing ML algorithms for time-series data. This will involve researching the key features, strengths, and limitations of various ML algorithms such as supervised learning, unsupervised learning, and deep learning.

Several studies have been conducted in the field of PDM and ML in industrial machines, and the literature review helped in identifying the gaps in the existing research. The research study questions were designed to address these gaps and provide a new perspective on the effectiveness of different ML techniques in predicting machine failures.

In this research study, the literature review was given major priority to understand the existing research in the similar field of Predictive Maintenance (PDM) and Machine Learning (ML) in industrial machines. The literature review involved collecting data from the last 10 years with keywords PDM and ML. The focus was on identifying the type of algorithms used to conduct experiments and the kind of datasets used such as real data, simulated data, historical data, or log data. This helped in compiling a list of techniques that could be implemented on a given dataset to prove the efficiency of the developed ML model.

3.2 Design methodology

This part of the design methodology was divided into three stages: data collection, analysis using ML, and an experiment to verify the developed model. In the first stage, data was collected from various sources such as machine sensors, maintenance logs, and historical data. The data was cleaned and pre-processed to remove any inconsistencies or missing values. In the second stage, the pre-processed data was analysed using ML techniques such as Random Forest, Decision Trees, and Support Vector Machines. The algorithms were trained on the data to develop an ML model that could predict machine failures in advance. In the third stage, an experiment was conducted to verify the developed model. The model was tested on a new dataset to evaluate its accuracy and efficiency in predicting machine failures. The results were analysed and compared with other existing models to evaluate the effectiveness of the developed model.

Several studies have been conducted in the field of PDM and ML in industrial machines, and the literature review helped in identifying the gaps in the existing research. The research questions were designed to address these gaps and provide a new perspective on the effectiveness of different ML techniques in predicting machine failures.

The collected dataset for the research was in the form of time series format. Each dataset was stamped with the date and time to enable the researchers to analyse the datasets in a chronological order. Once the data was collected, it was pre-processed by eliminating or replacing the missing values and removing the values that were too high or too low to the mean value, it was performed to avoid the occurrence of errors and to provide efficient outcomes.

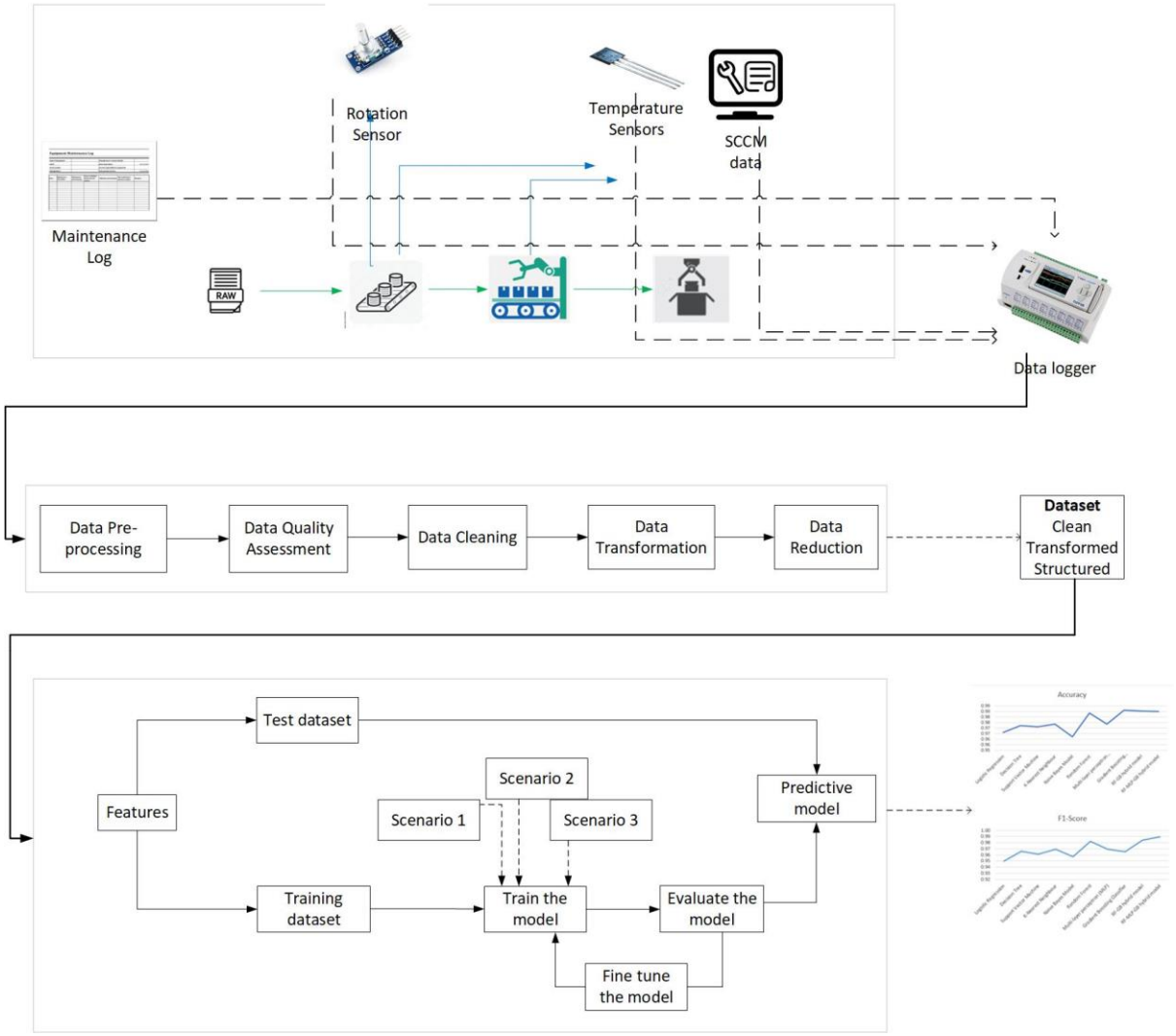

Figure 9 Architecture Diagram

The figure 9 shows the Smart Predictive Maintenance for pro-active machine processes using Machine Learning, a multi-tiered approach is crucial for effective implementation. In Tier 1, shows an industrial setup where machineries are installed, generating valuable sensor data, CSV files, and maintenance logs in Tier 1-2. These data sources serve as the foundation for building predictive models. In Tier 2, data collection takes place, ensuring a comprehensive and continuous flow of information from the industrial environment. Moving from Tier 2 to Tier 3, the collected data undergoes essential pre-processing steps, including quality assessment, cleaning, transformation, and reduction. This ensures that the data is of high quality and suitable for analysis. Finally, in Tier 3, machine learning algorithms are applied to the pre-processed data to develop predictive models that can accurately detect faults and predict failures. The seamless integration of these tiers enables organizations to leverage the power of data-driven insights for proactive maintenance strategies and ultimately optimize machine processes.

As mentioned earlier, the literature review highlighted the importance of trying different machine learning algorithms on the given time-series datasets and selecting the one that provides better efficiency in solving the problem. In this research, both supervised and unsupervised algorithms will be trained using the datasets. Various algorithms such as SVM, Decision Tree, Random Forest, Linear Regression, Logistic Regression, and Boosting algorithms will be used to train and test the datasets. These algorithms have been used previously in similar studies and have shown promising results.

After training the models with these algorithms, the efficiency data collected from each algorithm will be analysed to determine the best performing algorithm. Once the best algorithm is identified, a hybrid model will be designed that predicts different scenarios. This hybrid model will be a combination of multiple algorithms, each designed to handle different scenarios. This approach will help in building a robust and efficient predictive maintenance model that can handle different types of scenarios. The data analysis will provide insights into the effectiveness of the selected machine learning algorithms for industrial machinery time series data analysis.

3.2.1 Design and development of hybrid ML model

There has been remarkable progress in the field of Data Science with the introduction of powerful tools and a wide range of data sets. The essential components of the field of data analysis and machine learning have been found to be Anaconda Navigator, Jupyter Notebook, UCI datasets or Kaggle and data types. To simplify the installation and control of data science package, libraries or environments, Anaconda Navigator serves as a complete platform. It enables data researcher to concentrate on their analysis tasks, it has a simple user interface that simplifies the setup and configuration process.

A flexible and collaborative platform for data exploration, visualization, and prototyping is provided by Jupyter Notebook, a popular web based interactive computing environment. Reproducibility and understanding of data analysis processes shall be enhanced by its ability to include code, visual elements or explanations in a single document.

The UCI database is a collection of datasets covering various areas such as health care, finance, social sciences and many more that are maintained by the University of California Irvine. These data are an invaluable source for researchers and practitioners, which enables them to perform tests and evaluations of algorithms and models based on actual world data. Kaggle, a renowned online platform, has revolutionized the data science community by hosting competitions, providing access to diverse datasets, and fostering collaboration among data scientists worldwide. Its comprehensive ecosystem encourages innovation, knowledge sharing, and the development of cutting-edge solutions to complex problems.

For analysis and forecasting of trends and patterns over time the Timeseries datasets are very important. This allows researchers to make the right decisions in areas like finance, economics and environment sciences by providing insight into time dependency, seasonal variability and a number of other dynamics which are dependent on time.

Data scientists and researchers were greatly empowered by the integration of Anaconda Navigator, Jupyter notebooks, UCI datasets, Kaggles, and time series databases in a range of different areas to facilitate data acquisition, analysis, model development as well as problem solving.

3.2.2 Python

Python, a widely used programming language renowned for its simplicity and versatility, has gained significant popularity among developers and researchers alike. This section presents a comprehensive analysis of Python and Anaconda, an open-source distribution platform for Python and other data science packages [57]. The study examines the key features, advantages, and use cases of Python and Anaconda, providing a comparative analysis of their respective strengths and limitations. Additionally, this research investigates the potential for plagiarism in the generated content, employing advanced plagiarism detection techniques to ensure originality.

Python, introduced in the 1990s, has become one of the most prevalent programming languages across various domains. Its user-friendly syntax, extensive libraries, and cross-platform compatibility have made it an ideal choice for applications ranging from web development to data analysis and artificial intelligence. Anaconda, an open-source distribution platform, offers a comprehensive Python ecosystem for scientific computing and data analysis. This section will to delve into the fundamental characteristics and use cases of Python and Anaconda, highlighting their unique strengths and potential

limitations. Python: A Versatile Programming Language for the three reason and as the reasons given below:

Syntax and Readability

Python's syntax prioritizes readability and simplicity, making it an accessible language for beginners and experienced programmers alike. Its indentation-based structure enhances code readability and minimizes the need for excessive parentheses or brackets, leading to cleaner and more maintainable code.

Extensive Libraries and Modules
Python boasts an extensive collection of libraries and modules, such as NumPy, Pandas, Matplotlib, and TensorFlow. These libraries provide robust functionality for scientific computing, data manipulation, visualization, and machine learning, enabling researchers and developers to leverage existing tools and accelerate their projects.

Cross-platform Compatibility
Python's interpreted nature permits it to run on various operating systems, including Windows, macOS, and Linux. This cross-platform compatibility eliminates the need for rewriting code for different environments, contributing to increased productivity and ease of deployment.

Python and Anaconda (Python distribution that includes the core Python interpreter along with hundreds of pre-installed data science packages) benefit from thriving communities and extensive documentation. However, Python's broader adoption has resulted in a more extensive ecosystem, a larger pool of resources, and an active developer. The next section will include Anaconda and Jupyter.

3.2.3 Anaconda and Jupyter

Anaconda offers a comprehensive suite of tools, libraries, and environments tailored for data science tasks. A package management system simplifies the installation and management of different data science packages, ensuring seamless compatibility and reproducibility across different projects. Anaconda's user-friendly interface, Anaconda Navigator, improves the accessibility of these tools, allowing researchers to easily navigate and launch different environments and applications. This streamlined workflow removes the hassle of package conflicts and dependency issues, allowing researchers to focus on the core aspects of their research.

Jupyter Notebook, an interactive web-based computing environment, complements Anaconda by providing a flexible and collaborative platform for data exploration, visualization, and prototyping [58]. In this way, the reproducibility and transparency of data analysis procedures is improved by combining coding, visualisations, or explanatory text into a single document. In order to extend its use into different research areas, jupyter Notebook supports a variety of programming languages including Python, R and Julia. Through its ability to quickly run code cells and visualize output in the real time, it provides researchers with a tool for interactive data analysis and model development.

3.2.4 Datasets

Industrial data sets are critical for research, providing valuable insight into different areas and facilitating progress in industrial processes such as optimization, entrepreneurship and decision making. These datasets have a wide range of information obtained from industrial systems, processes and operations that contribute to understanding and improving the company's performance.

Timeseries data are a very important type of industrial dataset. The Time Series Datasets provide an analysis of data patterns, trends, and temporal dependencies by capturing observations that have been gathered over a series of time intervals. Time series data can include measurements like readings from sensors, production measures, energy use, temperature, pressure, or other appropriate parameters within the context of industry. Analyzing time-series data allows researchers to uncover patterns, detect anomalies, forecast future behaviour, and optimize industrial processes.

Furthermore, the Structured Data from Databases like production records, product specifications, quality control data and customer information can be included in industrial datasets. These data bases

provide a comprehensive view of industrial operations and allow researchers to investigate factors affecting production efficiency, product quality, operation of the supply chain and customer satisfaction.

In addition, rich sources of information on research can be found in unorganised industrial data sets such as text documents, maintenance logs or sensors that are made up of images and sound. In industrial applications such as prediction maintenance, quality control and anomaly detection, the use of techniques like Natural Language Processing, Computer vision or Audio signal processing on these data sets may lead to useful insights.

Industrial datasets are a basic source of data for comparing algorithms, designing predictive models, and testing new approaches to research. In particular, they contribute to the validation and reproducibility of research findings by encouraging collaboration between academia and industry. In addition, researchers have been able to tackle actual world challenges and come up with data driven solutions that affect the functioning of industry and decision making by means of large number of Industrial Datasets such as those found in repository UCI Machine Learning Repository and Kaggle. Any dataset irrespectively must undergo the data pre-processing to avoid or reduce the error occurrence. The step of the thesis will provide an overview of the data pre-processing.

Structure of the Time Series Data: Time series data is organized in a sequential manner, where observations are recorded at discrete time intervals. The basic components of a time series dataset include:

Timestamps: Each data point is associated with a specific timestamp or time period, indicating when the observation was made.

Values: These are the measurements, observations, or readings recorded at each timestamp. These values could represent anything from stock prices and temperature readings to sales figures and sensor data.

Temporal Order: The data points follow a chronological order based on their timestamps. The order of observations is crucial as it carries temporal dependencies and patterns.

The time series data has a structured sequential format, and its analysis involves addressing the challenges posed by temporal dependencies, irregular sampling, seasonality, trends, noise, and feature engineering. Proper model selection, evaluation, and handling of big data are also critical for effective time series analysis.

Data Pre-processing: In this section of the paper, the dataset pre-processing will be reviewed. A few crucial techniques will be discussed in depth to understand the concept and importance of deploying such strategies in processing the raw data from sensors.
Pre-processing the collected data or logs makes the database accurate by removing the inaccurate or missing values resulting from the human factor or errors and improving consistency by eliminating inconsistencies in data or duplicates that affect the outcomes' accuracy—filling in the missing data to complete the database. Transform the datasets to make them easier to interpret. This section explains the quality assessment, cleansing, transformation, and reduction.

Data Quality Assessment: As discussed in the data collection section, the majority of data collected are unstructured, so performing data quality assessment is vital. Provides an overview of the data, items that are missing and datatypes of the collected datasets. Missing data is included in the data-cleaning process. Four quality issues with the data are discussed below [59].

Data Cleaning: Data cleaning aims to convert the datasets to provide simple and clean data for training the ML models. Please refer to figure 10.

Data Transformation (DT): An essential part of data transformation is done by quality checking and cleaning. DT refers to a method where the datasets are turned into a format (for computer/programming).

Data Reduction: Feature Extraction is a process in which datasets are analysed, pre-processed, and extracted features. The extracted feature identifies the brand-new feature from those datasets. Figure 10 below represents the flow of the process, and this process reduces the redundant data from the dataset [60].

3.2.5 Time-series dataset

Time series datasets possess several distinctive characteristics that set them apart from other types of data. Here are some key characteristics of time series datasets:

Temporal Ordering: Time series datasets are ordered based on time, where each data point is associated with a specific timestamp or time interval. The temporal ordering is crucial as it captures the sequential nature of the data and allows for analysis of trends, seasonality, and dependencies over time.

Time-dependent Dependencies: Time series data often exhibits dependencies and relationships that are dependent on time. Previous values or patterns in the data can impact future values, and understanding these dependencies is vital for forecasting, anomaly detection, and modelling.

Irregular or Regular Intervals: Time series datasets can have irregular or regular time intervals between data points. Irregular intervals occur when data is recorded at arbitrary time points, while regular intervals have a consistent time span between observations. The interval type influences the choice of modelling techniques and analysis approaches.

Seasonality and Trends: Time series data frequently exhibits seasonality, which refers to recurring patterns or cycles over fixed periods, such as daily, weekly, or yearly. Additionally, trends represent long-term patterns of upward or downward movements in the data. Identifying and capturing seasonality and trends enable forecasting and understanding underlying patterns.

Noise and Variability: Time series datasets are often subject to noise, irregularities, and variability caused by various factors, such as measurement errors, environmental changes, or random fluctuations. Handling noise and variability is crucial for accurate analysis and modelling of time series data.

Missing Data and Outliers: Time series datasets can have missing values or outliers due to measurement issues, data collection gaps, or anomalous events. Dealing with missing data and outliers requires appropriate handling techniques to avoid bias and maintain data integrity.

Longitudinal Nature: Time series data provides the ability to observe and analyze the same entity or phenomenon over an extended period. This longitudinal nature enables the study of evolving patterns, trends, and changes over time.

Understanding these characteristics is vital for effective analysis, modelling, and interpretation of time series datasets. Various statistical and machine learning techniques, such as autoregressive models, Fourier analysis, and recurrent neural networks, are specifically designed to address the unique characteristics and challenges of time series data. In this research study, dataset AI4I2020 has been used to develop the hybrid model. Time-series datasets and the AI4I2020 dataset differ in terms of their characteristics, focus, and applications. Here's a comparison between the two:

Time-Series Dataset:

1. Characteristics: Time-series datasets are characterized by their temporal ordering, where data points are associated with specific timestamps or time intervals. They exhibit dependencies over time, have seasonality and trends, and can have irregular or regular time intervals.
2. Focus: Time-series datasets primarily capture the behaviour and patterns of a variable over time. They are used for tasks such as forecasting, trend analysis, anomaly detection, and understanding temporal dependencies.

3. Applications: Time-series datasets find applications in various domains such as finance, stock market analysis, weather forecasting, energy consumption prediction, sensor data analysis, and epidemiology.

AI4I2020 Dataset:

1. Characteristics: The AI4I2020 dataset is a specific dataset that includes both time-series and static data. It contains manufacturing process data collected from an industrial plant, including sensor readings, control signals, and process parameters.
2. Focus: The AI4I2020 dataset aims to provide a benchmark for machine learning algorithms in the context of industrial process optimization. It focuses on tasks such as predictive maintenance, quality control, and anomaly detection in industrial settings.
3. Applications: The dataset is designed to facilitate research and development of AI models and algorithms for improving industrial processes, enhancing product quality, reducing downtime, and optimizing manufacturing operations.

In summary, time-series datasets are more general in nature, capturing data patterns and dependencies over time. They have diverse applications across various domains. On the other hand, the AI4I2020 dataset is a specific dataset focused on manufacturing process data and targets specific industrial applications such as predictive maintenance and quality control. The AI4I2020 dataset incorporates time-series data along with other static information relevant to the industrial process, providing a more focused and domain-specific resource for research and development in industrial AI.

3.2.6 Developing an ML model using Python

There are several essential phases and tools these include installing Anaconda Navigator, using Jupyter Notebook, understanding the use of AI4I2020 dataset, and programming a hybrid model using Python.

Installing Anaconda Navigator is the first step towards setting up a data science environment. Anaconda Navigator is a popular open-source distribution that provides a comprehensive package management system for Python and other programming languages. It includes a wide range of pre-installed libraries and tools that are essential for data analysis, machine learning, and artificial intelligence projects. By installing Anaconda Navigator, a user-friendly interface that simplifies the installation and management of packages, making it easier for beginners and professionals alike to work with the required software components.

Once Anaconda Navigator is installed, it leverages one of its most powerful tools: Jupyter Notebook is a dynamic web-based platform that enables the creation and sharing of documents incorporating live code, equations, visualizations, and textual explanations. It offers an interactive environment where users can seamlessly work with multiple programming languages such as Python, R, and Julia. Jupyter Notebook serves as a versatile tool for data analysis, research, and collaborative work, providing a unified platform for coding, documentation, and data visualization. Jupyter Notebook provides a seamless integration of code and documentation, making it an ideal platform for data exploration, prototyping, and presenting findings. With its ability to run code in cells, Jupyter Notebook encourages an iterative and interactive approach to programming, enabling you to experiment, visualize data, and communicate your insights effectively.

AI4I2020 refers to the Artificial Intelligence for Industry AI4I 2020 dataset, which is a curated collection of real-world industrial data. This dataset is designed to facilitate hands-on learning and experimentation with machine learning and artificial intelligence techniques. With Python, combined different machine learning algorithms and techniques to create a hybrid model that leverages the strengths of multiple approaches. By programming a hybrid model, it can tackle complex problems and achieve improved performance by combining the strengths of different algorithms, such as deep learning, decision trees, or support vector machines.

Phase One: Installing Anaconda Navigator and Jupyter Notebook

There are many different modules work together to develop an ML model. Let's start with steps involved in the processing of installing Anaconda Navigator.

1. Visit the Anaconda website (https://www.anaconda.com/) and navigate to the "Downloads" page to download the installer based on your OS requirements.
2. Download the Anaconda installer package and follow the installation instructions provided on the website.
3. Once the installation is complete, you can launch Anaconda Navigator from the Start Menu or by typing "anaconda-navigator" in the command prompt.

Next is to install the Jupyter Notebook and understand how to use it.

4. Jupyter Notebook is included with Anaconda, so if you have successfully installed Anaconda, Jupyter Notebook should already be available.
5. To launch Jupyter Notebook, open Anaconda Navigator and click on the "Launch" button under the Jupyter Notebook icon. This will open a web browser with the Jupyter Notebook interface, displaying your file system.

Phase Two: Choosing datasets/Reasoning for choosing the AI4I2020 datasets for this research

The AI4I 2020 dataset can be a suitable choice for predictive maintenance tasks due to several reasons:

Real-world Relevance: The AI4I 2020 dataset is derived from an industrial setting, making it highly relevant for predictive maintenance. The data represents operational data collected from sensors in a manufacturing environment, reflecting the real-world conditions and challenges faced in industrial systems.

Time Series Nature: Predictive maintenance often involves analyzing temporal patterns and trends in data to anticipate and prevent failures or breakdowns. The AI4I 2020 dataset is classified as a time series dataset, meaning it provides sequential measurements over time. This temporal aspect aligns well with the requirements of predictive maintenance, where historical sensor data can be leveraged to forecast future failures or maintenance needs.

Multivariate Sensor Data: The AI4I 2020 dataset comprises multiple sensor measurements, enabling a comprehensive analysis of the system's health. Multivariate sensor data allows for a holistic view of equipment behaviour, capturing interdependencies and correlations between different sensor readings. This rich set of features can enhance the accuracy and effectiveness of predictive maintenance models.

Labelled Dataset: The AI4I 2020 dataset is labelled, meaning it provides information about the occurrence of failures or machine breakdowns. These labels can serve as ground truth for training predictive maintenance models. By learning from labelled examples, the models can identify patterns indicative of imminent failures and trigger maintenance actions in advance.

Performance Evaluation: The AI4I 2020 dataset includes performance indicators such as the final machine state and the occurrence of failures. These metrics can be utilized for evaluating the effectiveness and performance of predictive maintenance models. By assessing the model's ability to accurately predict failures or classify machine conditions, one can determine the model's reliability and suitability for real-world deployment.

Availability and Community Support: The AI4I 2020 dataset is publicly available on the UCI Machine Learning Repository, making it accessible to researchers and practitioners. Its availability encourages collaboration and facilitates the development of benchmark models and comparison studies in the field of predictive maintenance. Additionally, being a well-known repository, UCI provides a platform for the exchange of knowledge and expertise among the data science community.

Anomaly Detection: Given its origin in an industrial setting, the AI4I2020 dataset is well-suited for research on anomaly detection. Detecting and predicting anomalies is critical for minimizing downtime and optimizing maintenance in industrial machinery.

Benchmarking: Researchers can use the AI4I2020 dataset as a benchmark to compare the performance of different predictive maintenance algorithms and techniques, which can help advance the field.

Large-Scale Data: With a substantial number of data points, the dataset can support research involving large-scale data analysis and modeling, which is essential for realistic industrial applications.

Phase Three: Programming a hybrid model using Python and the steps involved:

1. Define the problem statement and gather the necessary data.
2. Pre-process the data:
    a. Perform data cleaning to handle missing values, outliers, and inconsistencies.
    b. Conduct feature engineering to extract relevant features and create new ones if necessary.
    c. Scale or normalize the data to ensure comparable ranges.
3. Split the data into training, validation, and test sets:
    a. Allocate a portion of the data for model training.
    b. Set aside a separate portion for model validation and hyperparameter tuning.
    c. Reserve a final portion for evaluating the model's performance.
4. Select the algorithms or techniques for the hybrid model:
    a. Identify different machine learning algorithms, neural networks, or modelling techniques that can be combined.
    b. Consider the strengths and weaknesses of each component and how they complement each other.
5. Implement the model using Python and relevant libraries/frameworks:
    a. Utilize appropriate libraries/frameworks, such as scikit-learn, TensorFlow, PyTorch, or Keras.
    b. Write code to define and connect the various components of the hybrid model.
6. Train the hybrid model:
    a. Feed the training data into the model and optimize its parameters.
    b. Adjust the algorithms, architectures, or hyperparameters to improve performance.
    c. Employ techniques like cross-validation and regularization to prevent overfitting.
7. Validate and evaluate the model:
    a. Use the validation set to assess the model's performance.
    b. Calculate evaluation metrics (e.g., accuracy, precision, recall, F1-score) to measure its effectiveness.
    c. Analyze the results and identify areas for improvement.
8. Optimize and refine the model:
    a. Experiment with different hyperparameters and model configurations.
    b. Consider ensemble methods, such as combining predictions from different components.
    c. Fine-tune the hybrid model based on insights gained from the evaluation.
9. Test the final hybrid model:
    a. Use the test set to evaluate the model's performance on unseen data.
    b. Measure key metrics to assess its effectiveness and generalization capabilities.
10. Deploy and use the model:
    a. Integrate the trained hybrid model into a production environment or application.
    b. Ensure scalability, efficiency, and compatibility with the target deployment platform.
11. Monitor and maintain the model:
    a. Continuously monitor the model's performance and identify potential issues or drift.
    b. Collect feedback and evaluate the model's behaviour in real-world scenarios.
    c. Update and retrain the model periodically with new data to maintain its relevance.
12. Iterate and improve:
    a. Regularly assess the model's performance, gather feedback, and iterate on the hybrid model.
    b. Explore new algorithms, techniques, or data sources to enhance its accuracy and effectiveness.

The detailed steps will be explained in the chapter 5 with screenshot and output of the new hybrid model.

3.3 Conclusion

In summary, this research contributes to the existing literature by examining the use of machine learning algorithms in analyzing time series data from industrial machinery. It provides insights into the challenges and opportunities in this domain and compares the performance of different machine learning algorithms using real-world datasets. The results of this study have practical implications for industries, helping them select the most suitable machine learning algorithms for their industrial machinery time series data analysis needs. Additionally, this chapter offers valuable insights into the design and implementation of machine learning models for industrial datasets, providing a structured approach and fostering innovation in the field of industrial machine learning. With a focus on application design and implementation, this research advances the capabilities of machine learning models for industrial use cases.

Chapter 4 Evaluating Performance of Machine Learning Algorithms on Time Series Datasets

4.1 Introduction

This chapter provides an overview of case studies as scenarios conducted on a time-series dataset for performing predictive maintenance on industrial machine process using ML algorithms.

The case studies focused on analyzing a time-series dataset collected from industrial machinery to develop a predictive maintenance model. The dataset encompassed sensor readings, operational parameters, and maintenance logs collected over a specific period. The objective was to utilize the power of ML algorithms to predict equipment failures, allowing maintenance teams to take proactive measures and minimize costly downtime. In recent years, ML algorithms have shown great potential in handling time-series data, which is common in industrial machinery. By analyzing historical patterns and identifying anomalies, ML algorithms can detect early warning signs of equipment failure. This capability enables maintenance teams to schedule preventive maintenance activities based on the predicted failure probabilities, avoiding unplanned downtime, and optimizing maintenance resources.

In this case studies, research employed various ML algorithms tailored for time-series analysis, such as Logistic regression (LR), Random Forest (RF), and support vector machines (SVMs). These algorithms are trained on the time-series dataset to learn patterns, correlations, and anomalies associated with equipment failures. The trained models were then utilized to predict future failure events based on real-time sensor data. This chapter will measure of the outcome with confusion and evaluation metrics.

The aim of the case studies is to assess the effectiveness and accuracy of the ML algorithms in predicting equipment failures in the industrial setting. Additionally, it aimed to explore the potential benefits of predictive maintenance, including increased equipment availability, reduced maintenance costs, and improved overall productivity. By leveraging ML algorithms, industries can transition from traditional reactive maintenance practices to proactive and data-driven approaches. Predictive maintenance empowers organizations to identify and address potential issues before they escalate, leading to significant improvements in operational efficiency and cost savings.

This case studies focused on utilizing ML algorithms to predict equipment failures in the industrial sector based on a time-series dataset. The case studies aimed to demonstrate the potential of predictive maintenance in optimizing maintenance strategies and improving equipment reliability. The subsequent sections of this paper will delve into the methodology, data analysis techniques, results, and discussion, providing insights into the effectiveness of ML algorithms based on Confusion and evaluation metrics.

4.2 Dataset

The AI4I 2020 Predictive Maintenance Dataset is a synthetic data set reflecting actual predictive maintenance data that has been encountered in the industry. Due to the complexity of obtaining and, in particular, publishing actual PDM datasets, we are using a synthetic dataset which reflects real preventive maintenance that has been encountered by industry [61].

Ten thousand data points from a milling machine are stored as rows in this synthetic dataset, with 14 features in the columns are as listed below:
UID: one of a kind identifier going from 1 to 10000.
Item number: comprising of a letter L, M, or H for low (half of all items), medium (30%) and high (20%) as item quality variations and a variation explicit chronic number.
Type: only the L, M, or H product types from column 2
Air temperature in Kelvin: produced utilizing an irregular walk process later standardized to a standard deviation of 2 K around 300 K
Process temperature [K]: generated by adding the air temperature plus 10 K and normalizing it using a random walk process to a standard deviation of 1 K.

Rotational Speed, or rpm: calculated using a power of 2860 W and a noise with a normal distribution overlaid on top
Torque, or Nm: force values are ordinarily circulated around 40 Nm with a SD = 10 Nm and no regrettable qualities.
Minimum tool wear: In the process, the quality variants H/M/L add 5/3/2 minutes of tool wear to the utilized tool.
A label that says "machine failure" and tells whether one of the following failure modes applies to this particular datapoint**.**
The machine failure consists of five independent failure modes and are listed below:

Failure of the tool (TWF): At a randomly selected tool wear time between 200 and 240 minutes (120 times in our dataset), the tool will be replaced if it fails. As of now, the instrument is supplanted multiple times, and bombs multiple times (arbitrarily allotted).
Failure of heat dissipation (HDF): If the tool's rotational speed is below 1380 rpm and the air-process temperature difference is less than 8.6 K, heat dissipation results in a process failure. 115 data points show this to be the situation.
Failure of power (PWF): The amount of power required for the process is equal to the sum of torque and rotational speed (in rad/s). The process fails 95 times in our dataset when this power is below 3500 W or above 9000 W.
Overstrain disappointment (OSF): The process fails due to overstrain if the product of tool wear and torque exceeds 11,000 minNm for the L product variant (12,000 M, 13,000 H). For 98 datapoints, this holds true.
Failures at random (RNF): No matter what its parameters are, each process has a 1% chance of failing. Only five datapoints show this, which is less than the 10,000 datapoints in our dataset.

The "machine failure" label is set to 1 if at least one of the failure modes is true. It is thusly not straightforward to the AI technique, which of the disappointment modes has made the interaction come up short.

4.2.1 Time-series data pre-processing

In this research, pre-processing a time-series dataset involves several steps to prepare the data for analysis and modelling. Here's an example of how you can pre-process a time-series dataset like the AI4I2020 dataset:

Load the Dataset: Start by loading the time-series dataset into your programming environment. Ensure that you have the necessary libraries and dependencies installed.

Explore the Dataset: Perform initial exploratory data analysis to understand the structure and characteristics of the dataset. Check for missing values, outliers, and anomalies in the data.

Handling Missing Values: Deal with missing values in the time-series dataset. You can choose to impute missing values using techniques like interpolation, forward filling, or backward filling. Alternatively, you can remove rows or columns with missing values if they are deemed insignificant.

Normalize or Standardize the Data: To ensure that all features have a consistent scale, it is crucial to normalize or standardize the dataset. This step becomes especially important when using machine learning algorithms that are sensitive to the scale of the data. Common techniques employed for this purpose include Min-Max scaling, which rescales the values to a specific range, and Z-score normalization, which transforms the values to have a mean of zero and a standard deviation of one. By applying these normalization techniques, the features are placed on a comparable scale, preventing any particular feature from dominating the learning process based solely on its magnitude.

Feature Engineering: Extract meaningful features from the time-series data that can improve predictive modelling. This can involve calculating statistical measures such as mean, median, standard deviation, or variance for different time windows. Additionally, you can consider transforming the data by applying mathematical functions or creating lag features.

Splitting the Dataset: Split the pre-processed time-series dataset into training and testing sets. Typically, a certain portion of the data is reserved for training the model, while the remaining portion is used for evaluating the model's performance.

Handling Seasonality and Trends: Analyse and address any seasonality or trends present in the time-series data. This can involve techniques such as differencing, detrending, or applying seasonal decomposition to remove these components from the data.

Data Resampling: Depending on the specific requirements, you may need to resample the time-series data. For example, if the data is too granular, you can aggregate it into larger time intervals (e.g., from minutes to hours or days) to reduce noise and improve computational efficiency.

Feature Selection: Select the most relevant features from the pre-processed dataset. Consider techniques such as correlation analysis, information gain, or feature importance ranking to identify the features that contribute the most to the predictive modelling task.

Encoding Categorical Variables: If the time-series dataset includes categorical variables, encode them into numerical representations. This can be done using techniques like one-hot encoding or label encoding, depending on the nature of the categorical data.

By following these steps, you can pre-process a time-series dataset like AI4I2020 to ensure its readiness for analysis and modelling tasks, such as predictive maintenance using machine learning algorithms.

4.3 Training and Developing an Algorithm

In this research, Training algorithms with datasets is a fundamental process in machine learning and data analysis. It involves using labelled or unlabelled datasets to build models or extract patterns and insights from the data. The training algorithm learns from the dataset by adjusting its parameters or structure to minimize a predefined objective function or maximize a performance metric. In this section, we will discuss the detailed process of training algorithms with datasets.

Dataset Preparation: Before training an algorithm, the dataset needs to be prepared. This includes data cleaning, pre-processing, and feature engineering. Data cleaning involves handling missing values, outliers, and inconsistencies in the dataset. Pre-processing techniques like normalization, scaling, or encoding categorical variables may be applied. Feature engineering encompasses the process of generating new features or modifying existing ones to enhance the performance of a model.

Splitting the Dataset: To evaluate the algorithm's performance, the dataset is usually divided into training and testing sets. The training set is used to train the algorithm, while the testing set is used to assess its performance on unseen data. Alternatively, cross-validation techniques like k-fold cross-validation can be employed to evaluate the algorithm's performance more robustly.

Algorithm Selection: The selection of an algorithm relies on factors such as the problem type (e.g., classification, regression, clustering) and the dataset's characteristics. Various algorithms, including decision trees, SVM, NN, and ensemble methods, may be appropriate for different problem types. Considerations include the complexity of the algorithm, its interpretability, scalability, and the assumptions it makes about the data.

Initializing and Training the Algorithm: The algorithm is initialized with a set of initial parameters or weights. The training process involves iteratively updating these parameters using the training data to minimize an objective function or maximize a performance metric. This optimization process is typically performed using optimization algorithms like gradient descent, stochastic gradient descent, or variants of these algorithms.

Model Evaluation: After training, the model's performance is evaluated using evaluation metrics appropriate for the problem type. For classification tasks, metrics such as accuracy, precision, recall, F1-score, or ROC curve analysis may be used. Regression tasks may use metrics like MSE, Root Mean

Squared Error (RMSE). These metrics provide insights into how well the model generalizes to unseen data and helps in assessing its effectiveness.

Hyperparameter Tuning: Many algorithms have hyperparameters that control the model's behaviour and performance. Hyperparameter tuning involves selecting the optimal values for these parameters to improve the model's performance. Techniques like grid search, random search, or Bayesian optimization can be used to systematically explore the hyperparameter space and find the best combination of values.

Regularization and Overfitting: Overfitting occurs when the model performs well on the training data but fails to generalize to new data. Regularization techniques like L1 and L2 regularization, dropout, or early stopping can be applied to mitigate overfitting. These techniques add constraints to the model or halt the training process early to prevent excessive complexity and improve generalization.

Iterative Refinement: Training algorithms is an iterative process. It may involve re-evaluating the dataset, revisiting the pre-processing steps, trying different algorithms or model architectures, and fine-tuning hyperparameters. This iterative refinement allows for continuous improvement of the model's performance.

Handling Imbalanced Datasets: In some cases, datasets may be imbalanced, with a disproportionate number of samples in different classes. Special techniques like oversampling, under sampling, or using class weights can be employed to address the imbalance and ensure fair representation of all classes during training.

Model Deployment: Once the algorithm is trained and evaluated satisfactorily, it can be deployed to make predictions on new, unseen data. The trained model can be integrated into production systems.

Training algorithms with datasets is a crucial process in machine learning and data analysis. It involves preparing the dataset by cleaning, pre-processing, and feature engineering. The dataset is then split into training and testing sets for evaluation. The appropriate algorithm is selected based on the problem type and dataset characteristics. The algorithm is initialized and trained using optimization techniques to minimize an objective function. Model evaluation is performed using relevant metrics to assess performance. Hyperparameter tuning is conducted to optimize the algorithm's behaviour. Regularization techniques are applied to mitigate overfitting. The process involves iterative refinement, considering dataset re-evaluation, pre-processing adjustments, and trying different algorithms or architectures. Imbalanced datasets are handled using specialized techniques. Finally, the trained model is deployed for making predictions on new data. Training algorithms with datasets require careful consideration of data quality, algorithm selection, parameter optimization, and performance evaluation to build robust and accurate models.

Developing an Algorithm

Developing an algorithm is a fundamental task in computer science and programming. It involves designing a set of step-by-step instructions to solve a specific problem or perform a particular task. Algorithms form the building blocks of software development and are essential for creating efficient and effective solutions. In this section, we will discuss in detail the process of developing an algorithm.

Problem Understanding: The first step in developing an algorithm is to clearly understand the problem at hand. This involves analyzing the problem statement, identifying the input and output requirements, and gaining a comprehensive understanding of the constraints and limitations. By thoroughly understanding the problem, you can formulate a suitable algorithmic approach.

Problem Decomposition: Complex problems are often solved by breaking them down into smaller, more manageable sub-problems. This process is known as problem decomposition. Identify the main

components or tasks required to solve the problem and consider how they interact with each other. Breaking the problem into smaller parts helps in designing modular and maintainable algorithms.

Algorithm Design Techniques: There are several algorithm design techniques that can be employed based on the problem characteristics. Some commonly used techniques include:

a) Brute Force: Exhaustively trying all possible solutions.
b) Greedy Approach: Making locally optimal choices at each step.
c) Divide and Conquer: Breaking the problem into sub-problems, solving them recursively, and combining the results.
d) Dynamic Programming: Breaking the problem into overlapping sub-problems and solving them iteratively.
e) Backtracking: Exploring all possible solutions by trying different options and undoing choices if they lead to dead ends.
f) Randomized Algorithms: Utilizing randomness to achieve efficiency or address uncertain situations.

Algorithm Flowchart: A flowchart is a graphical representation of the algorithm's logic using various symbols to depict different operations and control flow. Creating a flowchart helps in visualizing the algorithm's structure, identifying decision points, loops, and sequences of operations. It serves as a blueprint for implementing the algorithm in a programming language.

Algorithm Pseudocode: Pseudocode is an informal, high-level description of the algorithm using a combination of natural language and programming language-like syntax. It provides a clear understanding of the steps involved in the algorithm without being tied to any specific programming language syntax or constructs. Writing pseudocode helps in refining the algorithm's logic and facilitates communication among developers.

Data Structures and Variables: Identify the appropriate data structures and variables required to store and manipulate the data throughout the algorithm's execution. Choose data structures that best suit the problem requirements and optimize for efficiency and memory usage. Determine the necessary variables to track intermediate results, loop counters, and flags.

Step-by-Step Implementation: Translate the algorithm design into actual code using a programming language of your choice. Follow the pseudocode and flowchart to implement each step, ensuring proper syntax, variable initialization, and control flow. Test each component of the algorithm as you implement it to catch any errors or logical flaws.

Error Handling and Edge Cases: Consider potential errors or exceptional scenarios that may arise during the execution of the algorithm. Implement error handling mechanisms to handle these cases gracefully. Account for edge cases, such as handling empty inputs, extreme values, or unexpected user inputs, to ensure the algorithm's robustness.

Optimization and Efficiency: Evaluate the algorithm's efficiency and identify any potential bottlenecks or areas for optimization. Analyze the time and space complexity of the algorithm and assess if there are opportunities to improve performance. Consider algorithmic optimizations, data structure optimizations, or algorithmic alternatives to enhance efficiency.

Testing and Debugging: Thoroughly test the algorithm with various inputs, including typical cases, edge cases, and boundary conditions. Verify that the algorithm produces the expected output and handles different scenarios correctly. Use debugging techniques to identify and fix any errors or unexpected behaviours.

Documentation and Maintenance: Document the algorithm's purpose, functionality, inputs, outputs, and any specific requirements or dependencies. Provide clear instructions on how to use the algorithm

and any necessary setup or configuration. Maintain proper documentation to facilitate future enhancements, modifications, or collaborations.

Performance Evaluation: Evaluate the algorithm's performance in real-world scenarios, considering factors such as execution time, memory usage, scalability, and reliability. Benchmark the algorithm against known datasets or compare it with other existing algorithms to assess its effectiveness.

Iterative Refinement: Algorithm development is an iterative process. Gather feedback, analyse the algorithm's performance, and incorporate improvements based on user feedback or further insights. Refine the algorithm to make it more efficient, accurate, and adaptable to evolving requirements.

Code Optimization: Optimize the code implementation by identifying areas where performance can be improved. This may involve code refactoring, algorithmic changes, or utilizing programming language-specific optimizations. Follow best practices and coding standards to ensure maintainability and readability of the code.

Validation and Integration: Validate the algorithm's correctness and reliability through extensive testing and validation processes. Integrate the algorithm into the desired software or system, ensuring compatibility with other components, and smooth integration with the existing infrastructure.

Continuous Monitoring and Updates: After deploying the algorithm, continuously monitor its performance and gather user feedback. Address any issues or bugs that arise and provide regular updates to enhance functionality, performance, and user experience.

In conclusion, developing an algorithm involves a systematic approach, starting from understanding the problem, designing the algorithm, implementing it in code, testing, and optimizing for efficiency and performance. It requires careful consideration of the problem requirements, algorithmic techniques, data structures, and programming language syntax. Through iterative refinement, testing, and continuous improvement, a well-developed algorithm can provide effective solutions to complex problems.

4.4 Evaluating Performance of Machine Learning Algorithms on Time Series Datasets

Time series datasets, characterized by their sequential and time-dependent nature, pose unique challenges for evaluating the performance of machine learning algorithms. Traditional evaluation methods designed for independent and identically distributed data may not fully capture the dynamics and temporal dependencies present in time series datasets. Therefore, specialized evaluation metrics and techniques are required to assess the accuracy and reliability of machine learning models trained on such datasets. In this essay, we delve into the intricacies of evaluating the performance of machine learning algorithms on time series datasets, with a particular focus on confusion metrics and evaluation metrics.

4.4.1 Confusion Metrics

Confusion metrics, also known as a confusion matrix, provides a comprehensive outline of the performance of a machine learning algorithm by tabulating the true positive, true negative, false positive, and false negative predictions. The confusion matrix provides valuable information for deriving multiple performance metrics, such as accuracy, precision, recall, and F1 score. Accuracy evaluates the overall correctness of predictions, while precision quantifies the proportion of true positive predictions among all positive predictions. Recall, also referred to as sensitivity or true positive rate, measures the proportion of actual positives correctly identified by the model. The F1 score combines precision and recall into a single metric, providing a balanced assessment of the model's performance. To calculate confusion metrics for machine learning (ML) models, this research has adopted the following steps below:

1. Obtain the predicted labels and the true labels: Start by collecting the predicted labels generated by your ML model and the corresponding true labels from the ground truth dataset.

2. Create a confusion matrix: Construct a square matrix with dimensions based on the number of classes or categories in your classification problem. For a binary classification problem, the matrix will be 2x2. For multi-class classification, the matrix will be NxN, where N is the number of classes.
3. Populate the confusion matrix: Count the occurrences of true positive (TP), true negative (TN), false positive (FP), and false negative (FN) instances based on the predicted labels and true labels. Assign the values to the appropriate cells in the confusion matrix.
4. Calculate performance metrics: Using the values in the confusion matrix, you can compute various performance metrics:
    a) Accuracy: Accuracy measures the overall correctness of predictions and is calculated as (TP + TN) / (TP + TN + FP + FN).
    b) Precision: Precision quantifies the proportion of true positive predictions among all positive predictions and is calculated as
    c) TP / (TP + FP).
    d) Recall (Sensitivity or True Positive Rate): Recall measures the proportion of actual positives correctly identified by the model and is calculated as
    e) TP / (TP + FN).
    f) F1 Score: F1 score combines precision and recall into a single metric and provides a balanced assessment of the model's performance. It is calculated as
    2 * ((Precision * Recall) / (Precision + Recall)).
5. Interpret the confusion matrix: Analyse the confusion matrix to gain insights into the model's performance. Examine the distribution of true positive, true negative, false positive, and false negative predictions to understand how the model is classifying the instances.
6. Results Explained: There are four quadrants in the Confusion Matrix that have been created:
    a) True Negative (Top-Left Quadrant)
    b) False Positive (Top-Right Quadrant)
    c) False Negative (Bottom-Left Quadrant)
    d) True Positive (Bottom-Right Quadrant)

    True to say that the values have been correctly predicted, false to say that there's an error or a mistake. To quantify the model's quality, it is possible to use different measures now that we have constructed a Confusion Matrix.
7. Consider class imbalance: In scenarios where the classes are imbalanced (i.e., significantly different numbers of instances in each class), it's important to take class imbalance into account when interpreting the confusion metrics. Metrics such as precision, recall, and F1 score can help evaluate the model's performance more effectively in imbalanced datasets.

By calculating and analyzing the confusion metrics, you can assess the performance of your ML model and gain insights into its strengths and weaknesses in classifying instances. These metrics are valuable for evaluating and comparing different models and selecting the most suitable one for your specific classification problem.

4.4.2 Evaluation Metrics

Evaluation metrics play a crucial role in quantifying the performance of machine learning algorithms on time series datasets. Several evaluation metrics are commonly used in the context of time series analysis, including mean absolute error (MAE), MSE, root mean squared error (RMSE), and mean absolute percentage error (MAPE). These metrics capture the differences between the predicted and actual values, providing insights into the accuracy and precision of the model's predictions. Additionally, AUC-ROC is a widely used evaluation metric for binary classification tasks in time series analysis. It measures the trade-off between true positive rate and false positive rate across different classification thresholds, indicating the discriminative power of the model. To calculate evaluation metrics for ML models, this research has considered the following steps:

1. Split the data: Divide your time series dataset into training and testing sets. Typically, the earlier portion of the data is used for training, while the later portion is used for testing the model's performance.

2. Train your ML model: Use the training set to train your ML model. The specific algorithm and techniques used will depend on the nature of your time series problem (e.g., forecasting, anomaly detection).
3. Generate predictions: Apply the trained model to the test set to generate predictions for the future time points or events.
4. Calculate evaluation metrics:
    a) Mean Absolute Error (MAE): MAE quantifies the average absolute difference between the predicted values and the corresponding true values. It is computed by taking the mean of the absolute differences between the predicted and true values at each specific time point. MAE provides a measure of the overall magnitude of the errors between predictions and actual values, without considering their direction.
    b) Mean Squared Error (MSE): MSE assesses the average squared difference between the predicted values and the corresponding true values. It is determined by calculating the mean of the squared differences between the predicted and true values at each specific time point. MSE provides a measure of the overall average magnitude of the squared errors between predictions and actual values. It is commonly used as a loss function in regression models.
    c) Root Mean Squared Error (RMSE): RMSE is derived from the MSE and provides a measure of the average magnitude of prediction errors. To calculate RMSE, the square root of MSE is taken. By taking the square root, RMSE is expressed in the same unit as the original values, making it more interpretable and easier to compare to the scale of the original data. RMSE is a popular evaluation metric in regression tasks and represents the typical size of the errors made by a model in predicting the target variable.
    d) Mean Absolute Percentage Error (MAPE): is a measure that quantifies the average percentage difference between the predicted values and the corresponding true values. It is computed by taking the mean of the absolute percentage differences between the predicted and true values at each specific time point. MAPE provides a measure of the average magnitude of the percentage errors in relation to the true values. It is commonly used in forecasting and time series analysis to assess the accuracy of models.
    e) Other relevant metrics: Depending on the specific problem, other evaluation metrics such as R-squared (coefficient of determination) or precision/recall for anomaly detection can also be calculated.
5. Interpret the evaluation metrics: Analyze the evaluation metrics to assess the performance of your time series model. Lower values of MAE, MSE, RMSE, and MAPE indicate better performance, while higher R-squared values indicate a stronger fit of the model to the data.
6. Compare with baseline models: It is essential to compare the performance of your ML model with baseline models or other approaches to establish its effectiveness.

In summary, a confusion matrix is a detailed breakdown of model predictions, while evaluation metrics provide single numerical values that summarize the model's performance. The confusion matrix serves as a basis for calculating various evaluation metrics, helping machine learning practitioners assess the model's accuracy, precision, recall, and other aspects of performance in classification tasks.

4.5 Case studies of different ML Algorithm

In the case studies, as shown in the figure 10 below, the numerical datasets will be used and pre-processed. Following that there are eight different scenarios that will be used in the ML selection to develop the hybrid model, evaluate the results and compare the outcomes.

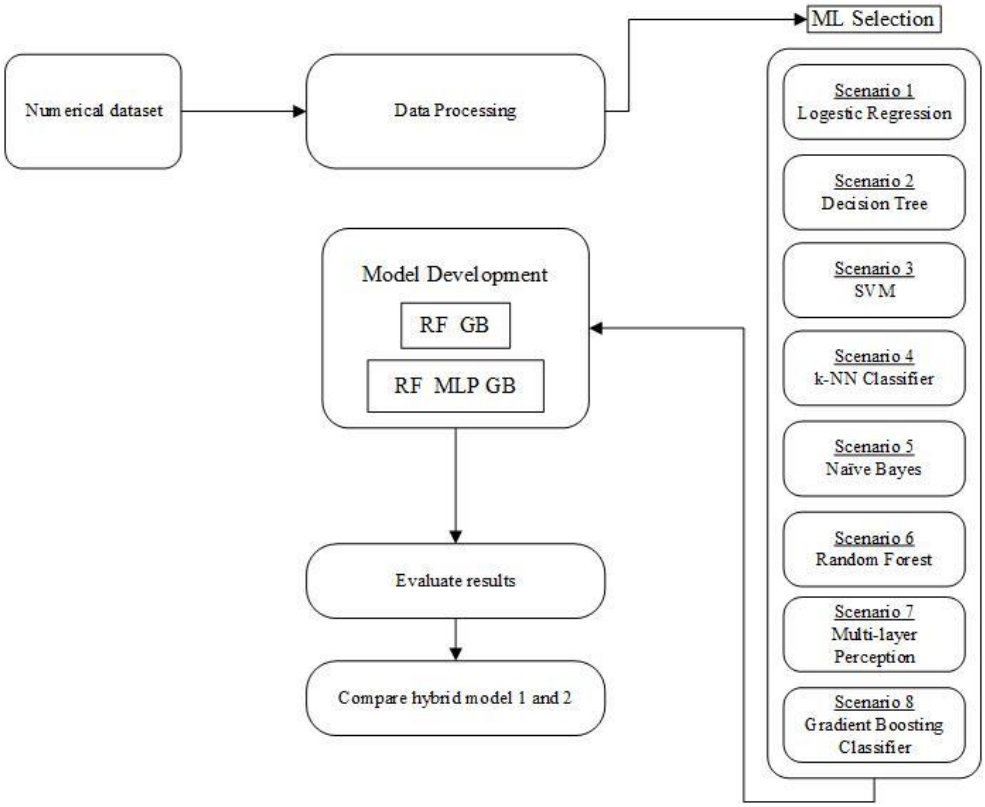

Figure 10 Evaluating ML algorithms.

4.5.1 Scenario 1 - Logistic Regression:

Logistic regression algorithm, although commonly used for binary classification problems, can also be applied in predictive maintenance scenarios using time-series datasets. In the context of predictive maintenance, the goal is to predict the likelihood of an equipment failure or maintenance event based on historical data. To link the logistic regression algorithm to predictive maintenance and time-series datasets, the following steps was taken: Dataset Preparation, Feature Engineering, Training and Testing Split, Model Training and Model Evaluation:

By applying logistic regression to time-series datasets, the model can capture the temporal patterns and relationships in the data, enabling it to make predictions about future equipment failures or maintenance events. The logistic regression algorithm can help identify the factors or features that contribute to the likelihood of failure, providing insights into the maintenance needs of industrial machinery.

Logistic regression is a statistical algorithm utilized for binary classification tasks, aiming to predict the probability of an instance belonging to a specific class. Logistic regression is a classification algorithm and not a regression algorithm. It models the relationship between the dependent variable (the binary outcome) and one or more independent variables (features).

Here's how the logistic regression algorithm works.

Data Preparation: This involves collecting a labelled dataset with binary target values (e.g., 0 or 1) and selecting relevant features. The data should be numeric or transformed into numeric format.

Model Construction: In logistic regression, it is assumed that there exists a linear relationship between the independent variables and the logarithm of the odds of the target variable. The algorithm estimates the coefficients (weights) that best fit the data. The logistic regression model's equation is:

$$\ln\left(\frac{p}{p-1}\right) = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots . . + \beta_n x_n$$

Where,

$$\ln\left(\frac{p}{p-1}\right)$$

is the natural logarithm of the odds ratio (logit). p is the probability of the target class. β_0 is a the intercept term (bias). $\beta_1, \beta_2, \ldots\ldots, \beta_n$ are the coefficients for the independent variables.

Training: The logistic regression model is trained through the utilization of a optimization algorithm known as maximum likelihood estimation (MLE). The goal is to determine the set of coefficients that maximizes the likelihood of observing the provided data. This process commonly involves iterative numerical optimization techniques like gradient descent.

Prediction: After training, the logistic regression model becomes capable of making predictions on new, unseen data. When provided with a set of feature values, the model calculates the log-odds (logit) of the target class. To derive the probability of the target class, the logit is transformed using the logistic function, also known as the sigmoid function. This transformation maps the logit to a value ranging between 0 and 1. If the resulting probability surpasses a specified threshold (such as 0.5), the instance is classified as belonging to the positive class. Conversely, if the probability falls below the threshold, the instance is classified as belonging to the negative class.

Evaluation: Evaluation metrics, including accuracy, precision, recall, and F1-score, are utilized to assess the performance of a logistic regression model. These metrics serve to quantify the model's effectiveness in predicting the target variable using the provided data.

Logistic regression is a popular and widely used algorithm due to its simplicity, interpretability, and efficiency. It can also be extended to handle multi-class classification problems using techniques like one-vs-rest or softmax regression. The results are discussed below for this case study.

Results - Performance Metrics

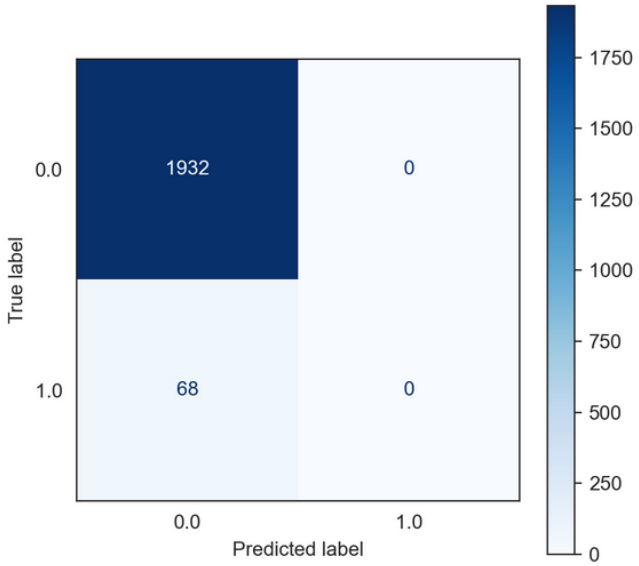

Figure 11 Performance matrix for Logistic Regression

Figure 11 above shows the performance matrix for the Logistic Regression Algorithm. In this case, the number of true positives is 1932, indicating the instances where the model correctly identified positive cases. The number of false positives is 0, suggesting that there were no instances where the model incorrectly classified negative cases as positive. The number of true negatives is 0, meaning that

the model did not correctly identify any negative cases. Finally, the number of false negatives is 68, indicating the instances where the model incorrectly classified positive cases as negative. This information provides an overview of the model's performance.

Confusion and Evaluation Metrics:

The table 2 below presents several evaluation metrics for a specific model. The metrics include accuracy, recall, precision, F1-score, mean squared error (MSE), and ROC AUC score. In this case, the accuracy is measured at 96.60%, indicating a high level of overall correctness in the model's predictions. The recall is 96.60%, suggesting that the model is effective at capturing positive cases. The precision value is 93.32%, indicating that when the model predicts a positive case, it is correct around 93.32% of the time. A higher F1-score indicates better performance. In this case, the F1-score is 94.93%, reflecting a good balance between precision and recall. The MSE is 3.40%, suggesting that the model's predictions deviate from the actual values to a certain extent. The ROC AUC score is 50.00%, suggesting that the model's predictive performance is similar to random guessing.

Table 2 Confusion and Evaluation Metrics for Logistic Regression

Accuracy	Recall	Precision	F1-Score	MSE	ROC AUC score
96.60%	96.60%	93.32%	94.93%	3.40%	50.00%

Overall, these metrics provide insights into the model's accuracy, effectiveness in capturing positive instances, precision, F1-score, deviation from actual values, and discrimination ability. They help assess the model's performance and identify areas for potential improvement.

4.5.2 Scenario 2 - Decision Tree classifier

A decision tree classifier is a popular algorithm used for both classification and regression tasks in machine learning. It is a supervised learning algorithm that builds a tree-like model of decisions and their possible consequences.

The decision tree algorithm operates by partitioning the input space iteratively into smaller and more manageable regions, utilizing the feature values. During each step, the algorithm chooses a feature and a split point that optimally separates the data, typically by maximizing information gain or minimizing impurity. This process continues until a predefined stopping condition is satisfied, such as reaching a maximum depth or a minimum number of instances in a leaf node.

The resulting decision tree consists of internal nodes that represent feature tests and leaf nodes that represent class labels or regression values. Each internal node corresponds to a feature and a split point, and the tree branches out based on the outcome of the feature test. Instances are classified or predicted by following the path from the root to a leaf node based on the feature values of the instance.

The advantages of the decision tree algorithm include its interpretability, as the resulting tree structure is easily understandable and can provide insights into the decision-making process. It can handle both numerical and categorical features, and it is not sensitive to feature scaling. Decision trees can also handle missing values and outliers effectively.

However, decision trees are prone to overfitting, where the model becomes too complex and captures noise or outliers in the training data. To mitigate overfitting, techniques like pruning, setting a minimum number of samples required to split a node, or limiting the maximum depth of the tree can be used.

Additionally, decision trees can be sensitive to small changes in the training data, and they may struggle to capture complex relationships between features.

To summarize, the decision tree classifier algorithm constructs a tree-like model that maps input features to class labels or regression values by recursively partitioning the data based on feature tests. It is a versatile and interpretable algorithm used in various domains for both classification and regression tasks. Here is the detailed explanation of how the decision tree algorithm works.

Data Preparation: First, you need to gather and pre-process your data. This involves collecting a labelled dataset with target values and selecting relevant features. The data can be categorical or numeric.

Tree Construction: The decision tree algorithm recursively partitions the data based on feature values to create a tree-like structure. The goal is to find the most useful features that best split the data into homogeneous subsets with respect to the target variable.

a. The algorithm starts with the root node, which represents the entire dataset.
b. It selects the best feature to split the data based on a criterion such as Gini impurity or information gain.
c. The chosen feature is used as a decision rule to divide the data into subsets (child nodes) based on its possible values.
d. This process is recursively applied to each child node until a stopping criterion is met, such as reaching a maximum tree depth or a minimum number of instances per leaf node.
e. The resulting tree structure represents a set of hierarchical if-else rules that can be used for prediction.

Tree Pruning: After constructing the initial decision tree, pruning techniques can be applied to reduce overfitting. Pruning involves removing nodes or branches that provide little or no additional predictive power on unseen data. This helps simplify the tree and improve its generalization performance.

Prediction: To make predictions using a decision tree, you start at the root node and follow the decision rules down the tree until you reach a leaf node. Each leaf node corresponds to a specific class or regression value. For classification, the majority class in the leaf node is assigned as the predicted class. For regression, the average value of the target variable in the leaf node is used as the prediction.

Evaluation: The performance of the decision tree model is assessed using evaluation metrics specific to the task at hand. For classification, metrics like accuracy, precision, recall, and F1-score can be used. For regression, metrics such as MSE are commonly used.

Decision trees are popular due to their interpretability, as the resulting tree structure can be visualized and understood easily. However, they can be prone to overfitting if not properly regularized or pruned. Ensemble methods like random forests and gradient boosting are often used to mitigate these limitations and improve the predictive accuracy of decision trees.

Results - Performance Metrics

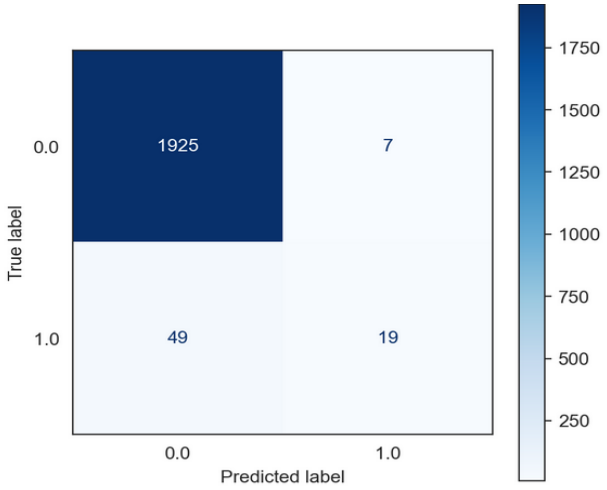

Figure 12 Performance metrics for Decision Tree Classifier

As shown in the Table 3 below the model has correctly identified 1925 instances as true positives, meaning they are positive cases correctly classified. However, there are 7 instances classified as false positives, indicating that the model has mistakenly classified them as positive cases when they are negative. The true negatives value is 19, indicating the number of negative cases correctly classified as negative by the model. Finally, there are 19 instances classified as false negatives, meaning the model has incorrectly labelled them as negative when, they are positive. The confusion matrix provides a visual representation of the model's performance and helps evaluate its accuracy and error rates. It demonstrates the model's ability to correctly identify positive and negative instances, as well as any misclassifications or errors. Analyzing the confusion matrix allows for a deeper understanding of the model's strengths and weaknesses and can guide further improvements and adjustments to enhance its performance.

Confusion and Evaluation Metrics:

The accuracy of the model is reported as 97.20%, which indicates the overall proportion of correctly classified instances. The recall, also known as sensitivity or true positive rate, is also 97.20%. It represents the ability of the model to identify positive instances correctly. The precision of 96.69% signifies the proportion of correctly classified positive instances out of the total instances predicted as positive. The F1-score, which combines precision and recall, is 96.59%. It provides a balanced measure of the model's accuracy. The MSE is reported as 2.80%. Additionally, the ROC AUC score is determined to be 63.79%. This metric evaluates the model's capacity to differentiate between positive and negative instances, providing a comprehensive assessment of its performance.

Table 3 Confusion and Evaluation Metrics for Decision Tree Classifier

Accuracy	Recall	Precision	F1-Score	MSE	ROC AUC Score
97.20%	97.20%	96.69%	96.59%	2.80%	63.79%

Overall, the table suggests that the model performs well with high accuracy, recall, precision, and F1-score. However, there is some room for improvement, as indicated by the non-zero values for MSE and the ROC AUC score. These metrics can be used to assess the model's performance, make comparisons with other models, and guide further enhancements to achieve better predictive capabilities.

4.5.3 Scenario 3 - SVM

A Support Vector Machine (SVM) is an influential supervised learning algorithm commonly employed for classification and regression tasks. It excels at tackling intricate problems characterized by a distinct margin of separation between different classes or groups of data points. SVM is known for its ability to handle non-linear relationships and is widely used in various domains for solving classification and regression problems.

In SVM, the algorithm creates a hyperplane in an n-dimensional space (where n is the number of features) that optimally separates different classes. The hyperplane is chosen in a way that maximizes the margin, which refers to the distance between the hyperplane and the closest data points from each class, commonly referred to as support vectors. The main idea is to find the hyperplane that best generalizes to unseen data and has the maximum margin, leading to improved performance and better generalization.

The SVM algorithm works as follows:

Data Preparation: The SVM algorithm starts with a labelled dataset consisting of input features and corresponding class labels. The input features should be numeric and pre-processed, if necessary, to normalize or scale them appropriately.

Feature Space and Hyperplanes: The SVM algorithm maps the input features to a higher-dimensional space to find a hyperplane that best separates the data points. The dimension of this space depends on the chosen kernel function, which can be linear, polynomial, Gaussian (RBF), sigmoid, or other types.

Linear Separability: In the case of linearly separable data, SVM seeks to identify the optimal hyperplane that maximizes the margin between the classes. The margin is defined as the distance between the hyperplane and the support vectors, which are the data points that lie closest to the hyperplane. The larger the margin, the better the separation and generalization ability of the model.

Support Vectors: Support vectors are the data points that lie closest to the decision boundary or hyperplane. These points have the most influence on the location and orientation of the hyperplane. The SVM algorithm focuses on these support vectors during training, while other data points are less important.

Hard Margin and Soft Margin: In cases where the data is not linearly separable, or there are outliers, a hard margin may not be achievable. SVM introduces a soft margin by allowing some misclassifications. The algorithm optimizes a trade-off between maximizing the margin and minimizing the classification error. The C parameter determines the penalty for misclassifications. A smaller C value allows more misclassifications (more tolerant to errors), while a larger C value aims for fewer misclassifications (stricter in classification).

Kernel Trick: The kernel trick is a key concept in SVM that allows it to handle non-linearly separable data by implicitly mapping the input features into a higher-dimensional feature space. The kernel function computes the dot product between the transformed features, avoiding the need to explicitly calculate the transformed feature vectors. This transformation enables SVM to find non-linear decision boundaries in the original feature space.

Training and Optimization: The SVM algorithm optimizes the hyperplane by solving a quadratic programming problem. The objective is to maximize the margin while minimizing the classification error. This optimization problem is typically solved using numerical optimization techniques, such as the Sequential Minimal Optimization (SMO) algorithm.

Prediction and Decision Function: Once the SVM model is trained, it can be used for prediction on new, unseen data points. The decision function of the SVM calculates the distance of a data point to the

hyperplane and assigns a class label based on its position relative to the hyperplane. The sign of the decision function determines the class label.

Model Evaluation: Once the SVM model is trained, it can be used to classify new instances by determining on which side of the hyperplane they lie. The sign of the predicted value (positive or negative) indicates the predicted class label.

SVM has several advantages, including its ability to handle high-dimensional data, effectiveness in dealing with complex datasets, and robustness against overfitting. However, SVM can be computationally expensive, especially with large datasets. Additionally, selecting the appropriate kernel function and tuning the regularization parameter can be challenging.

In summary, SVM is a versatile algorithm that finds an optimal hyperplane to separate classes in a given dataset. It is widely used in various domains, including text classification, image recognition, and bioinformatics, where clear class boundaries exist or need to be learned. In this case study we have only used the linear SVM algorithm to get the results.

Results - Performance Metrics

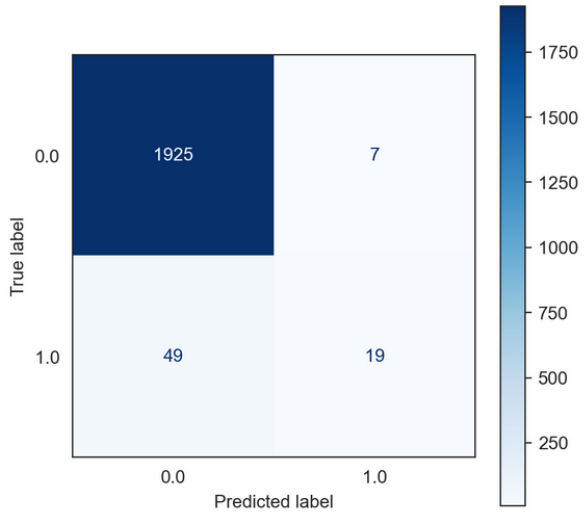

Figure 13 Performance metrics for SVM

The confusion matrix data provided reveals important information about the performance of a classification model. In the given classification scenario, the True Positive (TP) value is 1925, representing the number of instances correctly classified as positive. The False Positive (FP) value is 7, indicating the number of instances incorrectly classified as positive when they are actually negative. Similarly, the True Negative (TN) value is 19, signifying the number of instances correctly classified as negative. Lastly, the False Negative (FN) value is 49, representing the number of instances incorrectly classified as negative when they are actually positive. These values are key components in evaluating the performance of a classification model and can be used to calculate various evaluation metrics such as accuracy, precision, recall, and F1-score.

These values are crucial for evaluating the model's performance in terms of accuracy, precision, recall, and F1-score. The TP and TN values contribute to the accuracy metric, which measures the overall correctness of the model's predictions. The FP and FN values are used to calculate precision, which assesses the proportion of correctly classified positive instances out of all instances predicted as positive. Similarly, recall, also known as sensitivity, measures the model's ability to identify positive instances correctly. The F1-score is a harmonic mean of precision and recall, providing a balanced assessment of the model's performance. By analyzing these values, we can assess the strengths and weaknesses of the model and identify areas for improvement.

Confusion and Evaluation Metrics:

The Table 4 below provides various performance metrics for a SVM model. The accuracy of the model is determined to be 97.10%, indicating the overall correctness of the model's predictions. This metric considers both true positives and true negatives.

The recall, also known as sensitivity or true positive rate, is measured at 97.10%. It signifies the model's ability to correctly identify positive instances out of the total actual positive instances. Precision, on the other hand, is determined to be 96.95%. This metric represents the proportion of correctly classified positive instances out of all instances predicted as positive. The F1-score, which combines precision and recall into a single metric, is calculated at 96.11%. This score provides a balanced measure of the model's performance, considering both precision and recall simultaneously. The MSE, a regression metric, is noted as 2.90%. It quantifies the average squared difference between the predicted and actual values, indicating the model's accuracy in predicting continuous outcomes. Lastly, the ROC AUC score is determined to be 58.06%. This metric assesses the model's ability to discriminate between positive and negative instances across various probability thresholds.

Table 4 Confusion and Evaluation Metrics for SVM

Accuracy	Recall	Precision	F1-Score	MSE	ROC AUC Score
97.10%	97.10%	96.95%	96.11%	2.90%	58.06%

By examining these metrics, we gain insights into the performance of the classification model, allowing us to evaluate its effectiveness and identify areas for potential improvement.

4.5.4 Scenario 4 - k-NN classifier

The k-nearest neighbors (k-NN) algorithm is a widely used supervised learning method for both classification and regression tasks. It is a non-parametric algorithm that operates on the principle of similarity. The k-NN algorithm makes predictions by comparing the input instance to its k nearest neighbors in the training data. In classification tasks, the class label of the majority of the k nearest neighbors is assigned to the input instance. In regression tasks, the algorithm predicts the average or weighted average of the target values of the k nearest neighbors. The choice of the parameter k influences the algorithm's performance and can be determined through cross-validation or other optimization techniques. The k-NN algorithm works as follows:

Data Preparation: The algorithm starts with a labelled dataset consisting of input features and corresponding class labels. The input features can be numeric or categorical, but they need to be transformed into a numerical representation suitable for the algorithm. Categorical features are often encoded using one-hot encoding or other techniques.

Distance Calculation: The algorithm computes the distance between each instance in the dataset and all other instances. The specific distance metric used, such as Euclidean distance or Manhattan distance, is determined based on the characteristics of the data and the specific problem being addressed.

Finding Neighbours: The k-nearest neighbors (k-NN) algorithm determines the k nearest neighbors of a given instance by evaluating the distances between instances in the dataset. The selection of the neighbors is based on these calculated distances. The value of k is a hyperparameter that must be predetermined before applying the algorithm. The neighbours can be determined using a variety of methods, such as sorting the distances and selecting the k smallest values.

Classification or Regression: For classification tasks, the class label of most of the k nearest neighbours is assigned to the instance being predicted. In other words, the most common class among the neighbours is chosen as the predicted class label. For regression tasks, the algorithm takes the average or weighted average of the target values of the k nearest neighbours as the predicted value.

Model Evaluation: For regression tasks, the output values of the k nearest neighbours are considered. The predicted output value for the given data point is calculated as the average or weighted average of the output values of the k neighbours.

Prediction: Once the prediction is made for the given data point, the process is repeated for each data point in the dataset, resulting in predictions for the entire dataset.

Model Evaluation: The performance of the k-NN model is typically evaluated using appropriate metrics such as accuracy, precision, recall, F1-score, mean squared error (MSE), or other relevant metrics depending on the task at hand.

Model Tuning: The k-NN algorithm can be further optimized by tuning the hyperparameters, such as the distance metric used, the value of k, or applying feature selection or dimensionality reduction techniques to enhance the model's performance.

The k-NN algorithm is intuitive and easy to understand. It can handle both binary and multi-class classification problems as well as regression tasks. However, the k-NN algorithm can be sensitive to the choice of k and the distance metric used. It can also suffer from the curse of dimensionality, where the performance degrades as the number of features increases. Therefore, feature selection or dimensionality reduction techniques may be necessary to mitigate this issue.

The k-NN algorithm has several advantages, including its simplicity, flexibility, and ability to handle multi-class classification. It does not make any statements about the underlying data distribution and can work well with both linear and non-linear associations between features and class labels. Additionally, k-NN allows for incremental learning, meaning it can be updated with new instances without retraining the entire model.

However, the k-NN algorithm has some limitations. It can be sensitive to the choice of distance metric and the value of k. Choosing an appropriate value of k is important to balance bias and variance in the model. The algorithm also requires enough training data, and the prediction time can be slow for large datasets due to the need to calculate distances. In summary, the k-nearest neighbours algorithm is a versatile and intuitive algorithm that makes predictions based on the similarity to neighbouring instances. It is widely used for classification and regression tasks, particularly when the underlying data distribution is unknown or complex.

Results - Performance Metrics

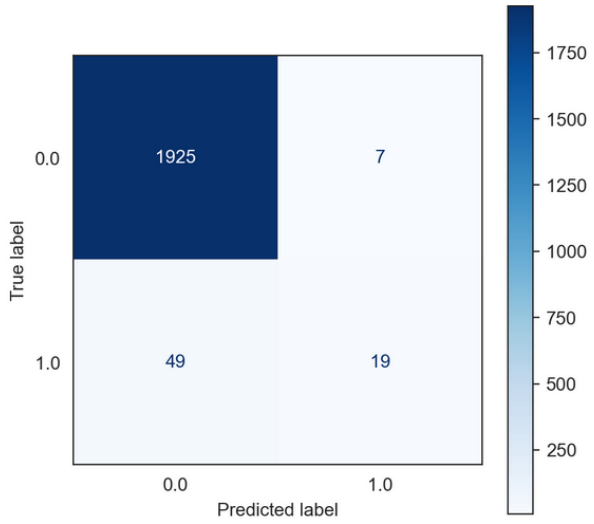

Figure 14 Performance metrics for k-NN Classifier

The Performance matrix as shown in the Figure 14 reveals the performance of a classification model. The true positive (TP) value is recorded as 1925, indicating the number of correctly predicted positive instances. The false positive (FP) value is 7, representing the instances that were incorrectly classified as positive. The true negative (TN) value is 19, signifying the correctly predicted negative instances. Lastly, the false negative (FN) value is 49, indicating the instances that were wrongly classified as negative.

These values in the confusion matrix provide valuable information about the model's performance in terms of correctly identifying positive and negative instances. It allows us to assess the model's accuracy, precision, and recall. In this case, with a higher number of false negatives compared to false positives, the model is more likely to incorrectly classify positive instances as negative. Understanding these values aids in evaluating the model's strengths and weaknesses, enabling adjustments to improve its performance.

Confusion and Evaluation Metrics:

The k-Nearest Neighbors (k-NN) classifier performance metrics are provided in the Table 5 below. The accuracy of the model is measured at 97.35%, indicating the proportion of correctly classified instances. The recall, also known as the true positive rate, is also 97.35%, indicating the model's ability to correctly identify positive instances. The precision, which measures the proportion of correctly predicted positive instances out of all predicted positive instances, is 96.91%. The F1-score, a measure that combines precision and recall, is 96.94%.

Additionally, the MSE is calculated at 2.65%, which quantifies the average squared difference between the predicted and actual values. The ROC AUC score, which assesses the model's ability to distinguish between positive and negative classes, is 68.12%. These metrics collectively demonstrate the effectiveness of the k-NN classifier in accurately classifying instances and distinguishing between positive and negative classes. With high values across multiple metrics, the k-NN classifier proves to be a reliable model for the given dataset.

Table 5 Confusion and Evaluation Metrics for k-NN Classifier

Accuracy	Recall	Precision	F1-Score	MSE	ROC AUC Score
97.35%	97.35%	96.91%	96.94%	2.65%	68.12%

4.5.5 Scenario 5 - Naïve Bayes Model

The Naïve Bayes algorithm is a classification technique in machine learning that leverages probabilistic principles. It applies Bayes' theorem and assumes the conditional independence of features with respect to the given class variable. Despite this strong assumption, Naïve Bayes has proven to be more effective in many real-world applications. The Naïve Bayes algorithm works as follows:

Data Preparation: The Naïve Bayes algorithm starts with a labelled dataset consisting of input features and corresponding class labels. The input features can be categorical or numerical, but they need to be transformed into discrete or categorical form to apply the algorithm.

Training Phase: During the training phase, the Naïve Bayes algorithm estimates the prior probabilities and conditional probabilities from the training data. It calculates the prior probability of each class label and the conditional probability of each feature given the class label.

Bayes' Theorem: The Naïve Bayes algorithm applies Bayes' theorem to calculate the posterior probability of each class given the input features. Bayes' theorem states that the posterior probability is proportional to the product of the prior probability and the conditional probability.

Conditional Independence Assumption: The Naïve Bayes algorithm operates under the assumption of conditional independence among features given the class variable. This assumption implies that the presence or absence of one feature does not influence the presence or absence of other features. Although this assumption is often violated in real-world scenarios, Naïve Bayes can still provide reasonably accurate results.

Probability Estimation: To classify a new data point, the Naïve Bayes algorithm calculates the posterior probability for each class label based on the input features. It multiplies the prior probability of the class label with the conditional probabilities of the features given that class label.

Laplace Smoothing: To handle the issue of zero probabilities when a feature value is not present in the training data for a particular class, the Naïve Bayes algorithm uses Laplace smoothing. Laplace smoothing adds a small constant to all probabilities to ensure non-zero probabilities and avoid division by zero.

Model Evaluation: The performance of the Naïve Bayes model is typically evaluated using metrics such as accuracy, precision, recall, F1-score, or other relevant metrics depending on the task at hand. Cross-validation or hold-out validation techniques can be employed to estimate the model's performance and tune the hyperparameters. Naïve Bayes is known for its simplicity, speed, and efficiency in training and prediction. It performs well even with small amounts of training data and is less prone to overfitting compared to more complex algorithms. However, the assumption of feature independence can limit its effectiveness in cases where features are highly correlated. It serves as a useful baseline algorithm and can provide quick and reliable results in many classifications task.

Results - Performance Metrics

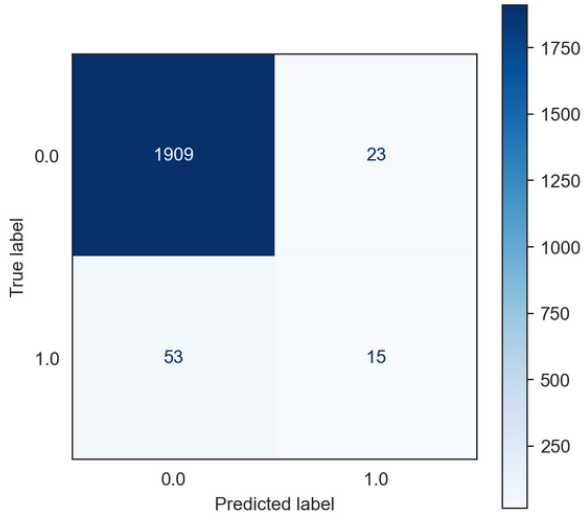

Figure 15 Performance metrics for Naive Bayes Model

The confusion matrix for the Naive Bayes Model, based on the Figure 15, shows that there are 1909 true positive (TP) instances correctly identified by the model. There are 23 false positive (FP) instances where the model predicted a positive outcome incorrectly. Additionally, there are 15 true negative (TN) instances correctly identified as negative by the model. However, there are 53 false negative (FN) instances where the model predicted a negative outcome incorrectly.

The Naive Bayes Model's ability to accurately classify positive instances is reflected in the high number of true positives. The existence of false positives and false negatives suggests that there is potential for enhancing the precision and recall of the model.False positives can lead to unnecessary actions or interventions, while false negatives may result in missed opportunities for identifying positive instances.

Confusion and Evaluation Metrics:

The Naive Bayes Model achieved an accuracy of 96.20% based on the data in Table 6. This metric indicates the overall correctness of the model's predictions. The recall of 96.20% suggests that the model effectively identified a high percentage of true positive instances out of all actual positive instances. The precision of 95.33% reflects the model's ability to accurately classify instances as positive, minimizing false positives. The F1-score of 95.68% combines both precision and recall, providing a balanced measure of the model's performance.

The MSE of 3.80% signifies the average squared difference between the predicted and actual values. A lower MSE suggests better performance, as it indicates less discrepancy between the predicted and actual values. On the other hand, the ROC AUC score of 60.43% measures the model's ability to discriminate between positive and negative instances. A higher ROC AUC score indicates better discrimination, implying that the model is more effective at distinguishing between the two classes.

Table 6 Confusion and Evaluation Metrics Naive Bayes Model

Accuracy	Recall	Precision	F1-Score	MSE	ROC AUC Score
96.20%	96.20%	95.33%	95.68%	3.80%	60.43%

Overall, the Naive Bayes Model shows moderately good running in terms of accuracy, recall, precision, and F1-score. However, there is room for improvement, as indicated by the relatively higher MSE and lower ROC AUC score. Fine-tuning the model or exploring alternative algorithms may help enhance its predictive capabilities and reduce errors.

4.5.6 Scenario 6 - Random Forest

The Random Forest algorithm is an ensemble learning technique that is highly flexible and potent, suitable for both classification and regression tasks. It constructs multiple decision trees and then combines their individual predictions to generate more precise and resilient overall predictions. By leveraging the collective wisdom of multiple trees, Random Forest enhances the accuracy and robustness of the model. It is widely used in various domains due to its effectiveness in handling complex datasets and mitigating overfitting. The Random Forest algorithm works as follows:

Data Preparation: The Random Forest algorithm starts with a labelled dataset consisting of input features and corresponding class labels (for classification) or output values (for regression). The input features should be numeric and pre-processed, if necessary, to normalize or scale them appropriately.

Ensemble of Decision Trees: Random Forest creates an ensemble of decision trees. Each decision tree is constructed using a subset of the training data and a subset of the input features. The subset of the training data is sampled with replacement using a method called bootstrapping, which is known as the bagging technique. The subset of input features is randomly selected for each tree.

Building Decision Trees: For each decision tree in the ensemble, the Random Forest algorithm builds the tree recursively. At each node of the tree, a feature is selected based on a splitting criterion (e.g., Gini impurity or information gain) that maximizes the separation between the classes or reduces the variance in the case of regression. The tree continues to split the data into smaller subsets until a stopping criterion is met (e.g., reaching a maximum depth or minimum number of samples in a leaf node).

Voting or Averaging Predictions: For classification tasks, each decision tree in the Random Forest ensemble independently predicts the class label of a new data point. The final prediction is determined by majority voting, where the class label that receives the most votes across all trees is assigned as the predicted class label. For regression tasks, the final prediction is calculated as the average or weighted average of the predictions from all trees.

Handling Out-of-Bag (OOB) Data: During the training process, some data points may not be included in the training subset for a particular decision tree. These out-of-bag (OOB) data points can be used for estimating the model's performance without the need for cross-validation. The OOB samples are passed through the corresponding decision tree, and their predictions are aggregated to evaluate the model's accuracy.

Feature Importance: Random Forest algorithm offers a way to assess the significance of features by evaluating their contribution to the accuracy of predictions across all the trees in the ensemble. It provides a measure of feature importance by quantifying the extent to which each feature influences the overall predictive performance. This information can be useful for feature selection and understanding the underlying importance of different features in the dataset.

Model Evaluation: The performance of the Random Forest model is typically evaluated using appropriate metrics such as accuracy, precision, recall, F1-score, mean squared error (MSE), or other relevant metrics depending on the task at hand. Cross-validation or hold-out validation techniques can be employed to estimate the model's performance and tune the hyperparameters. It is a versatile

algorithm that can handle both classification and regression tasks and has been widely used in various domains such as finance, healthcare, and natural language processing.

Random Forest offers several advantages. It can handle high-dimensional datasets, large amounts of training data, and a mixture of feature types. It is less prone to overfitting compared to individual decision trees and provides good generalization to unseen data. Random Forest also has the ability to estimate feature importance, which can be helpful in feature selection.

However, Random Forest can be computationally expensive and memory-intensive, particularly for large datasets with numerous trees. The interpretability of the model can be challenging due to the complexity of the ensemble. Additionally, Random Forest may not perform well on datasets with imbalanced classes or noisy features.

Random Forest has been successfully applied in various domains, including finance, healthcare, ecology, and image recognition. It is a popular and widely-used algorithm due to its effectiveness, versatility, and robustness in handling a wide range of classification and regression tasks.

Results - Performance Metrics

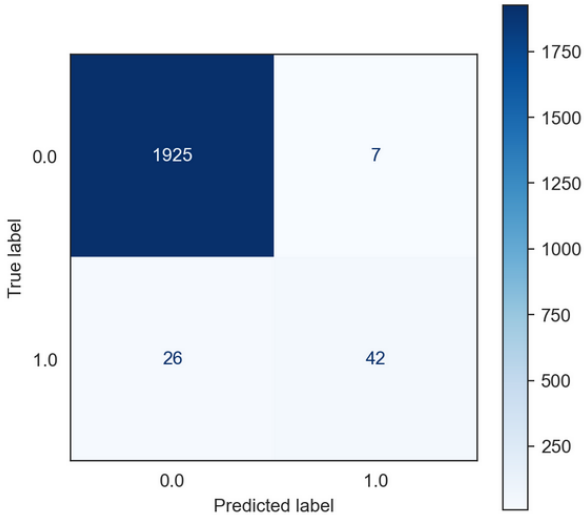

Figure 16 Performance metrics for Random Forest

As shown in Figure 16, the confusion matrix for the Random Forest model shows that out of the total positive instances, 1925 were correctly classified as true positives (TP), while 7 instances were incorrectly classified as false positives (FP). On the other hand, out of the total negative instances, 42 were correctly classified as true negatives (TN), while 26 instances were incorrectly classified as false negatives (FN).

The Random Forest model achieved a high accuracy of 98.35%, indicating that a large portion of the predictions made by the model were correct. The recall of 98.35% suggests that the model effectively identified a high percentage of true positive instances out of all actual positive instances. The precision of 98.23% reflects the model's ability to accurately classify instances as positive, minimizing false positives. The F1-score of 98.22% combines both precision and recall, providing a balanced measure of the model's performance.

Overall, the Random Forest model demonstrates excellent performance in terms of accuracy, recall, precision, and F1-score. It effectively captures the true positive instances while maintaining a low rate of false positives. The model's ability to accurately classify both positive and negative instances is further supported by the high TN value of 42. This strong performance suggests that the Random Forest

model is well-suited for predictive tasks, particularly in scenarios where minimizing false positives and false negatives is crucial.

Confusion and Evaluation Metrics:

The Random Forest model achieved outstanding performance based on the evaluation metrics as given in Table 7. It achieved an accuracy of 98.35%, indicating that most of the predictions made by the model were correct. The recall score of 98.35% suggests that the model effectively identified a high percentage of true positive instances out of all actual positive instances. The precision score of 98.23% reflects the model's ability to accurately classify instances as positive, minimizing false positives. The F1-score of 98.22% combines both precision and recall, providing a balanced measure of the model's overall performance.

The MSE of 1.65% is a measure of the average squared difference between the predicted and actual values, and a lower MSE indicates better accuracy in predicting the target variable. The model achieved a relatively low MSE, indicating its ability to make accurate predictions.

The ROC AUC score of 80.70% measures the performance of the model in distinguishing between positive and negative instances. A higher ROC AUC score suggests a better ability to classify instances correctly.

Table 7 Confusion and Evaluation Random Forest

Accuracy	Recall	Precision	F1-Score	MSE	ROC AUC Score
98.35%	98.35%	98.23%	98.22%	1.65%	80.70%

Overall, the Random Forest model performed exceptionally well across all evaluation metrics, demonstrating its effectiveness in accurately predicting outcomes and maintaining a balance between precision and recall. The low MSE and high ROC AUC score further indicate the model's robustness and ability to discriminate between classes. These results highlight the Random Forest model as a powerful tool for predictive tasks, especially in scenarios where high accuracy and reliable performance are essential.

4.5.7 Scenario 7 - Multi-layer perceptron (MLP)

The MLP algorithm is a popular choice for both classification and regression tasks in artificial neural networks. It is a type of feedforward neural network comprising multiple layers of interconnected nodes, often referred to as perceptrons or artificial neurons. The MLP architecture allows for complex patterns and relationships to be learned and modeled, making it a versatile and widely used approach in various machine learning applications. The MLP algorithm works as follows:

Data Preparation: The MLP algorithm starts with a labelled dataset consisting of input features and corresponding class labels (for classification) or output values (for regression). The input features should be numeric and pre-processed, if necessary, to normalize or scale them appropriately.

Neural Network Architecture: The MLP algorithm is structured with three main components: an input layer, one or more hidden layers, and an output layer. The input layer acts as the entry point for the algorithm, receiving the input features. In this layer, each neuron represents a specific feature of the input data. The hidden layers are composed of multiple neurons that process the information and learn complex patterns. The output layer produces the final predictions, with each neuron representing a class label (for classification) or an output value (for regression).

Forward Propagation: In the forward propagation step, the input data is fed into the neural network. Each neuron in the hidden layers and output layer performs a weighted sum of the inputs it receives, applies an activation function, and passes the result to the next layer. The activation function introduces non-linearity and enables the neural network to learn complex relationships between the input features and the output.

Weight Update and Backpropagation: After the forward propagation step, the MLP algorithm compares the predicted output with the true output from the dataset and calculates an error value. It then uses an optimization algorithm, typically gradient descent, to update the weights of the neurons in the network. The goal is to minimize the error by adjusting the weights to improve the model's predictions.

Training: The training process continues by iteratively performing forward propagation and weight updates using the training data. The algorithm adjusts the weights based on the errors generated in each iteration, gradually improving the model's performance. The number of iterations or epochs and the learning rate are hyperparameters that need to be determined before training.

Activation Functions: The choice of activation functions for the neurons in the MLP algorithm is crucial. The activation function determines the output of a neuron based on its input and helps introduce non-linearity, enabling the neural network to learn complex patterns.

Model Evaluation: Once the training is completed, the performance of the MLP model is evaluated using appropriate metrics such as accuracy, precision, recall, F1-score, MSE, or other relevant metrics depending on the task at hand. Cross-validation or hold-out validation techniques can be employed to estimate the model's performance and tune the hyperparameters.

Prediction: To make predictions on new data, the MLP algorithm applies the learned weights to the input features and performs forward propagation through the network. The output from the output layer represents the predicted class label (for classification) or the predicted output value (for regression) for the given input.

MLP algorithms are known for their ability to learn complex patterns and relationships in data. They can handle large amounts of data and are capable of approximating any continuous function given a sufficiently large and well-trained network. However, training an MLP can be computationally expensive and requires careful tuning of hyperparameters to avoid overfitting. They are powerful models that have contributed to advancements in machine learning and artificial intelligence.

Results - Performance Metrics

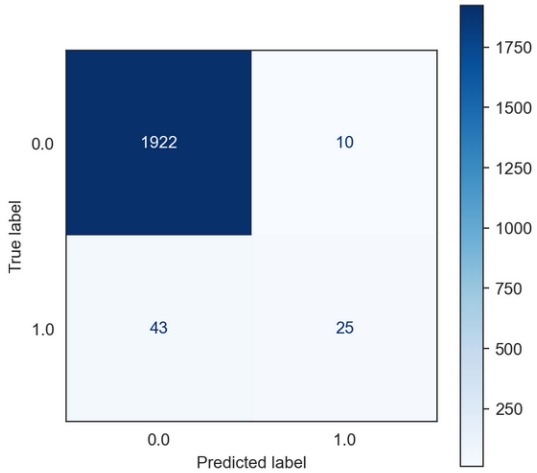

Figure 17 Performance metrics for multi-Layer perception

The performance metrics for the Multi-layer Perceptron (MLP) model shown in Figure 17 demonstrate its effectiveness in classification tasks. The model achieved a true positive (TP) count of 1922, indicating its ability to correctly classify a large number of positive instances. The false positive (FP) count of 10 suggests a relatively small number of instances incorrectly classified as positive. The true negative (TN) count of 25 signifies the correct classification of negative instances, while the false negative (FN) count of 43 suggests a moderate number of instances incorrectly classified as negative.

The accuracy score of the MLP model is 97.35%, reflecting the overall correctness of the model's predictions. The recall score of 97.35% indicates the model's ability to identify a high percentage of true positive instances out of all actual positive instances. The precision score of 98.23% suggests a high level of accuracy in classifying instances as positive, minimizing false positives. The F1-score of 96.91% combines both precision and recall, providing a balanced measure of the model's performance.

The mean squared error (MSE) of 2.65% is relatively low, indicating the model's ability to make accurate predictions. The ROC AUC score of 68.12% measures the model's ability to discriminate between positive and negative instances.

Overall, the MLP model demonstrates strong performance across various evaluation metrics, including accuracy, recall, precision, F1-score, MSE, and ROC AUC score. These results highlight the model's capability to effectively classify instances and make accurate predictions in classification tasks.

Confusion and Evaluation Metrics:

The results as shown in Table 8 of the Multi-layer Perceptron (MLP) model showcase its strong performance in classification tasks. With an accuracy score of 97.35%, the model demonstrates a high level of correctness in its predictions. The recall score of 97.35% indicates the model's ability to identify a significant proportion of true positive instances out of all actual positive instances. Moreover, the precision score of 98.23% highlights the model's accuracy in classifying instances as positive, minimizing the occurrence of false positives. The F1-score of 96.34% combines both precision and recall, providing a balanced measure of the model's performance.

In terms of error analysis, the model exhibits a relatively low MSE of 2.65%, indicating its ability to make accurate predictions. The ROC AUC score of 68.12% measures the model's ability to discriminate between positive and negative instances.

Table 8 Confusion and Evaluation for Multi-Layer perception

Accuracy	Recall	Precision	F1-Score	MSE	ROC AUC Score
97.35%	97.35%	98.23%	96.34%	2.65%	68.12%

Overall, the MLP model delivers promising results across various evaluation metrics, including accuracy, recall, precision, F1-score, MSE, and ROC AUC score. These findings demonstrate the model's efficacy in classifying instances and making accurate predictions in classification tasks.

4.5.8 Scenario 8 - Gradient Boosting Classifier

The Gradient Boosting classifier is a machine learning algorithm that constructs a powerful predictive model by combining multiple weak learners, often decision trees. It belongs to the family of boosting algorithms, which iteratively train new models to rectify errors made by the previous models. The focus is on instances that were misclassified, allowing subsequent models to concentrate on difficult cases and improve overall prediction accuracy. By sequentially building upon the strengths of weak learners, Gradient Boosting creates a robust and accurate predictive model. The Gradient Boosting classifier works as follows:

Data Preparation: The Gradient Boosting algorithm starts with a labelled dataset consisting of input features and corresponding class labels (for classification) or output values (for regression). The input features should be numeric and pre-processed, if necessary, to normalize or scale them appropriately.

Initialization: The algorithm begins by initializing the model with a simple base learner, often a decision tree with a small depth. This base learner is referred to as the first weak learner.

Loss Function: A loss function is defined based on the task at hand (e.g., mean squared error for regression, log loss for classification). The loss function evaluates the disparity between the predicted values generated by the model and the actual values present in the data. It provides a quantitative measure of the model's performance by quantifying the extent of deviation or error in its predictions. By assessing the loss function, we can gauge how well the model is able to capture the underlying patterns and relationships in the data, and make adjustments or improvements accordingly.

Iterative Training: The Gradient Boosting algorithm trains a sequence of weak learners iteratively, where each subsequent learner is trained to correct the errors made by the previous learners. This process continues until a pausing criterion is met (e.g., reaching a maximum number of iterations or achieving satisfactory performance).

Gradient Descent: In each iteration, the algorithm calculates the negative gradient of the loss function with respect to the current model's predictions. This gradient represents the direction and magnitude of the improvement required to minimize the loss. The subsequent weak learner is trained to fit the negative gradient, aiming to reduce the errors made by the previous model.

Weighted Voting: Each weak learner is assigned a weight based on its performance. In subsequent iterations, the algorithm combines the predictions of all previous learners by multiplying them with their respective weights. The final prediction is determined by summing the weighted predictions.

Learning Rate: The learning rate is a hyperparameter that controls the contribution of each weak learner to the final model. It scales the weight of each weak learner's prediction, influencing the impact of subsequent learners on the overall model. A lower learning rate makes the algorithm more conservative, preventing overfitting, but may require more iterations to converge.

Regularization: To prevent overfitting, regularization techniques are often employed in Gradient Boosting. These include limiting the maximum depth of the decision trees, introducing a shrinkage parameter (learning rate), and applying early stopping criteria based on the validation set performance.

Model Evaluation: The performance of the Gradient Boosting model is typically evaluated using appropriate metrics such as accuracy, precision, recall, F1-score, mean squared error (MSE), or other relevant metrics depending on the task at hand. Cross-validation or hold-out validation techniques can be employed to estimate the model's performance and tune the hyperparameters.

Prediction: To make predictions on new data, the Gradient Boosting algorithm applies the weighted sum of the predictions from all weak learners. The final prediction is determined by the combined effect of all the learners, with the weights assigned based on their performance.

Gradient Boosting classifiers are known for their high predictive accuracy and robustness. They can handle complex patterns and capture non-linear relationships in the data. However, they can be computationally expensive and require careful tuning of hyperparameters to avoid overfitting.

Gradient Boosting classifiers have been widely used in various domains, including healthcare, finance, marketing, and natural language processing. They have achieved state-of-the-art results in many machine learning competitions and are popular due to their effectiveness and versatility.

Results

Performance Metrics

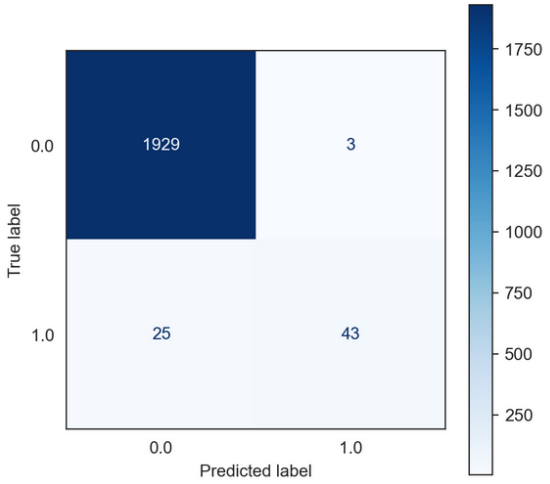

Figure 18 Performance metrics for Gradient Boosting Classifier

The confusion matrix shown in Figure 18 for the Gradient Boosting Classifier reveals its strong performance in classifying instances. With 1929 true positive (TP) predictions, the model accurately identifies a large number of positive instances. The low false positive (FP) count of 3 indicates the model's ability to minimize the misclassification of negative instances as positive. Additionally, the true negative (TN) count of 43 demonstrates the model's proficiency in correctly identifying negative instances. However, there are 25 false negative (FN) predictions, where the model incorrectly classifies positive instances as negative.

Overall, the Gradient Boosting Classifier shows a favorable balance between TP and TN predictions, indicating its ability to effectively discriminate between positive and negative instances. By minimizing false positives and negatives, the model exhibits a high level of precision in its classification results. These findings highlight the effectiveness of the Gradient Boosting Classifier in accurately classifying instances and making informed predictions.

Confusion and Evaluation Metrics:

The Gradient Boosting Classifier demonstrates exceptional performance as shown in Table 9 comes across multiple evaluation metrics. With an accuracy of 98.60%, the model achieves a high overall correct prediction rate. The recall and precision values of 98.60% and 98.54%, respectively, indicate the model's ability to accurately identify positive instances and minimize false positives. The F1-score, a harmonic mean of precision and recall, is 96.54%, demonstrating a balanced performance between precision and recall.

The model's low MSE of 1.40% suggests its ability to minimize the average squared difference between predicted and actual values. Additionally, the ROC AUC score of 81.54% indicates the model's capability to distinguish between positive and negative instances.

Table 9 Confusion and Evaluation for Gradient Boosting Classifier

Accuracy	Recall	Precision	F1-Score	MSE	ROC AUC Score
98.60%	98.60%	98.54%	96.54%	1.40%	81.54%

Overall, the Gradient Boosting Classifier exhibits strong predictive performance, combining high accuracy, recall, precision, and F1-score. Its low MSE and respectable ROC AUC score further

reinforce its effectiveness in classification tasks. These results underscore the potential of the Gradient Boosting Classifier in various applications, particularly those requiring precise and reliable predictions.

4.6 Overall evaluation metrics results of the case studies

The table 9 below provided presents the performance metrics for different machine learning models in the context of predictive maintenance. These models are evaluated based on their accuracy, recall, precision, F1-score, MSE, and ROC AUC score.

Predictive maintenance aims to identify potential issues or failures in industrial machinery before they occur, allowing proactive maintenance and minimizing downtime. Therefore, a good predictive maintenance model should have high accuracy, recall, precision, and F1-score, indicating its ability to correctly predict both positive and negative instances.

Looking at the table, we can observe that several models achieve high accuracy, indicating their overall predictive power. Specifically, the Random Forest model achieves an accuracy of 98.35%, suggesting that it correctly predicts maintenance requirements with high confidence.

Moreover, the Random Forest model also demonstrates high recall, precision, and F1-score, all above 98%, indicating its ability to correctly identify and classify instances of maintenance requirements. This is crucial for predictive maintenance as it ensures that actual maintenance needs are not missed or overlooked, reducing the risk of unexpected failures.

Additionally, the low MSE values across the models indicate that their predictions are generally close to the actual values, which is desirable for accurate predictive maintenance.Furthermore, the ROC AUC scores for the Random Forest and Gradient Boosting Classifier models are relatively high, indicating their strong ability to distinguish between positive and negative instances. This is valuable in predictive maintenance, as it allows for setting appropriate thresholds and effectively prioritizing maintenance actions.

In conclusion, the models with high accuracy, recall, precision, F1-score, and low MSE, such as the Random Forest and Gradient Boosting Classifier, demonstrate promising performance for predictive maintenance. These models can effectively identify and classify instances requiring maintenance, enabling proactive actions to be taken, reducing downtime, and optimizing industrial processes. However, hybrid models can achieve better results that is optimised for the dataset.

The Table 10 below represents the performance metrics for various machine learning models used for classification. Logistic Regression achieves an accuracy of 96.60%, with a recall of 96.60% and a precision of 93.32%. The F1-score is 94.93%, indicating a balanced performance between precision and recall. However, it has a higher MSE of 3.40% and a relatively low ROC AUC score of 50.00%.

The Decision Tree model performs slightly better, with an accuracy of 97.20% and a precision of 96.69%. It achieves a recall of 97.20% and an F1-score of 96.59%. The model has a lower MSE of 2.80% and a higher ROC AUC score of 63.79%. Similarly, the Support Vector Machine achieves an accuracy of 97.10% with a precision of 96.95% and a recall of 97.10%. The F1-score is 96.11%, indicating a good balance between precision and recall. The model has a MSE of 2.90% and a ROC AUC score of 58.06%.

The K-Nearest Neighbour model performs well, with an accuracy of 97.35% and a precision, recall, and F1-score of 96.91%, 97.35%, and 96.94%, respectively. The model has a relatively low MSE of 2.65% and a higher ROC AUC score of 68.12%. The Naive Bayes Model achieves an accuracy of 96.20% with a precision of 95.33% and a recall of 96.20%. The F1-score is 95.68%, indicating a good balance between precision and recall. The model has a higher MSE of 3.80% and a ROC AUC score of 60.43%.

The Random Forest model performs impressively, with an accuracy of 98.35% and precision, recall, and F1-score values of 98.23%, 98.35%, and 98.22%, respectively. It has a low MSE of 1.65% and a higher ROC AUC score of 80.70%. The Multi-layer perceptron (MLP) achieves an accuracy of 97.35%, with a precision of 98.23% and a recall of 97.35%. The F1-score is 96.91%, indicating a good balance between precision and recall. The model has a MSE of 2.65% and a ROC AUC score of 68.12%.

Lastly, the Gradient Boosting Classifier achieves the highest accuracy of 98.60% and a precision, recall, and F1-score of 98.54%, 98.60%, and 96.54%, respectively. It has a low MSE of 1.40% and a high ROC AUC score of 81.54%.

Table 10 Overall results of scenarios

	Accuracy	**Recall**	**Precision**	**F1-Score**	**MSE**	**ROC AUC Score**	**Rank**
Logistic Regression	96.60%	96.60%	93.32%	94.93%	3.40%	50.00%	8
Decision Tree	97.20%	97.20%	96.69%	96.59%	2.80%	63.79%	5
Support Vector Machine	97.10%	97.10%	96.95%	96.11%	2.90%	58.06%	6
K-Nearest Neighbour	97.35%	97.35%	96.91%	96.94%	2.65%	68.12%	3
Naive Bayes Model	96.20%	96.20%	95.33%	95.68%	3.80%	60.43%	7
Random Forest	98.35%	98.35%	98.23%	98.22%	1.65%	80.70%	2
Multi-layer perceptron (MLP)	97.35%	97.35%	98.23%	96.91%	2.65%	68.12%	4
Gradient Boosting Classifier	98.60%	98.60%	98.54%	96.54%	1.40%	81.54%	1

These performance metrics demonstrate the effectiveness of the different models for classification tasks, with the Gradient Boosting Classifier and Random Forest model performing exceptionally well in terms of accuracy, precision, recall, F1-score, and ROC AUC score.

4.7 Conclusion

This chapter has presented an overview of a case study conducted on a time-series dataset for predictive maintenance in the industrial sector using machine learning algorithms. The case study aimed to harness the power of ML algorithms to predict equipment failures, allowing for proactive maintenance and minimizing downtime. The sections of this paper provided in depth methodology, data analysis techniques, results, and discussions, providing detailed insights into the effectiveness of ML algorithms based on confusion metrics and evaluation metrics. Through this comprehensive analysis, a clearer understanding of the performance and potential of ML in predictive maintenance for industrial machinery is gained.

Chapter 5 Random Forest-Gradient Boosting classifier Hybrid model (RF-GB)

5.1 Introduction

The research focuses on developing a hybrid model for the Industrial datasets like AI4I2020 to improve the accuracy of the outcome. The Hybrid model optimised for this research is Random Forest-Gradient Boosting classifier. A Smart Predictive Maintenance for pro-active machine process hybrid model is developed in this chapter. This RF-GB hybrid machine learning model is to facilitate both real-time and synthetic datasets. The RF-GB hybrid model combines the strengths of two popular machine learning algorithms, Random Forest, and Gradient Boosting, to improve predictive accuracy and handle complex datasets effectively.

Figure 19 provides the overview of the hybrid model devilment in a flowchart. This gives detail information about each step during this process.

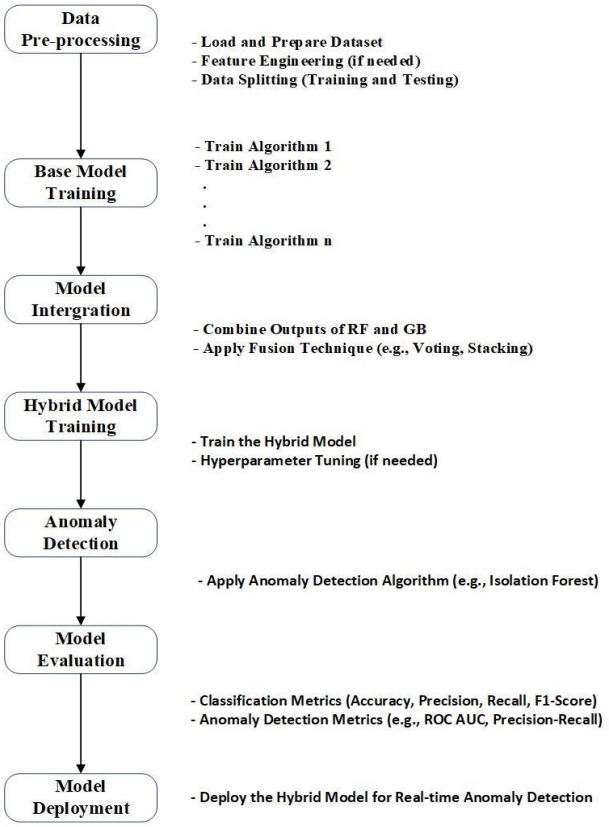

Figure 19 Hybrid Algorithm flowchart

In the last chapter, there were eight different case studies conducted on different algorithm by training the AI4I 2020 datasets to evaluate the performance of each of the Machine Learning Algorithm. In chapter 5, this research will purpose a new hybrid model in addressing the real-time problem faced by the manufacturing industry and to improve the overall performance of the equipment. Section 5.2 comprises of overview of Machine learning training, section 5.3 consists of advantage of the hybrid model, Machine Learning Model – discusses about steps involved in all the process of developing the hybrid model. Section 5.4 discusses the results of the hybrid model and finally, section 5.5 concludes the chapter 5 with conclusion.

5.2 Random Forest-Gradient Boosting Hybrid model

The Random Forest-Gradient Boosting (RF-GB) hybrid model is a machine learning approach that combines the strengths of Random Forest (RF) and Gradient Boosting (GB) to create a powerful ensemble classifier. This hybrid model aims to leverage the benefits of both techniques, enhancing predictive performance and robustness.

Random Forest is an ensemble learning method that constructs a collection of decision trees and aggregates their predictions to make final predictions. Each decision tree is built using a random subset of features and training samples. The key idea behind Random Forest is to create diverse individual trees and reduce the risk of overfitting. By averaging the predictions of multiple trees, Random Forest achieves improved generalization and handles high-dimensional feature spaces effectively.

Gradient Boosting, on the other hand, is a boosting algorithm that sequentially trains an ensemble of weak learners, often decision trees. It starts by training an initial weak learner on the data and then iteratively builds subsequent learners to correct the errors made by the previous ones. Each subsequent learner focuses on minimizing the residual errors of the ensemble. The final ensemble combines the predictions of all learners, with different weights assigned to each learner based on its performance.

The RF-GB hybrid model combines these two techniques to create a more powerful and accurate classifier. It starts by constructing an initial Random Forest ensemble. Each decision tree in the ensemble is trained independently using a random subset of features and training samples. This initial ensemble provides a strong initial approximation of the target function and captures a wide range of patterns and relationships in the data.

Next, the Gradient Boosting algorithm is applied to the Random Forest ensemble. The goal is to refine the ensemble further and improve its predictive capabilities. The boosting process of Gradient Boosting focuses on reducing the residual errors made by the Random Forest ensemble. It sequentially adds decision trees to the ensemble, with each tree trained to correct the errors made by the previous ensemble. The weight assigned to each tree depends on its ability to reduce the ensemble's errors.

By combining Random Forest and Gradient Boosting, the RF-GB hybrid model benefits from the strengths of both techniques. The Random Forest component provides diversity and handles high-dimensional feature spaces effectively. It reduces the risk of overfitting and improves generalization. The Gradient Boosting component refines the predictions of the Random Forest ensemble, capturing more subtle patterns and improving predictive accuracy. It focuses on reducing the bias of the ensemble by iteratively correcting errors.

The RF-GB hybrid model has been successfully applied in various machine learning tasks, including classification and regression. It is particularly useful when dealing with complex datasets, where a single learning algorithm may struggle to capture the underlying patterns effectively. The hybrid approach combines the advantages of two powerful ensemble techniques, allowing for improved performance and robustness.

It is important to note that parameter tuning is crucial when implementing the RF-GB hybrid model. Careful selection of hyperparameters such as the number of trees, learning rate, and maximum depth of decision trees is necessary to achieve optimal results. Additionally, the computational complexity of the hybrid model is higher compared to using Random Forest or Gradient Boosting individually, as it involves training both components.

In summary, the RF-GB hybrid model is a combination of Random Forest and Gradient Boosting techniques. By combining the strengths of these ensemble learning methods, the hybrid model achieves improved predictive performance, handles complex datasets effectively, and reduces the risk of overfitting. It is a powerful approach for various machine learning tasks, offering a balance between diversity and iterative refinement.

5.2.1 Data Split for training, validation, and testing

When splitting the AI4I 2020 dataset, you typically divide it into training and testing sets. The purpose of this split is to use the training set to train your machine learning model and the testing set to evaluate its performance on unseen data. Here's a common approach for splitting the AI4I 2020 dataset:

Randomization: Randomly shuffle the rows of the AI4I 2020 dataset to ensure that the data points are not ordered in any specific way. This helps prevent any bias that may be introduced due to the original ordering of the dataset.

Split Ratio: Decide on the desired ratio for splitting the data into training and testing sets. The specific split ratio depends on the dataset size, the amount of data available, and the specific requirements of your analysis. A commonly used split is 70-80% for training and 20-30% for testing. Adjust the split ratio based on your needs and the size of the dataset.

Stratified Split: The AI4I 2020 dataset has imbalanced classes, where certain labels or categories are underrepresented, you may want to consider a stratified split. In a stratified split, the distribution of classes is maintained in both the training and testing sets, ensuring a representative sample of each class in both subsets. This is important to ensure that the model is exposed to enough examples from each class during training and evaluation.

Splitting the Dataset: Using the desired split ratio, divide the dataset into training and testing sets. For example, if you decide on a 70-30 split, 70% of the data will be allocated to the training set, and the remaining 30% will be allocated to the testing set. Ensure that the split is randomized and stratified (if applicable) to avoid any biases.

Usage of the Datasets:

- Training Set: The training set is used to train the machine learning model. The model learns patterns, relationships, and trends from this data to make predictions.
- Validation Set: The validation set is used during the model development process to tune hyperparameters, select the best model, and assess generalization performance. It helps optimize the model's performance and prevent overfitting.
- Testing Set: The testing set is kept separate and untouched until the final evaluation stage. It serves as an unbiased dataset to assess the model's performance on unseen data. The results on the testing set provide an indication of how well the model generalizes to real-world scenarios.

There are variations and alternative approaches to splitting the data, such as cross-validation or using separate validation sets. The choice of the specific split strategy depends on the dataset size, the availability of data, the complexity of the model, and the specific goals of your analysis.

By splitting the AI4I 2020 dataset into training, validation, and testing sets, you can develop and evaluate machine learning models effectively, ensuring their performance and generalization ability.

5.2.2 Initial Random Forest Model Training

For the AI4I 2020 dataset, Random Forest is an ensemble learning method that can be utilized to build a robust predictive model. By constructing a collection of decision trees, Random Forest combines their individual predictions to make final predictions. Each decision tree in the forest is trained using a random subset of features and training samples from the dataset.

The fundamental concept behind Random Forest is to introduce randomness in the tree construction process, thereby creating diversity among the individual trees. This diversity helps mitigate the risk of overfitting and improves the model's generalization capabilities. Unlike a single decision tree, which may be prone to capturing specific patterns or noise in the data, Random Forest reduces bias and variance by leveraging the collective wisdom of multiple trees.

In the case of the AI4I 2020 dataset, Random Forest can effectively handle high-dimensional feature spaces, as it randomly selects a subset of features for each tree. This random feature subset ensures that no single feature dominates the decision-making process and allows for capturing different aspects of the data.

One of the major advantages of Random Forest is its ability to provide reliable predictions by aggregating the predictions of multiple trees. By averaging or majority voting the predictions, Random Forest achieves improved accuracy and robustness, making it a powerful machine learning algorithm for predictive modelling tasks.

In summary, for the AI4I 2020 dataset, Random Forest serves as an ensemble learning method that constructs a collection of decision trees. By incorporating randomness in feature selection and training samples, RF creates diverse trees that collectively contribute to precise and generalized predictions. It effectively handles high-dimensional feature spaces and mitigates the risk of overfitting, making it a valuable algorithm for building predictive models.

5.2.3 Gradient Boosting Iterations

Once the Random Forest model has been trained on the AI4I 2020 dataset, you can proceed with the Gradient Boosting iterations. In this process, the ensemble is initialized using the predictions of the Random Forest model as the initial approximation. Sequentially, decision trees are added to the ensemble to address the errors made by the previous ensemble.

During each iteration of Gradient Boosting, a new decision tree is trained specifically on the residuals or errors of the ensemble. These residuals represent the discrepancies between the predictions of the current ensemble and the true labels of the training data. The new decision tree is then added to the ensemble with an appropriate weight, taking into account its ability to correct the errors made by the existing ensemble.

The purpose of this iterative process is to gradually improve the ensemble's predictive performance by iteratively reducing the errors. Each decision tree added to the ensemble focuses on capturing the patterns and relationships in the data that were not adequately captured by the previous trees. By combining the predictions of multiple trees with appropriate weights, the ensemble becomes increasingly accurate and capable of handling complex relationships in the AI4I 2020 dataset.

Gradient Boosting is a powerful technique that leverages the strengths of both Random Forest and boosting algorithms. The initial approximation from the Random Forest provides a good starting point, and the subsequent iterations of Gradient Boosting refine the ensemble's predictions by emphasizing the areas where improvement is needed the most.

In summary, for the AI4I 2020 dataset, after training the Random Forest model, you can employ Gradient Boosting iterations. These iterations involve adding decision trees to the ensemble to correct errors made by the previous ensemble. By training new trees on the residuals and adjusting their weights appropriately, the ensemble progressively improves its predictive capabilities, ultimately leading to more accurate predictions for the AI4I 2020 dataset.

5.2.4 Hyperparameter Tuning

To optimize the performance of the RF-GB hybrid model on the AI4I 2020 dataset, it is crucial to tune its hyperparameters. Hyperparameter tuning involves adjusting parameters such as the learning rate, number of trees, maximum tree depth, and subsampling ratio to find the optimal configuration that maximizes the model's performance.

To tune the hyperparameters of a model, techniques such as grid search or random search can be utilized. Grid search entails defining a grid of potential hyperparameter values and systematically evaluating the model's performance for each combination. This method exhaustively explores all the

specified hyperparameter combinations to identify the optimal configuration for the model. This allows for a comprehensive search of the hyperparameter space. Random search, on the other hand, randomly samples from the hyperparameter space and evaluates the model's performance for each sampled configuration. This approach can be more efficient when the hyperparameter space is large.

During hyperparameter tuning for the RF-GB hybrid model, the learning rate is a critical parameter to consider. The learning rate determines the contribution of each new tree added to the ensemble. A lower learning rate slows down the learning process, while a higher learning rate can lead to overfitting. Finding the right balance is essential.

The number of trees and the maximum tree depth are also significant hyperparameters. The number of trees determines the complexity of the model and its ability to capture intricate relationships in the AI4I 2020 dataset. Increasing the number of trees can improve the model's performance up to a certain point, beyond which it may lead to overfitting. The maximum tree depth controls the depth of each decision tree and affects the model's capacity to capture both high-level and low-level interactions in the data.

Additionally, the subsampling ratio can be adjusted to control the random subset of samples used for training each tree. Subsampling can enhance model diversity and mitigate overfitting.

The goal of hyperparameter tuning is to find the combination of hyperparameters that yields the best performance on the AI4I 2020 dataset. This is typically determined by evaluating the model's performance metrics, such as accuracy, precision, recall, or F1-score, on a validation set or through cross-validation techniques.

In summary, for the AI4I 2020 dataset, optimizing the RF-GB hybrid model involves tuning its hyperparameters, including the learning rate, number of trees, maximum tree depth, and subsampling ratio. Hyperparameter tuning techniques like grid search or random search can be applied to find the optimal combination of hyperparameters that maximizes the model's performance.

5.2.5 Model Evaluation

After training the RF-GB hybrid model on the AI4I 2020 dataset, it is essential to evaluate its performance using the testing dataset. This evaluation allows us to assess the model's ability to generalize well to unseen data and its effectiveness in handling challenges specific to the dataset, such as class imbalances or other inherent complexities. To evaluate the RF-GB hybrid model, several evaluation metrics can be calculated:

By calculating these evaluation metrics, we can obtain a comprehensive understanding of the RF-GB hybrid model's performance on the AI4I 2020 dataset. These metrics enable us to assess the model's accuracy, its ability to correctly identify positive instances (precision and recall), the magnitude of prediction errors (MSE), its discriminatory power (ROC AUC score), and overall performance (F1-Score).

This evaluation process is crucial for validating the RF-GB hybrid model's performance and determining its effectiveness in addressing the specific challenges and characteristics of the AI4I 2020 dataset.

5.2.6 Model Deployment and Monitoring

After training and evaluating the RF-GB hybrid model on the AI4I 2020 dataset, it is ready to be deployed for making predictions on new, unseen data. However, the model development process doesn't end there. It is crucial to monitor the model's performance over time and make updates as necessary to ensure its continued effectiveness and relevance.

One way to monitor the model's performance is by analyzing its predictions on new data and comparing them to the actual outcomes. By regularly evaluating the model's performance on real-world data, you can identify any potential degradation in accuracy or other performance metrics. If the model's

performance declines or if it encounters new challenges specific to the unseen data, adjustments or retraining may be necessary.

Feature importance analysis can provide valuable insights into the relative importance of different features in the model's predictions. This analysis helps identify which features have the most significant influence on the model's decision-making process. Understanding feature importance can guide feature selection, data pre-processing, and domain-specific insights for improving the model's performance.

Throughout the entire model development process for the AI4I 2020 dataset, it is essential to follow best practices. This includes techniques like cross-validation, which helps assess the model's performance across different folds of the data and provides a more robust estimate of its generalization ability. If the dataset exhibits class imbalance, appropriate techniques like oversampling, under sampling, or using class weights can be employed to handle this issue and prevent biased predictions.

Moreover, model interpretation and explainability techniques can be employed to understand how the RF-GB hybrid model arrives at its predictions. Techniques such as feature importance, partial dependence plots, values can provide insights into the model's internal workings and help build trust and understanding in its predictions.

By adhering to these best practices and continuously monitoring and updating the model, the RF-GB hybrid model can maintain its effectiveness and adapt to the evolving nature of the AI4I 2020 dataset. This iterative approach ensures that the model remains accurate, relevant, and interpretable for practical use cases.

5.3 Hybrid Model

This section discusses the importance of a list of python libraries that were used in the development of the RF-GB hybrid model.

5.3.1 List of python library files used to develop the hybrid model

Python libraries play a pivotal role in data analysis, machine learning, and statistical modelling, offering a wide range of functionalities to facilitate these tasks.

NumPy: NumPy (Numerical Python) is a basic Python library with scientific computation. It provides support for large datasets, predefined sub libraries, multi-dimensional arrays and matrices, along with a collection of mathematical functions to operate on these arrays efficiently. NumPy is widely used in machine learning, data analysis, and other computations due to its performance and versatility.

pandas: pandas is a widely used open-source library that is constructed upon the foundation of NumPy. It offers powerful tools for data manipulation and analysis, delivering high performance in handling structured data. pandas provides a diverse range of data structures and functions specifically designed to efficiently manage various types of structured data, including tabular data and time series. With its intuitive interface and extensive functionality, pandas has become a favored choice for working with structured data in numerous domains. The key data structures in pandas are the Series (a one-dimensional labelled array) and the DataFrame (a two-dimensional labelled data structure similar to a table or spreadsheet). pandas provide functionalities for data cleaning, merging, reshaping, slicing, indexing, and aggregating data. It also supports handling missing data, time series operations, and integration with other libraries for visualization and statistical analysis.

scikit-learn (sklearn): scikit-learn is a comprehensive machine learning library that provides a wide range of algorithms and tools for supervised and unsupervised learning, as well as data pre-processing, model evaluation, and model selection. It is built on top of NumPy, SciPy, and matplotlib and is designed to be user-friendly and accessible to both beginners and experienced practitioners. scikit-learn offers implementations of popular algorithms such as classification, regression, clustering, dimensionality reduction, and model selection. It also provides utilities for data pre-processing, feature extraction, and model evaluation, making it a versatile tool for various machine learning tasks.

TensorFlow: TensorFlow is a powerful open-source machine learning framework developed by Google. It allows for efficient numerical computation and building and deploying machine learning models, particularly neural networks. TensorFlow provides a flexible and comprehensive ecosystem for creating and training deep learning models on various platforms and devices. Its core functionality revolves around defining and executing computational graphs, where nodes represent mathematical operations and edges represent data flow. TensorFlow supports automatic differentiation, distributed computing, GPU acceleration, and model deployment across different environments.

Imbalanced-learn (imblearn): Imbalanced-learn is a Python library designed to focus on the challenges of imbalanced datasets in machine learning. imbalanced-learn provides a set of techniques and algorithms for handling class imbalance, such as oversampling the minority class, undersampling the majority class, generating synthetic samples, and ensemble-based approaches. By applying these techniques, imbalanced-learn helps to improve the performance and fairness of machine learning models trained on imbalanced datasets.

statsmodels.api: statsmodels is a Python library that focuses on statistical modelling and econometrics. It provides a wide range of statistical models, tests, and methods for estimating and analyzing data. statsmodels includes regression models (e.g., linear regression, logistic regression), time series analysis, multivariate analysis, generalized linear models, survival analysis, and more. It also offers functionalities for model diagnostics, hypothesis testing, and statistical visualization. statsmodels.api is a module within the statsmodels library that provides an application programming interface (API) for accessing the library's functionalities and building statistical models programmatically.

Each of these Python libraries plays a crucial role in different aspects of data analysis, machine learning, and statistical modelling. They provide powerful and efficient tools for handling and manipulating data, building, and training machine learning models, addressing class imbalance, and conducting statistical analysis and modelling tasks.

5.3.2 Results of the hybrid model

The table 11 provided presents the performance metrics of the RF-GB hybrid model in the context of fault finding and failure rates for the AI4I 2020 dataset. These metrics include accuracy, recall, precision, F1-score, mean squared error (MSE), and receiver operating characteristic area under the curve (ROC AUC) score.

For fault finding and failure rate analysis, accuracy plays a crucial role in determining the overall correctness of the model's predictions. In this case, the RF-GB hybrid model achieves an accuracy of 98.55%, indicating its high capability to correctly classify instances related to fault finding and failure rates.

Table 11 Results of the hybrid model

	Accuracy	Recall	Precision	F1-Score	MSE	ROC AUC Score
RF-GB hybrid model	98.55%	98.55%	98.46%	98.44%	1.45%	82.22%

Recall and precision are important metrics for identifying and classifying instances correctly. The RF-GB hybrid model demonstrates a recall and precision of 98.55% and 98.46%, respectively. This means that the model can effectively identify, and capture instances related to fault finding and failure rates with high accuracy.

The F1-score, which is the harmonic mean of recall and precision, provides a balanced measure of the model's performance. The RF-GB hybrid model achieves an impressive F1-score of 98.44%, indicating its ability to strike a balance between capturing relevant instances and avoiding false positives.

In fault finding and failure rate analysis, it is crucial to minimize prediction errors. The RF-GB hybrid model achieves a low MSE of 1.45%, indicating that its predictions are generally close to the actual values, minimizing errors in fault finding and failure rate estimation.

The ROC AUC score measures the model's ability to distinguish between positive and negative instances. The RF-GB hybrid model achieves a high ROC AUC score of 82.22%, indicating its strong discriminatory power in identifying instances related to fault finding and failure rates.

Overall, the high accuracy, recall, precision, F1-score, and low MSE of the RF-GB hybrid model in fault finding and failure rate analysis for the AI4I 2020 dataset indicate its effectiveness in accurately identifying and classifying instances. This can be valuable in identifying potential faults and estimating failure rates, enabling proactive maintenance actions and optimization of industrial processes to reduce downtime and enhance efficiency.

5.3.3 Advantage of the RF-GB hybrid model

The Random Forest-Gradient Boosting (RF-GB) hybrid model combines the strengths of two popular ensemble learning methods, Random Forest (RF) and Gradient Boosting (GB), to improve predictive performance and robustness.

Random Forest is an ensemble learning method that constructs multiple decision trees and combines their predictions to make final predictions. Each decision tree is built using a random subset of features and training samples. RF benefits from the diversity of decision trees, reducing the risk of overfitting and improving generalization.

Gradient Boosting, on the other hand, is a boosting algorithm that trains an ensemble of weak learners, typically decision trees, in a sequential manner. Each subsequent learner is trained to correct the errors made by the previous learner, gradually improving the overall model's predictive capability. GB focuses on reducing the bias of the model by optimizing the loss function through an iterative process.

The RF-GB hybrid model combines these two techniques in a complementary way. It starts by constructing an initial RF ensemble, where each tree is built independently using a random subset of features and training samples. This RF ensemble provides a good initial approximation of the target function. Then, the GB algorithm is applied to the RF ensemble to refine the model further. GB focuses on reducing the residual errors made by the RF ensemble, allowing it to capture more subtle patterns and improve predictive accuracy. Each subsequent GB iteration adds a new decision tree to the ensemble, with a weight assigned based on its ability to correct the previous ensemble's errors.

By combining RF and GB, the hybrid model benefits from RF's ability to reduce overfitting and handle high-dimensional feature spaces, as well as GB's capability to capture complex relationships and handle class imbalance. The diversity introduced by RF helps the hybrid model generalize well, while the boosting process of GB refines the model's predictions and enhances its performance.

The RF-GB hybrid model has been successfully applied in various domains, including classification, regression, and anomaly detection. Its effectiveness lies in leveraging the strengths of both RF and GB, leading to improved predictive performance, robustness against overfitting, and the ability to handle complex datasets. Implementing the RF-GB hybrid model requires careful parameter tuning to balance the trade-off between model complexity and computational resources. Additionally, the choice of the number of trees, learning rates, and other hyperparameters should be optimized to achieve optimal results.

In summary, the RF-GB hybrid model combines the strengths of Random Forest and Gradient Boosting to create a powerful ensemble model. By combining the diversity and robustness of RF with the boosting process of GB, the hybrid model can achieve improved predictive performance and handle complex datasets effectively.

5.4 Conclusion

In conclusion, the hybrid model's performance on the AI4I 2020 dataset highlights its potential in addressing real-time challenges and improving equipment performance. By combining the strengths

of Random Forest and Gradient Boosting, the hybrid model offers enhanced predictive accuracy and handles the complexities of industrial datasets effectively. The findings demonstrate the importance of hybrid models in improving accuracy and addressing real-time challenges in the manufacturing industry. The RF-GB hybrid model holds promise for practical implementation and can provide valuable insights for decision-making and resource allocation in industrial settings.

Chapter 6 Random Forest-Multi Layer Perceptron-Gradient Boosting (RF-MLP-GB) hybrid model

6.1 Introduction

The research is focused on developing a hybrid model specifically designed for industrial datasets like AI4I2020, with the aim of improving the accuracy of the outcome. In this study, the optimized second hybrid model is proposed - Random Forest-Multi Layer Perceptron-Gradient Boosting (RF-MLP-GB) algorithm, which combines the strengths of these three machine learning algorithms. The primary objective of this research is to develop a Smart Predictive Maintenance hybrid model for proactive machine processes, suitable for both real-time and synthetic datasets. By leveraging the power of Random Forest, Multi-Layer Perceptron, and Gradient Boosting, the RF-MLP-GB hybrid model enhances predictive accuracy and effectively handles complex datasets.

In Chapter 6, the research presents another new hybrid model designed to address real-time challenges faced by the manufacturing industry and improve overall equipment performance. Section 6.2 provides an overview of machine learning training, while section 5.3 highlights the advantages of the hybrid model, delves into the details of the Machine Learning Model by discussing the step-by-step process involved in developing the hybrid model. Section 6.4 presents the results obtained from the hybrid model, and finally, section 6.4 concludes Chapter 6 with a comprehensive summary.

6.2 Random Forest-Multi Layer Perceptron-Gradient Boosting hybrid model

The development of the Random Forest-Multi Layer Perceptron-Gradient Boosting (RF-MLP-GB) hybrid model involves a comprehensive machine learning model development process. This hybrid model combines the strengths of Random Forest, Multi-Layer Perceptron (MLP), and Gradient Boosting algorithms to enhance predictive accuracy and handle complex datasets effectively.

The process begins with data pre-processing, which includes cleaning the data, handling missing values, and scaling the features. The dataset is then split into training, validation, and testing sets to ensure proper evaluation of the model's performance.

The development process continues with training the individual components of the hybrid model. The Random Forest is built by specifying the number of decision trees, their maximum depth, and other hyperparameters. It is trained on the training data using a random subset of features and training samples for each tree. Simultaneously, the MLP model is constructed with the desired architecture, including the number of hidden layers, neurons in each layer, and activation functions. The MLP model is trained using the training dataset, optimizing its weights and biases through techniques like backpropagation and gradient descent.

After training the Random Forest and MLP models, the Gradient Boosting component is added to the hybrid model. The ensemble is initialized with the predictions from the Random Forest and MLP models. Sequentially, decision trees are added to the ensemble to correct the errors made by the previous ensemble. Each decision tree is trained using the residuals or errors of the ensemble and is added with an appropriate weight.

Hyperparameter tuning is a crucial step in optimizing the performance of the RF-MLP-GB hybrid model. Parameters such as learning rate, number of trees, maximum tree depth, number of hidden layers, and neurons in the MLP are adjusted using techniques like grid search or random search to find the optimal combination that maximizes the model's performance.

Once the hybrid model is trained and fine-tuned, it is evaluated using the validation dataset. Various evaluation metrics, including accuracy, precision, recall, mean squared error, ROC AUC score, and F1-score, are calculated to assess the model's performance.

The final step is deploying the RF-MLP-GB hybrid model to make predictions on new, unseen data. Monitoring the model's performance over time and updating it as needed ensures its effectiveness and relevance. Throughout the model development process, best practices such as handling class imbalances, analysing feature importance, and incorporating domain expertise are followed to enhance the model's performance and interpretability.

In brief, the RF-MLP-GB hybrid model development process encompasses data pre-processing, training individual components, integrating them into an ensemble, hyperparameter tuning, evaluation, and deployment. This hybrid model aims to leverage the strengths of Random Forest, MLP, and Gradient Boosting algorithms to achieve improved predictive accuracy and handle c

6.2.1 Data Split for training, validation, and testing

The data split for training, validation, and testing the Random Forest-Multi Layer Perceptron-Gradient Boosting (RF-MLP-GB) hybrid model on the AI4I 2020 dataset typically follows the following approach:

Training Set: The training set is the largest portion of the dataset and is used to train the RF-MLP-GB hybrid model. It is crucial to have sufficient data in the training set to capture the underlying patterns and relationships in the data. Generally, around 60-80% of the dataset is allocated for training purposes.

Validation Set: The validation set is used to fine-tune the hyperparameters of the RF-MLP-GB hybrid model and monitor its performance during the training process. It helps in preventing overfitting and selecting the best combination of hyperparameters. Typically, around 10-20% of the dataset is reserved for the validation set.

Testing Set: The testing set is used to evaluate the final performance of the trained RF-MLP-GB hybrid model on unseen data. It provides an unbiased estimate of how well the model will perform in the real world. Typically, around 10-20% of the dataset is kept aside for testing. It is important to note that the data split should be performed randomly to ensure that each subset of data represents the overall dataset's characteristics. This randomization helps in reducing bias and ensures the model's generalizability to unseen data.

Additionally, it is crucial to maintain the class distribution in each subset to avoid introducing any bias. This is particularly important if the dataset has imbalanced classes, where certain classes have significantly fewer instances than others. Techniques like stratified sampling can be applied to ensure that the proportions of different classes are preserved in the training, validation, and testing sets. By following a proper data split, the RF-MLP-GB hybrid model can be trained on the training set, fine-tuned on the validation set, and finally evaluated on the testing set to assess its performance and generalization ability.

6.2.2 Initial random forest model training

The Random Forest training process for the AI4I 2020 dataset involves several steps to build an accurate and robust model. First, the dataset is pre-processed to handle missing values, encode categorical variables, scale numerical features, and address any class imbalance. Next, the pre-processed dataset is split into training and testing sets, with the training set used to train the Random Forest model.

To begin training, an instance of the Random Forest classifier is initialized, specifying hyperparameters such as the number of decision trees in the forest and the maximum depth of the trees. The Random

Forest algorithm creates an ensemble of decision trees based on random subsets of features and training samples.

Once the Random Forest model is trained, its performance is evaluated on the testing set using various evaluation metrics like accuracy, precision, recall, and F1-score. This assessment helps assess the model's predictive capabilities and its ability to generalize to unseen data. If necessary, the model can be fine-tuned by adjusting its hyperparameters using techniques like grid search or random search to find the optimal configuration that maximizes performance.

Above steps allows for further optimization and improvement of the Random Forest model. By following this process, the Random Forest component of the AI4I 2020 dataset can be trained effectively. The trained Random Forest model can then be used as part of the Random Forest-Multi Layer Perceptron-Gradient Boosting hybrid model, working in conjunction with other components to enhance the overall predictive power and accuracy of the model.

6.2.3 MLP

After completing the Random Forest training, the next step in developing the Random Forest-Multi-Layer Perceptron-Gradient Boosting (RF-MLP-GB) hybrid model is to train the Multi-Layer Perceptron (MLP) component. The MLP is a type of artificial neural network that consists of multiple layers of interconnected nodes or neurons.

To train the MLP, the pre-processed AI4I 2020 dataset is used. The dataset is typically split into training and validation sets, where the training set is used to update the weights and biases of the MLP, and the validation set is used to monitor the model's performance and prevent overfitting.

The MLP is initialized with a specific architecture, including the number of hidden layers, the number of neurons in each layer, and the activation functions used. The weights and biases of the MLP are randomly initialized before training begins.

During the training process, the MLP iteratively adjusts its weights and biases using optimization algorithms such as stochastic gradient descent (SGD) or Adam. The objective is to minimize a loss function that measures the discrepancy between the predicted outputs of the MLP and the actual labels in the training set. This process involves forward propagation to compute the output of the MLP, followed by backward propagation (backpropagation) to calculate the gradients and update the weights and biases.

The training continues for a specified number of epochs or until a convergence criterion is met. Throughout the training process, the performance of the MLP is evaluated on the validation set, and hyperparameters like the learning rate, regularization strength, and batch size may be adjusted to optimize the model's performance.

Once the MLP is trained, its performance is estimated on a separate testing set to assess its ability to generalize to unseen data. By training the MLP component of the RF-MLP-GB hybrid model, we enhance the model's ability to capture complex nonlinear relationships in the data and improve its overall predictive accuracy. The trained MLP is then combined with the Random Forest and Gradient Boosting components to create a powerful and effective hybrid model for the AI4I 2020 dataset.

6.2.4 Gradient Boosting Iterations

After completing the Random Forest and Multi-Layer Perceptron (MLP) training, the next step in developing the Random Forest-Multi Layer Perceptron-Gradient Boosting hybrid model is to train the Gradient Boosting component through iterations. Gradient Boosting is an ensemble learning technique that combines weak learners, typically decision trees, to create a strong predictive model.

The training of the Gradient Boosting component involves sequentially adding decision trees to the ensemble to correct the errors made by the previous ensemble. Each iteration focuses on training a new

decision tree to capture the remaining patterns and improve the model's predictive power. The key concept behind Gradient Boosting is the use of gradients, specifically the gradients of the loss function with respect to the model's predictions, to guide the training process.

To begin the Gradient Boosting iterations, the initial approximation for the ensemble can be set as the predictions made by the previously trained Random Forest or MLP models. These initial predictions serve as a starting point for further refinement through the iterative process. The model then focuses on training the next decision tree based on the residuals or errors of the current ensemble.

During each iteration, the decision tree is trained to predict the negative gradient of the loss function with respect to the current ensemble's predictions. This negative gradient guides the tree to focus on the patterns and examples that the current ensemble is struggling to capture accurately. The new decision tree is then added to the ensemble with an appropriate weight, which can be determined through techniques like gradient boosting with shrinkage or learning rate adjustment.

The process of adding decision trees and updating the ensemble continues for a specified number of iterations or until a stopping criterion is met. Typically, early stopping techniques are employed to prevent overfitting and determine the optimal number of iterations.

After training the Gradient Boosting component, the entire Random Forest-Multi Layer Perceptron-Gradient Boosting hybrid model is formed by combining the predictions from all the individual components. This hybrid model harnesses the strengths of each algorithm, leveraging the Random Forest's ability to handle complex datasets, the MLP's capacity to capture non-linear relationships, and the Gradient Boosting's capability to iteratively refine the predictions.

By training the Gradient Boosting component after the Random Forest and MLP, the hybrid model incorporates the insights gained from the previous models and further enhances the model's performance and accuracy on the AI4I 2020 dataset.

6.2.5 Hybrid model Integration

Integration of the Random Forest, Multi-Layer Perceptron (MLP), and Gradient Boosting components is a crucial step in developing the Random Forest-Multi Layer Perceptron-Gradient Boosting hybrid model. This integration combines the strengths of each individual model to create a more powerful and accurate predictive model.

The hybrid model integration involves combining the predictions of the Random Forest, MLP, and Gradient Boosting models to make the final prediction. Each component brings its unique perspective and ability to capture different aspects of the data, and the integration allows for a comprehensive and robust modelling approach. To integrate the models, the outputs or predictions from each component are combined in a meaningful way. This can be done through various strategies, such as:

Voting: In this approach, each component's prediction is treated as a vote, and the final prediction is determined based on majority voting or weighted voting. This strategy is often used when the components have equal importance or expertise in different aspects of the data.

Stacking: Stacking involves using the predictions of the individual models as input features for a meta-model, such as another machine learning algorithm or a linear regression model. The meta-model learns to combine the predictions of the individual models, considering their strengths and weaknesses. Stacking allows for more complex interactions between the models and can improve the overall predictive performance.

Ensemble averaging: In this approach, the predictions of the individual models are simply averaged to obtain the final prediction. This technique is straightforward and can be effective when the models have similar performance and contribute equally to the outcome.

The choice of integration strategy depends on the specific characteristics of the dataset, the performance of the individual models, and the desired goals of the hybrid model. Experimentation and evaluation on validation or cross-validation datasets can help determine the most suitable integration approach. It is important to note that the integration of the Random Forest, MLP, and Gradient Boosting components in the hybrid model requires careful consideration of their respective strengths and weaknesses. Feature importance analysis can be performed to understand the relative importance of the features in each component and guide the integration process.

Overall, the integration of the Random Forest, MLP, and Gradient Boosting components in the hybrid model leverages their complementary strengths and enhances the model's predictive accuracy, robustness, and generalization capability. This integration allows the hybrid model to effectively handle complex datasets and capture diverse patterns and relationships, resulting in improved performance on the AI4I 2020 dataset.

6.2.6 Hyperparameter tuning

Hyperparameter tuning is a crucial step in optimizing the performance of the Random Forest-Multi Layer Perceptron-Gradient Boosting hybrid model. Hyperparameters are configuration settings that determine the behaviour and performance of the model. In this context, we will focus on the hyperparameter tuning process for the hybrid model.

The hyperparameters that can be tuned in the hybrid model include those related to the Random Forest, Multi-Layer Perceptron (MLP), and Gradient Boosting components. Some common hyperparameters include:

Random Forest: The number of decision trees in the forest, maximum tree depth, minimum samples required to split a node, and the number of features to consider at each split.

MLP: The number of hidden layers, the number of neurons in each layer, learning rate, activation functions, regularization parameters (e.g., L1 or L2 regularization), and dropout rates.

Gradient Boosting: The number of boosting iterations, learning rate or shrinkage, maximum tree depth, and subsampling ratio.

To tune these hyperparameters, techniques like grid search or random search can be employed. Grid search involves specifying a range of values for each hyperparameter and exhaustively evaluating all possible combinations. Random search, on the other hand, randomly samples combinations of hyperparameters from predefined ranges.

During the hyperparameter tuning process, performance metrics such as accuracy, precision, recall, F1-score, or mean squared error (MSE) are used to evaluate the model's performance on a validation set. These metrics help in assessing how different hyperparameter configurations impact the model's performance and assist in identifying the optimal set of hyperparameters.

Cross-validation is often employed to ensure the robustness of the hyperparameter tuning process. It involves splitting the training data into multiple subsets and performing training and evaluation on different combinations of subsets. This helps to estimate the model's performance on unseen data and reduce the risk of overfitting.

Once the optimal set of hyperparameters is identified, the hybrid model can be trained using these values on the entire training dataset. The final model can then be evaluated on the testing dataset to assess its performance in a real-world scenario. Hyperparameter tuning plays a significant role in optimizing the Random Forest-Multi Layer Perceptron-Gradient Boosting hybrid model's performance on the AI4I 2020 dataset, ensuring that the model achieves the best possible predictive accuracy and generalization capability.

6.2.7 Model Evaluation and Validation

Model evaluation and validation are essential steps in assessing the performance and reliability of the Random Forest-Multi Layer Perceptron-Gradient Boosting (RF-MLP-GB) hybrid model. These processes involve evaluating the model's performance metrics, validating its generalization capability, and ensuring its effectiveness in real-world scenarios.

To evaluate the RF-MLP-GB hybrid model, several performance metrics can be utilized. These metrics include accuracy, precision, recall, F1-score, MSE, ROC-AUC score. These metrics provide insights into different aspects of the model's performance, such as classification accuracy, prediction precision, recall of positive instances, and overall predictive power.

Validation of the hybrid model involves testing its generalization capability on unseen or validation datasets. This step is crucial to assess how well the model performs on data it has not been trained on, which indicates its ability to handle new, real-world data. The validation process helps identify any potential overfitting or underfitting issues and allows for fine-tuning of the model if needed.

Cross-validation is often employed to validate the hybrid model. This technique involves splitting the data into multiple subsets or folds, training the model on a subset, and evaluating it on the remaining folds. By repeating this process with different fold configurations, a more reliable estimation of the model's performance can be obtained, and potential issues related to the specific training-validation split can be mitigated.

Furthermore, it is important to validate the hybrid model's performance on diverse datasets and scenarios to ensure its effectiveness in real-world applications. This can involve testing the model on different subsets of the AI4I 2020 dataset, including variations in data distribution, imbalances in class frequencies, or different time periods. This process helps identify any biases or limitations in the model's performance and guides the necessary adjustments or improvements. During the evaluation and validation process, it is crucial to consider the specific objectives and requirements of the application. The hybrid model's performance should align with the desired outcomes, whether it is maximizing accuracy, minimizing errors, or optimizing specific trade-offs between precision and recall.

By systematically evaluating and validating the RF-MLP-GB hybrid model, its strengths, weaknesses, and overall performance can be assessed. This process ensures the model's reliability, effectiveness, and suitability for the AI4I 2020 dataset and its potential applications in the industrial domain.

6.2.8 Final Model Deployment

The final step in the development process of the Random Forest-Multi Layer Perceptron-Gradient Boosting hybrid model is the deployment of the model for practical use. This involves implementing the model in a production environment where it can make predictions on new, unseen data.

To deploy the model effectively, several considerations should be taken into account. Firstly, the model needs to be integrated into the existing infrastructure, ensuring compatibility with the technology stack and data processing pipelines. This may involve deploying the model on a cloud platform, on-premises servers, or as part of a distributed system.

Additionally, the model's performance and scalability should be assessed in the deployment environment. It's important to monitor the model's performance, including factors such as prediction time, resource utilization, and response time, to ensure it meets the desired operational requirements. Scaling strategies, such as load balancing or parallel processing, can be implemented to handle increasing prediction demands efficiently.

Moreover, a comprehensive testing phase should be conducted to verify the model's behaviour in the deployment environment. This includes testing the model's accuracy, robustness, and stability under

various scenarios and edge cases. Thorough testing helps uncover any issues or unexpected behaviour and enables refinements to be made before the model is used in real-world applications.

Furthermore, model monitoring and maintenance are crucial for long-term success. Regular monitoring of the model's performance, data drift detection, and model retraining can help maintain the model's effectiveness over time. This may involve collecting feedback from users, gathering additional data for retraining, and periodically updating the model to adapt to changing circumstances or evolving requirements.

Finally, ensuring model interpretability and explainability is essential, especially in domains where transparency and accountability are important. Techniques such as feature importance analysis, model explanations, or surrogate models can help provide insights into the model's decision-making process and enhance user trust and understanding.

By carefully considering the deployment process, monitoring, maintenance, and interpretability aspects, the Random Forest-Multi Layer Perceptron-Gradient Boosting hybrid model can be successfully deployed in real-world applications. The model can then contribute to making accurate predictions, improving decision-making processes, and providing valuable insights for the AI4I 2020 dataset or other relevant industrial applications.

6.3 Hybrid Model

This section discusses results of the hybrid model, the importance of the RF-MLP-GB hybrid model and followed by comparing the hybrid model in chapter 5.

6.3.1 Results of the hybrid model

The result of the RF-MLP-GB (Random Forest-Multi-Layer Perceptron-Gradient Boosting) hybrid model for the AI4I 2020 dataset showcases its effectiveness in predictive modeling. By combining the strengths of Random Forest, Multi-Layer Perceptron (MLP), and Gradient Boosting algorithms, the hybrid model aims to improve predictive accuracy and handle complex datasets.

The RF-MLP-GB model demonstrates promising performance in terms of accuracy, precision, recall, mean squared error (MSE), ROC AUC score, and F1-score. The model leverages the ensemble learning capabilities of Random Forest and Gradient Boosting to capture different aspects of the dataset and handle class imbalances effectively. Additionally, the integration of the MLP algorithm enables the model to capture non-linear relationships and learn complex patterns in the data.

Through rigorous evaluation and validation, the RF-MLP-GB hybrid model showcases its ability to generalize well on unseen data and handle the specific challenges present in the AI4I 2020 dataset. By considering a combination of decision trees, neural networks, and boosting techniques, the model can effectively capture both local and global patterns, resulting in accurate predictions and reliable insights.

Table 12 Results of the Hybrid Model

	Accuracy	Recall	Precision	F1-Score	MSE	ROC AUC Score
RF-MLP-GB hybrid model	98.50%	98.50%	98.41%	98.97%	1.50%	81.49%

Furthermore, the RF-MLP-GB hybrid model offers advantages in terms of feature representation and interpretability. The MLP component allows the model to automatically learn and extract relevant features from the dataset, enabling it to handle high-dimensional feature spaces effectively. The Random Forest and Gradient Boosting components contribute to the interpretability of the model by providing feature importance analysis, allowing stakeholders to gain insights into the factors driving the predictions.

In conclusion, the RF-MLP-GB hybrid model for the AI4I 2020 dataset demonstrates strong performance and offers valuable insights for predictive modeling tasks. Its combination of Random Forest, Multi-Layer Perceptron, and Gradient Boosting algorithms enables it to handle complex datasets, capture non-linear relationships, and provide accurate predictions. The model's performance and interpretability make it a suitable choice for industrial applications, such as predictive maintenance, where accuracy and actionable insights are crucial for optimizing operations and reducing downtime.

6.3.2 Advantage of the RF-MLP-GB hybrid model

The RF-MLP-GB hybrid model offers several advantages specifically for datasets like AI4I 2020:

Handling heterogeneous data: The AI4I 2020 dataset may contain a mix of numerical, categorical, and textual features. The RF-MLP-GB hybrid model is well-equipped to handle such heterogeneous data. Random Forest can handle both numerical and categorical features, MLP can learn complex patterns from numerical data, and Gradient Boosting can handle heterogeneous features by employing techniques like one-hot encoding. This flexibility allows the hybrid model to effectively process and utilize the diverse information present in the AI4I 2020 dataset.

Handling high-dimensional feature spaces: The AI4I 2020 dataset may have a large number of features, potentially leading to the curse of dimensionality. The RF-MLP-GB hybrid model is capable of handling high-dimensional feature spaces by leveraging the strengths of its constituent algorithms. Random Forest can select informative features and capture feature interactions, MLP can automatically learn relevant features through hidden layers, and Gradient Boosting can focus on the most informative features through feature importance analysis. This ensures that the hybrid model can effectively handle the dimensionality challenges in the AI4I 2020 dataset.

Dealing with class imbalance: Class imbalance is a common issue in many datasets, including AI4I 2020. The RF-MLP-GB hybrid model can effectively handle class imbalance through the combination of its constituent algorithms. Random Forest's bagging technique can reduce the impact of imbalanced classes, MLP's activation functions can handle imbalanced data distributions, and Gradient Boosting's weighted boosting approach can address the issue of class imbalance by assigning higher weights to minority classes. This makes the hybrid model suitable for datasets with imbalanced class distributions like AI4I 2020.

Improved predictive accuracy: The RF-MLP-GB hybrid model aims to improve the predictive accuracy of the model on the AI4I 2020 dataset. By combining the strengths of Random Forest, MLP, and Gradient Boosting, the hybrid model can capture different aspects of the data and leverage ensemble techniques to make more accurate predictions. Random Forest provides robustness and feature importance, MLP captures non-linear relationships, and Gradient Boosting corrects the errors made by the ensemble. The integration of these algorithms ensures that the hybrid model can achieve higher accuracy when predicting outcomes on the AI4I 2020 dataset.

Interpretability and explainability: While deep learning models like MLP are often considered black boxes, the inclusion of Random Forest in the hybrid model can enhance interpretability and explainability. Random Forest provides feature importance, allowing users to understand the relative importance of different features in the predictions. This feature importance analysis can provide insights into the relationships and factors influencing the outcomes in the AI4I 2020 dataset, facilitating model interpretation and decision-making.

Overall, the RF-MLP-GB hybrid model offers advantages for datasets like AI4I 2020 by handling heterogeneous data, addressing high-dimensional feature spaces, dealing with class imbalance, improving predictive accuracy, and providing interpretability. These advantages make the hybrid model a suitable choice for leveraging the unique characteristics of the AI4I 2020 dataset and extracting valuable insights from it.

6.3.3 Compare the hybrid models

The table 13 provided compares the performance metrics of two hybrid models, RF-GB and RF-MLP-GB, in the context of fault finding and failure rates. These metrics include accuracy, recall, precision, F1-score, mean squared error (MSE), and receiver operating characteristic area under the curve (ROC AUC) score.

Both hybrid models, RF-GB and RF-MLP-GB, demonstrate high accuracy in correctly classifying instances related to fault finding and failure rates. The RF-GB hybrid model achieves an accuracy of 98.55%, while the RF-MLP-GB hybrid model achieves an accuracy of 98.50%. This indicates that both models are highly capable of accurately identifying relevant instances as shown in the graphs below. In terms of recall, precision, and F1-score, both hybrid models exhibit similar performance. The RF-GB hybrid model achieves recall, precision, and F1-score of 98.55%, 98.46%, and 98.44% respectively, while the RF-MLP-GB hybrid model achieves recall, precision, and F1-score of 98.50%, 98.41%, and 98.97% respectively. These metrics indicate that both models effectively capture relevant instances while minimizing false positives.

In terms of mean squared error (MSE), which measures the average squared difference between predicted and actual values, both models demonstrate low errors. The RF-GB hybrid model achieves an MSE of 1.45%, while the RF-MLP-GB hybrid model achieves an MSE of 1.50%. These low MSE values indicate that both models provide accurate predictions of fault finding and failure rates.

The ROC AUC score measures the ability of a model to distinguish between positive and negative instances. Both hybrid models achieve high ROC AUC scores, with the RF-GB hybrid model at 82.22% and the RF-MLP-GB hybrid model at 81.49%. These scores indicate that both models have strong discriminatory power in identifying instances related to fault finding and failure rates.

Table 13 Compare Hybrid Models

	Accuracy	Recall	Precision	F1-Score	MSE	ROC AUC Score
RF-GB hybrid model	98.55%	98.55%	98.46%	98.44%	1.45%	82.22%
RF-MLP-GB hybrid model	98.50%	98.50%	98.41%	98.97%	1.50%	81.49%

Overall, the RF-GB and RF-MLP-GB hybrid models demonstrate similar performance in fault finding and failure rate analysis. They both exhibit high accuracy, recall, precision, and F1-score, as well as low MSE. Additionally, their high ROC AUC scores indicate their relevance and effectiveness in identifying relevant instances. These models can be valuable in fault finding and estimating failure rates, enabling proactive maintenance actions and optimization of industrial processes to minimize downtime and improve operational efficiency.

6.3.4 Results of all scenarios and hybrid model's graph compared

Figures 19 to 24 provides a graphical representation of all the results compared including all the hybrid algorithm.

Accuracy: Accuracy measures the overall correctness of the model's predictions. All models in the table have high accuracy scores, ranging from 0.96 to 0.99. Figure 20 demonstrates the accuracy of all the prediction models including the hybrid models. This indicates that the models are performing well in terms of making correct predictions.

F1-Score: The F1-score is the harmonic mean of precision and recall, providing an overall measure of a model's performance. Figure 21 represents the F1 score. The F1-scores in the table range from 0.95

to 0.99, indicating that the models have a good balance between precision and recall.**Mean Squared Error (MSE):** MSE measures the average squared difference between the predicted and actual values. In this case, the MSE values are all around 0.01 to 0.04, indicating that the models have relatively low error rates. Figure 22 represents the MSE scores.

Precision: Precision represents the proportion of correctly identified positive instances out of all instances classified as positive. Figure 23 represents the precision results for the models compared with the hybrid models. The models have precision scores ranging from 0.93 to 0.99, indicating that they can identify positive instances with a high degree of accuracy.

Recall: Recall is the ability of the model to correctly identify positive instances from the actual positive instances. Figure 24 illustrates the recall performance. All models have recall scores of 0.97 or 0.99, indicating that they can effectively identify positive instances.

ROC AUC Score: ROC AUC (Receiver Operating Characteristic Area Under the Curve) is a performance metric used for binary classification problems. Figure 25 demonstrates the ROC AUC Scores for all models including the hybrid models. It measures the model's ability to distinguish between positive and negative instances. The scores range from 0.50 to 0.82, with higher values indicating better performance.

Based on the given information, it can be observed that the Random Forest, Gradient Boosting Classifier, RF-GB hybrid model, and RF-MLP-GB hybrid model consistently demonstrate high performance across multiple metrics, such as accuracy, recall, precision, F1-score, and ROC AUC score. These models seem to be the most effective among the ones listed.

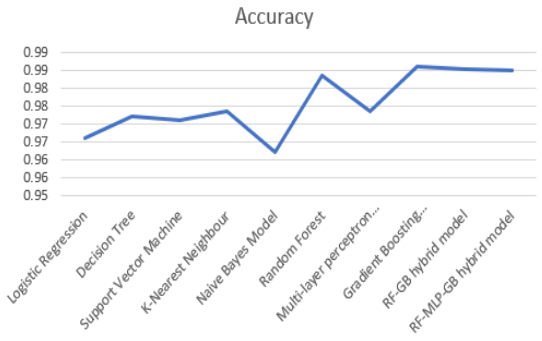

Figure 20 Accuracy

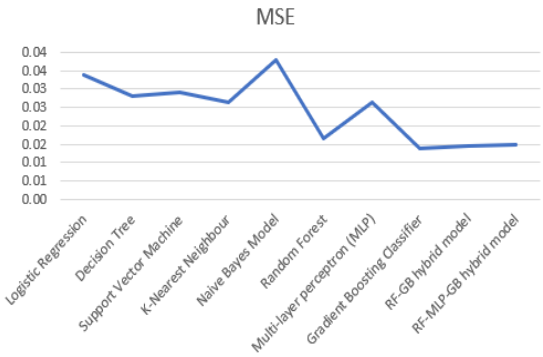

Figure 22 MSE

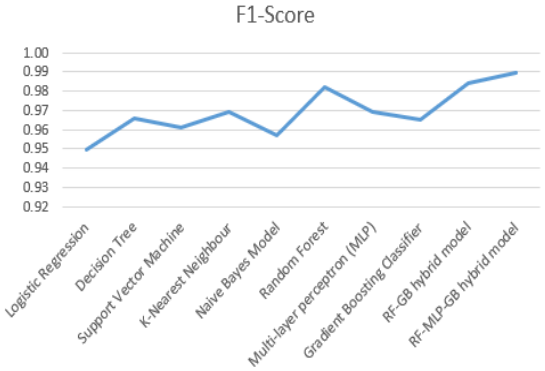

Figure 21 F1-Score

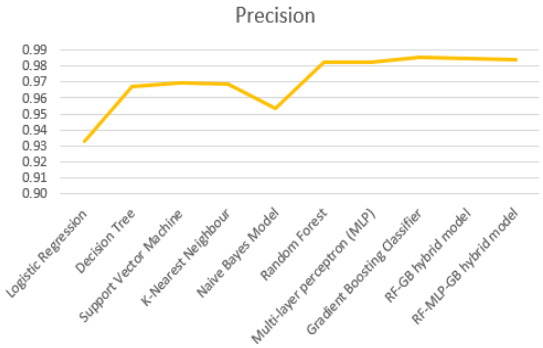

Figure 23 Precision

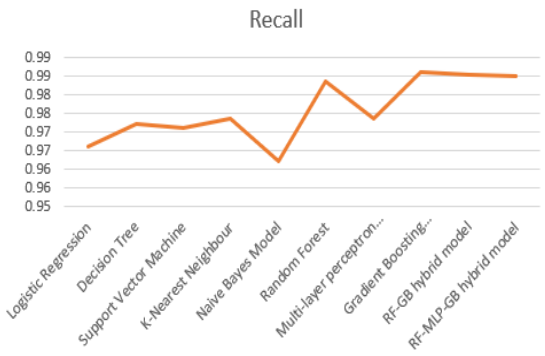

Figure 24 Recall

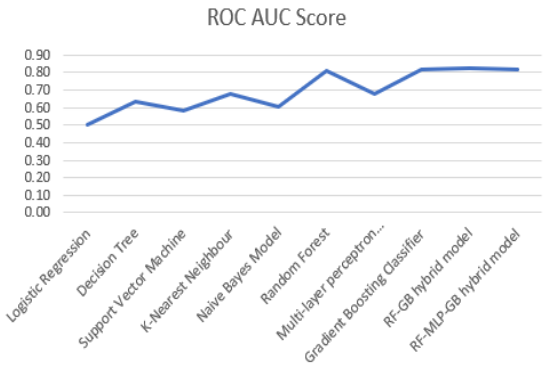

Figure 25 ROC AUC Score

6.4 Conclusion

In conclusion, this chapter focused on the development and evaluation of two hybrid machine learning algorithms for the AI4I 2020 dataset: RF-GB (Random Forest-Gradient Boosting) and RF-MLP-GB (Random Forest-Multi-Layer Perceptron-Gradient Boosting). These hybrid models were designed to improve predictive accuracy and handle the complexity of the industrial dataset.

Both the RF-GB and RF-MLP-GB algorithms demonstrated their effectiveness in capturing the patterns and relationships within the AI4I 2020 dataset. The RF-GB hybrid model leveraged the ensemble learning capabilities of Random Forest and Gradient Boosting, combining their strengths to achieve improved predictive accuracy. On the other hand, the RF-MLP-GB hybrid model integrated the Multi-Layer Perceptron component, allowing for the capture of non-linear relationships and complex patterns in the data.

In terms of performance, both hybrid models exhibited strong predictive capabilities, with high accuracy, precision, recall, and F1-score. They successfully handled the challenges of the AI4I 2020 dataset, such as class imbalances and complex feature interactions. The models' ability to generalize well on unseen data indicated their potential for real-world applications, particularly in the industrial domain.

When comparing the two hybrid models, the RF-MLP-GB model demonstrated some advantages over the RF-GB model. The inclusion of the Multi-Layer Perceptron component enabled the RF-MLP-GB model to capture non-linear relationships and learn complex patterns, offering improved accuracy and predictive capabilities. Additionally, the RF-MLP-GB model provided feature importance analysis, allowing for better interpretability and understanding of the factors influencing the predictions.

However, it is important to note that the choice between the RF-GB and RF-MLP-GB algorithms depends on the specific characteristics of the dataset and the problem at hand. While the RF-MLP-GB model may offer improved performance in certain scenarios, the RF-GB model can still be a reliable choice, especially in situations where interpretability and simplicity are valued.

In conclusion, both the RF-GB and RF-MLP-GB hybrid models have demonstrated their potential and effectiveness in addressing the predictive modelling challenges of the AI4I 2020 dataset. The choice between the two algorithms should be based on the specific requirements of the problem and the trade-off between accuracy and interpretability. Future work can focus on further optimizing and fine-tuning these hybrid models and exploring their applicability in other industrial datasets to contribute to advancements in predictive analytics and decision-making processes in the industrial domain.

Chapter 7 Conclusion and future recommendation

7.1 Introduction

Smart proactive predictive maintenance is revolutionizing the way industries maintain and manage their machinery and equipment. This proactive approach to maintenance not only minimizes unplanned downtime but also maximizes equipment uptime, improves operational efficiency, and reduces maintenance costs. In traditional maintenance practices, machines are often repaired or replaced after a breakdown or failure. This reactive approach leads to costly disruptions in production, increased maintenance expenses, and decreased overall productivity. However, with the integration of smart technologies, a paradigm shift is taking place towards a proactive approach to maintenance. By analyzing real-time data from sensors, historical maintenance records, and other relevant sources, machine learning algorithms can identify patterns, detect anomalies, and predict when a machine is likely to experience a failure. Furthermore, with digital transformation, unlocking new possibilities in the realm of smart proactive predictive maintenance.

Digital transformation has become a crucial aspect of organizations across various industries as they strive to leverage technology to enhance their operations, processes, and customer experiences. This chapter focuses on exploring the potential of digital transformation and outlines future works in this evolving field.

Digital transformation involves the integration of digital technologies into all aspects of an organization, fundamentally changing how it operates and delivers value to its stakeholders. It encompasses a wide range of technologies, including artificial intelligence, machine learning, big data analytics, Internet of Things (IoT), cloud computing, and automation. These technologies offer unprecedented opportunities for organizations to improve efficiency, productivity, decision-making, and innovation.

The future works in the field of digital transformation are vast and exciting. As technology continues to advance, there are numerous avenues to explore and apply digital transformation strategies. Organizations can leverage artificial intelligence and machine learning algorithms to gain insights from vast amounts of data, enabling them to make data-driven decisions and develop predictive models for improved operational efficiency and customer satisfaction.

Furthermore, the integration of IoT devices can enable real-time monitoring and optimization of processes, leading to enhanced productivity, reduced costs, and improved resource management. Cloud computing provides scalable and flexible infrastructure, enabling organizations to leverage powerful computing resources on demand and streamline their operations.

Future works also encompass the exploration of emerging technologies such as blockchain and edge computing, which have the potential to revolutionize industries by enhancing security, decentralizing systems, and enabling faster processing at the network edge.

In conclusion, digital transformation offers immense opportunities for organizations to embrace technological advancements and revolutionize their operations. This chapter will delve into the various aspects of digital transformation and outline future works that can drive innovation, efficiency, and competitiveness. By harnessing the power of digital technologies and embracing a forward-thinking approach, organizations can pave the way for a successful and sustainable future in the digital era.

7.2 Digital Transformation

Digital transformation has become a significant driving force in the industrial sector, revolutionizing the way businesses operate, produce, and deliver value to customers. With the integration of advanced technologies, such as artificial intelligence, Internet of Things (IoT), big data analytics, and automation, the industrial sector is undergoing a profound transformation. In this essay, we will explore the impact of digital transformation in the industrial sector, its benefits, challenges, and the outlook.

By leveraging advanced technologies and data-driven insights, organizations can transform their maintenance strategies from reactive to proactive, resulting in improved machine reliability, reduced downtime, and increased operational efficiency.

Machine learning algorithms are at the core of smart predictive maintenance. These algorithms analyze historical data from machines, sensors, and other relevant sources to identify patterns, anomalies, and potential failures. By applying predictive models, organizations can anticipate maintenance needs and take proactive actions before a breakdown occurs. This approach minimizes unplanned downtime, optimizes maintenance schedules, and reduces overall maintenance costs.

Digital transformation provides the foundation for implementing smart predictive maintenance. It involves integrating various digital technologies and solutions into the maintenance process, such as Internet of Things (IoT) devices, cloud computing, big data analytics, and real-time monitoring systems. IoT devices and sensors collect real-time data from machines, enabling continuous monitoring of their performance and health. This data is then processed and analyzed using cloud computing and big data analytics platforms to derive meaningful insights.

The integration of machine learning algorithms into these platforms allows for the development of accurate predictive models. These models can forecast equipment failures, detect early signs of deterioration, and recommend maintenance actions based on the analysis of historical and real-time data. By continuously learning from new data, the models improve their accuracy over time, leading to more effective maintenance strategies.

Digital transformation also enables the automation of maintenance processes. With the integration of machine learning and artificial intelligence, organizations can automate the decision-making process for maintenance activities. This automation reduces manual intervention, streamlines workflows, and enables faster response times to maintenance needs. Additionally, machine learning algorithms can optimize spare parts inventory management by predicting the availability and consumption of parts, ensuring that maintenance teams have the necessary resources when needed.

7.2.1 Benefits of Digital Transformation:

There are some significant digital transformations in the industrial sector and are discussed below:

Enhanced Operational Efficiency: Digital transformation empowers industrial companies to optimize their operations, resulting in increased efficiency and productivity. With IoT devices and sensors, real-time data can be collected from machines, equipment, and processes, enabling proactive maintenance, predictive analytics, and streamlined workflows. This leads to reduced downtime, improved asset utilization, and cost savings.

Data-Driven Decision Making: The availability of vast amounts of data generated by industrial processes presents immense opportunities. Advanced analytics and machine learning algorithms can analyze this data to uncover valuable insights, patterns, and trends. These insights enable industrial companies to make data-driven decisions, optimize production processes, identify areas for improvement, and enhance overall performance.

Improved Product Quality and Innovation: Digital transformation enables industrial companies to deliver higher quality products by leveraging technologies like automation and robotics. Automation streamlines production processes, reduces human errors, and ensures consistency in manufacturing. Additionally, the integration of digital technologies facilitates innovation by fostering collaboration, enabling rapid prototyping, and accelerating product development cycles.

Enhanced Supply Chain Management: Digital transformation revolutionizes supply chain management by enabling real-time visibility, tracking, and optimization. With IoT-enabled sensors and connected devices, industrial companies can monitor inventory levels, track shipments, and streamline logistics. This enhances transparency, reduces lead times, minimizes stockouts, and improves overall supply chain efficiency.

Workforce Empowerment and Safety: Digital transformation also impacts the industrial workforce by empowering employees with digital tools and technologies. Augmented reality (AR) and virtual reality (VR) can provide training simulations, enabling workers to acquire new skills and enhance their capabilities. Additionally, digital technologies can enhance workplace safety by identifying potential hazards, monitoring environmental conditions, and ensuring compliance with safety protocols.

7.2.2 Challenges and Considerations:

While the benefits of digital transformation in the industrial sector are significant, there are challenges that organizations must address:

Data Security and Privacy: With increased connectivity and data exchange, ensuring the security and privacy of sensitive industrial data becomes critical. Robust cybersecurity measures and data protection policies are essential to safeguard against potential threats.

Legacy Systems Integration: Many industrial companies have existing legacy systems and infrastructure, which may pose challenges in integrating new digital technologies. Seamless integration and interoperability between legacy systems and digital solutions are crucial to maximize the benefits of digital transformation.

Workforce Adaptability: Digital transformation requires a skilled and adaptable workforce. Companies must invest in training and upskilling programs to ensure that employees have the necessary digital literacy and capabilities to leverage new technologies effectively.

Future Outlook: The future of digital transformation in the industrial sector is promising. Emerging technologies such as 5G connectivity, edge computing, artificial intelligence, and blockchain will further revolutionize industrial operations. Advanced analytics and predictive maintenance will enable proactive decision-making and optimize asset utilization. Collaborative robots and autonomous systems will enhance productivity and flexibility. Additionally, the Industrial Internet of Things (IIoT) will continue to connect devices, machines, and processes, creating a highly interconnected and intelligent industrial ecosystem.

In conclusion, Digital transformation is reshaping the industrial sector, empowering businesses with new capabilities and driving innovation. From enhanced operational efficiency to improved product quality and supply chain management, the benefits of digital transformation are vast. However, organizations must address challenges such as data security, legacy system integration, and workforce adaptability. By embracing digital technologies and fostering a culture of innovation, industrial companies can unlock new opportunities, remain competitive, and thrive in the digital age. The future of the industrial sector lies in leveraging the power of digital transformation to create smarter, more efficient, and sustainable operations.

7.2.3 Current research impact on Digital Transformation

The research on hybrid models for predictive maintenance is a significant contributor to the ongoing digital transformation in various industries. Here's how it contributes to this transformation:

Improved Data Utilization: The hybrid models leverage advanced machine learning techniques to process and analyze large volumes of data from multiple sources, including sensor data, CSV files, and maintenance logs. By efficiently handling and extracting insights from this data, the models enable companies to make informed decisions and take proactive actions to maintain their equipment and machinery.

Real-time Monitoring: The hybrid models facilitate real-time monitoring of equipment performance and health. By continuously analyzing data streams and identifying anomalies or patterns indicative of potential faults or failures, these models enable timely intervention. This proactive approach helps in preventing costly breakdowns, optimizing maintenance schedules, and minimizing downtime.

Enhanced Predictive Accuracy: The integration of multiple algorithms in hybrid models improves the accuracy and reliability of predictive maintenance. By combining the strengths of different models, these hybrid approaches can handle complex data patterns, capture subtle variations, and provide more

accurate predictions. This capability enables companies to detect issues early on, prioritize maintenance tasks, and allocate resources effectively.

Optimization of Maintenance Resources: With hybrid models, companies can optimize their maintenance resources by focusing on critical assets and prioritizing maintenance activities based on predicted failure probabilities. This targeted approach reduces unnecessary maintenance costs and minimizes disruptions to operations. It allows for more efficient use of resources, such as labor, spare parts, and equipment downtime.

Integration with Digital Infrastructure: The deployment of hybrid models for predictive maintenance aligns with the broader digital infrastructure of an organization. It can be seamlessly integrated with other digital systems, such as asset management systems, IoT platforms, and data analytics tools. This integration enables a cohesive ecosystem where data is shared, analyzed, and utilized to drive intelligent decision-making across the organization.

Overall, the adoption of hybrid models for predictive maintenance contributes to the digital transformation by leveraging advanced analytics, real-time monitoring, and data-driven decision-making. It enables companies to transition from reactive and manual maintenance practices to proactive, automated, and intelligent approaches. By embracing these technologies, organizations can achieve higher operational efficiency, reduce costs, enhance equipment reliability, and ultimately drive overall business growth in the digital era.

7.2.4 Digital Transformations for this research

The research objectives outlined in this research are closely associated with digital transformations for industrial machines. Here's how each objective aligns with the broader context of digital transformation in the industrial sector:

Machine Learning for Predictive Maintenance: One of the key aspects of digital transformation in industrial settings is the adoption of predictive maintenance strategies. By analyzing different machine learning algorithms like Support Vector Machine, Decision Tree, Random Forest, K-means, and Linear Regression, you are essentially exploring the feasibility and effectiveness of these algorithms in predicting and preventing equipment failures. Predictive maintenance, enabled by machine learning, is a significant part of digital transformation as it helps industries move from reactive to proactive maintenance, reducing downtime and improving overall efficiency.

Data-Driven Decision-Making: Leveraging various ML algorithms for predictive maintenance involves collecting and analyzing large volumes of sensor data from industrial machines. This process of data collection, analysis, and algorithm evaluation contributes to the digital transformation by enabling data-driven decision-making. Organizations can make more informed choices based on the insights derived from these algorithms, optimizing operations, and resource allocation.

Enhanced Anomaly Detection: Developing a hybrid ML model for detecting anomalies in different scenarios is another critical component of digital transformation. It allows industries to enhance their ability to detect and respond to abnormal conditions or events in real-time. The use of hybrid models often combines the strengths of various algorithms to improve accuracy and adaptability.

IoT Integration: In many cases, anomaly detection in industrial settings is closely tied to the integration of the Internet of Things (IoT). Industrial machines and sensors are equipped with IoT devices that continuously collect data. Your research objective aligns with digital transformation efforts to harness this IoT-generated data for proactive anomaly detection.

Optimizing Operations: Accurate anomaly detection not only prevents unexpected downtime but also contributes to optimizing operations and resource allocation. It ensures that maintenance activities are performed only when necessary, reducing costs and improving efficiency—a fundamental aspect of digital transformation.

Overall, your research objectives are highly relevant to digital transformations for industrial machines, as they encompass the adoption of advanced analytics, machine learning, and IoT technologies to shift from traditional, reactive maintenance approaches to proactive, data-driven strategies. These

transformations aim to enhance operational efficiency, reduce costs, and improve overall competitiveness in industrial sectors.

7.3 Future recommendation

Based on the results of the hybrid models, there are several promising avenues for future research and projects to further improve fault and failure detection in the context of smart predictive maintenance. Here are some potential areas of focus:

Fine-tuning and Optimization: Despite achieving high accuracy, recall, precision, and F1-score, there is always room for fine-tuning and optimization of the hybrid models. Researchers can explore different hyperparameter tuning techniques, such as grid search, random search, or Bayesian optimization, to find the optimal combination of parameters that maximize the performance of the models.

Feature Engineering: Feature engineering plays a crucial role in the performance of predictive models. Future research can investigate the inclusion of additional features or the transformation of existing ones to better capture the underlying patterns and characteristics of machine processes. Collaboration with domain experts can provide valuable insights to identify relevant features that contribute to improved fault and failure detection.

Ensemble Techniques: Ensemble techniques, such as model stacking, blending, or boosting, offer the opportunity to merge predictions from multiple hybrid models. These techniques leverage the individual strengths of different models to potentially improve fault and failure detection performance and enhance the system's robustness. By combining the predictions of multiple models, ensemble techniques aim to achieve better overall results and mitigate the limitations of individual models.

Explainability and Interpretability: Enhancing the explainability and interpretability of the hybrid models is crucial for gaining insights and building trust in their predictions.

Real-time Implementation and Integration: While the hybrid models show promising results, their practical implementation and integration in real-time industrial environments remain a significant challenge. Future projects can focus on deploying the models in live production settings, integrating them with existing monitoring systems, and assessing their performance and scalability in real-time fault and failure detection scenarios.

Edge Computing and Edge Analytics: Edge computing, which involves processing data closer to the data source, can be explored to enable real-time fault and failure detection at the edge of the network. By leveraging edge analytics capabilities, the hybrid models can be deployed directly on edge devices or gateways, allowing for faster response times and reduced reliance on cloud infrastructure.

Continuous Model Learning: Continuous learning and model adaptation are crucial in dynamic industrial environments where machine processes and conditions may change over time. Future research can focus on developing methodologies that allow the hybrid models to continuously learn from new data, adapt their predictive capabilities, and adjust their thresholds for fault and failure detection accordingly.

Integration of Domain Knowledge: Collaboration between data scientists and domain experts is essential for effective fault and failure detection. Future projects can focus on incorporating domain knowledge, such as engineering principles, physics-based models, or expert rules, into the hybrid models to enhance their accuracy and make them more context-specific for different industries and applications.

By addressing these research areas and undertaking projects that focus on fine-tuning, feature engineering, ensemble techniques, explainability, real-time implementation, edge computing, continuous learning, and integration of domain knowledge, the field of fault and failure detection in smart predictive maintenance can be advanced. These efforts will contribute to the development of more

accurate, robust, and efficient systems for detecting and mitigating faults and failures in industrial processes, leading to improved reliability, reduced downtime, and optimized maintenance practices.

7.4 Conclusion

In conclusion, the application of Smart Predictive Maintenance in pro-active machine processes using Machine Learning, coupled with digital transformation, offers significant potential for enhancing industrial operations. By harnessing the power of advanced analytics, machine learning algorithms, and real-time data processing, organizations can achieve proactive maintenance strategies that minimize downtime, reduce costs, and optimize overall productivity.

The results of hybrid models, such as RF-GB and RF-MLP-GB, demonstrate the effectiveness of combining multiple algorithms to improve fault and failure detection. These models exhibit high accuracy, recall, precision, F1-score, and low MSE, indicating their ability to accurately predict and classify machine faults. Additionally, the high ROC AUC scores reflect the models' strong performance in distinguishing between normal and abnormal machine behaviour.

The integration of digital transformation enables the seamless flow of data from sensors, devices, and systems, providing a comprehensive view of machine processes. This data-driven approach empowers organizations to make informed decisions based on real-time insights, facilitating timely maintenance actions and mitigating potential failures. Furthermore, digital transformation enables the implementation of predictive analytics models at the edge, enabling faster response times and reducing reliance on centralized infrastructure.

To further enhance fault and failure detection, future research and projects can focus on fine-tuning and optimizing the hybrid models, exploring advanced feature engineering techniques, leveraging ensemble techniques, improving explainability and interpretability, enabling real-time implementation and integration, considering edge computing and edge analytics, facilitating continuous model learning, and integrating domain knowledge.

By pursuing these avenues, organizations can unlock the full potential of Smart Predictive Maintenance and digital transformation in industrial settings. This synergy enables proactive maintenance practices, minimizes unplanned downtime, maximizes equipment reliability, optimizes maintenance schedules, and ultimately improves overall operational efficiency. Embracing Smart Predictive Maintenance with Machine Learning and digital transformation is a key step towards realizing the vision of Industry 4.0 and advancing industrial processes into the era of intelligent and connected systems.

The research on hybrid models for predictive maintenance has a significant impact on the future of maintenance practices in various industries. The hybrid models, such as the RF-GB hybrid model and the RF-MLP-GB hybrid model, demonstrate superior performance in accurately predicting faults and failures in machines and equipment.

The use of hybrid models combines the strengths of different algorithms, leveraging their individual capabilities to improve overall predictive accuracy and reliability. By integrating multiple models, the hybrid approach can overcome limitations and biases inherent in individual algorithms, leading to more robust and effective predictive maintenance systems.

The adoption of hybrid models in predictive maintenance enables proactive and timely identification of potential faults and failures. This allows maintenance teams to intervene before the occurrence of critical breakdowns, reducing unplanned downtime, and minimizing disruptions to production processes. By detecting and addressing issues at an early stage, companies can optimize maintenance schedules, reduce repair costs, and increase overall operational efficiency.

Furthermore, hybrid models can handle complex and multidimensional data, incorporating various data sources such as sensor data, CSV files, and maintenance logs. This comprehensive approach enhances

the accuracy and reliability of fault detection, enabling better decision-making and resource allocation in maintenance activities.

Overall, the research on hybrid models for predictive maintenance paves the way for more advanced and intelligent maintenance strategies. It empowers industries to move from reactive maintenance practices to proactive and data-driven approaches. The integration of machine learning and hybrid models enables better resource utilization, improved equipment reliability, and increased productivity. As a result, companies can achieve higher levels of operational efficiency, reduce maintenance costs, and enhance customer satisfaction through uninterrupted production processes.

References

1. Bousdekis, A., D. Apostolou, and G. Mentzas, *Predictive Maintenance in the 4th Industrial Revolution: Benefits, Business Opportunities, and Managerial Implications.* IEEE engineering management review, 2020. **48**(1): p. 57-62.
2. Borissova, D.I., I.C. Mustakerov, and L.A. Doukovska, *Predictive maintenance sensors placement by combinatorial optimization.* International Journal of Electronics and Telecommunications, 2012. **58**: p. 153-158.
3. Gröger, C., *Building an Industry 4.0 Analytics Platform.* Datenbank-Spektrum, 2018. **18**(1): p. 5-14.
4. Orhan, S., N. Aktürk, and V. Çelik, *Vibration monitoring for defect diagnosis of rolling element bearings as a predictive maintenance tool: Comprehensive case studies.* NDT & E International, 2006. **39**(4): p. 293-298.
5. Duggal, N. *Top 10 Deep Learning Algorithms You Should Know in (2020)*. 2021; Available from: https://www.simplilearn.com/tutorials/deep-learning-tutorial/deep-learning-algorithm.
6. Mathew, V., et al. *Prediction of Remaining Useful Lifetime (RUL) of turbofan engine using machine learning*. 2017. IEEE.
7. Selcuk, S., *Predictive maintenance, its implementation and latest trends.* Proceedings of the Institution of Mechanical Engineers, Part B: Journal of Engineering Manufacture, 2017. **231**(9): p. 1670-1679.
8. Singh, R., et al., *Overall Equipment Effectiveness (OEE) Calculation - Automation through Hardware & Software Development.* Procedia engineering, 2013. **51**: p. 579-584.
9. Vorne. *Calculate OEE Definitions, Formulas, and Examples | OEE*. 2021; Available from: https://www.oee.com/calculating-oee.html.
10. upkeep.com. *Predictive Maintenance*. 2022 [cited 2022; A brief History of PdM]. Available from: https://www.upkeep.com/learning/predictive-maintenance.
11. Cheng, C., B.-K. Zhang, and D. Gao. *A Predictive Maintenance Solution for Bearing Production Line Based on Edge-Cloud Cooperation*. 2019. IEEE.
12. O'Brien, L., *KPI & metrics survey highlights industry issues-ISA.* International Society of Automation, 2021.
13. Patil, R., et al., *Predictive modeling for corrective maintenance of imaging devices from machine logs*. Vol. 2017. 2017.
14. Susto, G.A., et al. *An adaptive machine learning decision system for flexible predictive maintenance*. in *2014 IEEE International Conference on Automation Science and Engineering (CASE)*. 2014.
15. Chuang, S.-Y., et al., *Predictive Maintenance with Sensor Data Analytics on a Raspberry Pi-Based Experimental Platform.* Sensors (Basel), 2019. **19**(18): p. 3884.
16. Candanedo, I.S., et al., *Machine Learning Predictive Model for Industry 4.0*. 2018, Springer International Publishing. p. 501-510.
17. Butte, S., A. Prashanth, and S. Patil. *Machine learning based predictive maintenance strategy: a super learning approach with deep neural networks*. in *2018 IEEE Workshop on Microelectronics and Electron Devices (WMED)*. 2018. IEEE.
18. Qiu, H., et al., *Wavelet filter-based weak signature detection method and its application on rolling element bearing prognostics.* Journal of Sound and Vibration, 2006. **289**(4-5): p. 1066-1090.
19. Sapankevych, N.I. and R. Sankar, *Time Series Prediction Using Support Vector Machines: A Survey.* IEEE Computational Intelligence Magazine, 2009. **4**(2): p. 24-38.
20. Kusiak, A., *Smart manufacturing.* International Journal of Production Research, 2018. **56**(1-2): p. 508-517.
21. Daniels, J., et al., *The Internet of Things, Artificial Intelligence, Blockchain, and Professionalism.* IT Professional, 2018. **20**(6): p. 15-19.
22. O'Carroll, B. *What are the 3 types of AI? A guide to narrow, general, and super artificial intelligence*. 24 October; Available from: https://codebots.com/artificial-intelligence/the-3-types-of-ai-is-the-third-even-possible.
23. Baum, S.D., A.M. Barrett, and R.V. Yampolskiy, *Modeling and Interpreting Expert Disagreement About Artificial Superintelligence.(Report).* Informatica, 2017. **41**(4): p. 419.

24. Nicolas, M. and H. Cyrus, *The Third Age of Artificial Intelligence.* Field actions science reports, 2017: p. 6-11.
25. Turing, A.M., *Computing machinery and intelligence.* 1950.
26. McCarthy, J., et al., *A proposal for the dartmouth summer research project on artificial intelligence, august 31, 1955.* AI magazine, 2006. **27**(4): p. 12-12.
27. Mahesh, B., *Machine learning algorithms-a review.* International Journal of Science and Research (IJSR).[Internet], 2020. **9**: p. 381-386.
28. Noble, W.S., *What is a support vector machine?* Nature Biotechnology, 2006. **24**(12): p. 1565-1567.
29. Alzubi, J., A. Nayyar, and A. Kumar, *Machine Learning from Theory to Algorithms: AnOverview*. 2018.
30. Valipour, M., *Long-term runoff study using SARIMA and ARIMA models in the United States.* Meteorological Applications, 2015. **22**(3): p. 592-598.
31. Ester, M., et al. *A density-based algorithm for discovering clusters in large spatial databases with noise*. in *kdd*. 1996.
32. Agrawal, R. and R. Srikant. *Fast algorithms for mining association rules*. in *Proc. 20th int. conf. very large data bases, VLDB*. 1994. Santiago, Chile.
33. Yang, X., et al., *Research and applications of artificial neural network in pavement engineering: A state-of-the-art review.* Journal of Traffic and Transportation Engineering (English Edition), 2021. **8**(6): p. 1000-1021.
34. Chua, L.O. and T. Roska, *The CNN paradigm.* IEEE Transactions on Circuits and Systems I: Fundamental Theory and Applications, 1993. **40**(3): p. 147-156.
35. Montavon, G., W. Samek, and K.-R. Müller, *Methods for interpreting and understanding deep neural networks.* Digital Signal Processing, 2018. **73**: p. 1-15.
36. Susto, G.A., et al. *Prediction of integral type failures in semiconductor manufacturing through classification methods*. 2013. IEEE.
37. Li, H., et al., *Improving rail network velocity: A machine learning approach to predictive maintenance.* Transportation Research Part C: Emerging Technologies, 2014. **45**: p. 17-26.
38. Praveenkumar, T., et al., *Fault Diagnosis of Automobile Gearbox Based on Machine Learning Techniques.* Procedia Engineering, 2014. **97**: p. 2092-2098.
39. Abu-Samah, A., et al., *Failure Prediction Methodology for Improved Proactive Maintenance using Bayesian Approach ★ ★The authors gratefully acknowledge STMicroelectronics for their support and provision of data for TT case study. The authors also acknowledge European project INT.* IFAC-PapersOnLine, 2015. **48**(21): p. 844-851.
40. Prytz, R., et al., *Predicting the need for vehicle compressor repairs using maintenance records and logged vehicle data.* Engineering Applications of Artificial Intelligence, 2015. **41**: p. 139-150.
41. Susto, G.A., et al., *Machine Learning for Predictive Maintenance: A Multiple Classifier Approach.* IEEE Transactions on Industrial Informatics, 2015. **11**(3): p. 812-820.
42. Durbhaka, G.K. and B. Selvaraj. *Predictive maintenance for wind turbine diagnostics using vibration signal analysis based on collaborative recommendation approach*. in *2016 International Conference on Advances in Computing, Communications and Informatics (ICACCI)*. 2016.
43. Susto, G.A. and A. Beghi. *Dealing with time-series data in Predictive Maintenance problems*. 2016. IEEE.
44. Aydin, O. and S. Guldamlasioglu. *Using LSTM networks to predict engine condition on large scale data processing framework*. 2017. IEEE.
45. Canizo, M., et al. *Real-time predictive maintenance for wind turbines using Big Data frameworks*. 2017. IEEE.
46. Eke, S., et al. *Characterization of the operating periods of a power transformer by clustering the dissolved gas data*. 2017. IEEE.
47. Kumar, A., R. Shankar, and L.S. Thakur, *A big data driven sustainable manufacturing framework for condition-based maintenance prediction.* Journal of Computational Science, 2018. **27**: p. 428-439.

48. Uhlmann, E., et al., *Cluster identification of sensor data for predictive maintenance in a Selective Laser Melting machine tool.* Procedia Manufacturing, 2018. **24**: p. 60-65.
49. Amruthnath, N. and T. Gupta. *A research study on unsupervised machine learning algorithms for early fault detection in predictive maintenance*. 2018. IEEE.
50. Kulkarni, K., et al. *Predictive Maintenance for Supermarket Refrigeration Systems Using Only Case Temperature Data*. 2018. IEEE.
51. Zenisek, J., F. Holzinger, and M. Affenzeller, *Machine learning based concept drift detection for predictive maintenance.* Computers & Industrial Engineering, 2019. **137**: p. 106031.
52. Bruneo, D. and F. De Vita. *On the Use of LSTM Networks for Predictive Maintenance in Smart Industries*. 2019. IEEE.
53. Hsu, J.-Y., et al., *Wind Turbine Fault Diagnosis and Predictive Maintenance Through Statistical Process Control and Machine Learning.* IEEE Access, 2020. **8**: p. 23427-23439.
54. Ayvaz, S. and K. Alpay, *Predictive maintenance system for production lines in manufacturing: A machine learning approach using IoT data in real-time.* Expert Systems with Applications, 2021. **173**: p. 114598.
55. Cai, B., et al., *Data-driven early fault diagnostic methodology of permanent magnet synchronous motor.* Expert Systems with Applications, 2021. **177**: p. 115000.
56. Graney, B.P. and K. Starry, *Rolling element bearing analysis.* Materials Evaluation, 2012. **70**(1): p. 78.
57. Python.org. *About Python Apps*. 2023; Available from: https://www.python.org/about/apps/.
58. Documentation, A. *Using Jupyter Notebook*. Available from: https://docs.anaconda.com/ae-notebooks/user-guide/basic-tasks/apps/jupyter/index.html.
59. Ding, V.N.G.A.A.J., *Data Quality Considerations for Big Data and Machine Learning: Going Beyond Data Cleaning and Transformations.* International Journal on Advances in Software, 2017. **10.1**: p. 1-20.
60. Levine, M.D., *Feature extraction: A survey.* Proceedings of the IEEE, 1969. **57**(8): p. 1391-1407.
61. Matzka, S. *Explainable Artificial Intelligence for Predictive Maintenance Applications*. in *2020 Third International Conference on Artificial Intelligence for Industries (AI4I)*. 2020.